

Tecnológico de Monterrey

Aplicación de Métodos Multivariados en Ciencia de Datos

Módulo 1: Relaciones de Interdependencia

Raúl Gómez Muñoz
Blanca Rosa Ruiz Hernández

Tabla de contenidos

I Fundamentos de análisis univariado

1. Estructura y organización de datos **1**
1.1. Datos ordenados . . . 1
1.1.1. Tibbles y pipes . . . 1
1.1.2. Ejemplo: Ordenando el conjunto de datos `who` . . . 2
1.2. La Gramática de los Gráficos . . . 6
1.2.1. Capas en ggplot2 . . . 7
1.2.2. Graficación con ggplot2 . . . 8
1.3. Actividad: Limpieza de datos . . . 9

2. Evaluación de normalidad univariada **11**
2.1. Los momentos estandarizados . . . 11
2.1.1. Asimetría . . . 12
2.1.2. Curtosis . . . 18
2.1.3. Asimetría y curtosis muestrales . . . 24
2.2. Gráficos QQ . . . 36
2.3. Pruebas de normalidad . . . 41

3. Escalamiento y transformaciones **47**
3.1. Escalamiento de variables aleatorias . . . 47
3.2. Escalamiento de muestras . . . 52
3.3. Transformaciones . . . 56
3.3.1. Familias de transformaciones . . . 56
3.3.2. Perfil de log-verosimilitud y transformación óptima . . . 60
3.4. Actividad: Visualizando la no normalidad de muestras . . . 66

II Introducción al análisis multivariado **67**

4. Introducción al análisis multivariante **69**
4.1. El vector de medias y la matriz de covarianzas . . . 69
4.2. La distancia de Mahalanobis . . . 71
4.3. Escalamiento y estandarización . . . 77
4.4. Actividad: La geometría de los datos multivariantes . . . 77

5. Distribución normal multivariante **79**
5.1. Definición y propiedades básicas . . . 79
5.2. Contornos de la normal multivariante . . . 80
5.3. Estandarización de la normal multivariante . . . 83
5.4. Interpretación Geométrica de la SVD . . . 83
5.5. Clausura bajo transformaciones lineales . . . 86
5.6. Pruebas de normalidad multivariante . . . 87

6. Análisis de componentes principales **89**
6.1. Entendiendo la varianza total . . . 89
6.2. Varianza acumulada y gráficos scree . . . 94
6.3. Cargas (loadings) y biplots . . . 98
6.4. Resumen del proceso de PCA . . . 102

III Análisis factorial **105**

7. Análisis factorial: modelos teóricos **107**
7.1. Planteamiento del análisis factorial . . . 107
7.2. Descomposición de varianza . . . 108
7.3. Estimación de puntuaciones factoriales . . . 109
7.4. Rotación de factores . . . 109

8. Análisis factorial: métodos prácticos **111**
8.1. Evaluación de la factorizabilidad . . . 111
8.2. Decidir el número de factores . . . 112
8.3. Extracción de factores . . . 112
8.4. Rotación de factores . . . 112
8.5. Cálculo de puntuaciones factoriales . . . 113
8.6. Evaluación del ajuste global . . . 113
8.7. Visualización del modelo . . . 113
8.8. Ejemplo: análisis factorial del conjunto Holzinger-Swineford . . . 114
8.8.1. Evaluar la factorizabilidad . . . 114
8.8.2. Decidir el número de factores . . . 116
8.8.3. Extracción, rotación y puntuación de factores . . . 118
8.8.4. Evaluación del ajuste del modelo . . . 119
8.8.5. Visualización del modelo . . . 120

IV Análisis de conglomerados **125**

9. Análisis de conglomerados jerárquicos **127**
9.1. Entendiendo los dendrogramas . . . 127
9.2. Realización del agrupamiento jerárquico . . . 133

10. Agrupamiento no jerárquico **135**
10.1. Métodos de particionado . . . 135

10.2. Métodos basados en densidad 135
10.3. Métodos basados en modelos 136
10.4. Elección del método adecuado 136
10.5. Evaluación de los resultados de agrupamiento 137
10.6. Ejemplo en R 137

Parte I

Fundamentos de análisis univariado

Capítulo 1

Estructura y organización de datos

Todo proyecto de ciencia de datos debe comenzar con la adquisición y organización de los conjuntos de datos requeridos. La forma en la que esta información se estructura y organiza puede tener un impacto significativo en la calidad del análisis que se desea desarrollar. En esta sección introduciremos los conceptos fundamentales del Tidyverse, un conjunto de herramientas en R que facilita la manipulación y visualización de nuestros datos siguiendo principios específicos de estructura y organización [5, 6, 7].

1.1. Datos ordenados

La idea central detrás del diseño del Tidyverse es la definición de datos ordenados [3]:

Definición 1.1.1. *Datos ordenados* es una forma de estructurar un conjunto de datos para facilitar su uso. En datos ordenados:

1. Cada variable forma una columna.
2. Cada observación forma una fila.
3. Cada tipo de unidad observacional forma una tabla.

Esta estructura facilita la manipulación, análisis y visualización de nuestra información, ya que se alinea con la forma en la que los datos se procesan típicamente en R.

1.1.1. Tibbles y pipes

En el Tidyverse, los datos suelen representarse mediante un tipo especial de data frame llamado *tibble*. Los tibbles son más amigables que los data frames tradicionales, ya que ofrecen una mejor impresión y manejo de subconjuntos de datos. Además, están diseñados para trabajar de manera fluida con el operador pipe (ya sea `|>` o bien `%>%`), que permite encadenar múltiples operaciones (conocidas como "verbos" en el contexto del Tidyverse) de manera clara y concisa. Para ilustrar estas ideas, en la siguiente sección realizaremos la limpieza del conjunto de datos `who` de la OMS, lo que nos permitirá generar un tibble que contenga esta información y que cumpla con los principios de datos ordenados. Este suele ser el primer paso en el Análisis Exploratorio de Datos (EDA) y es crucial para garantizar que los datos estén en un formato adecuado para su análisis y visualización.

1.1.2. Ejemplo: Ordenando el conjunto de datos `who`

Antes de comenzar, debemos cargar las librerías necesarias y el conjunto de datos `who`:

```
library(tidyverse)
library(here)
library(krulRutils)
library(magrittr)
library(kableExtra)

options(scipen = 999)
data(who)
```

Ahora podemos iniciar nuestro proceso de ordenamiento. El problema principal de este conjunto es que contiene variables (como "new_sp_m014") que no son muy claras y que incluyen más de una unidad de información.

```
who |>
  glimpse()
```

```
Rows: 7,240
Columns: 60
$ country      <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan~
$ iso2         <chr> "AF", "AF", "AF", "AF", "AF", "AF", "AF", "AF", "AF", "AF~
$ iso3         <chr> "AFG", "AFG", "AFG", "AFG", "AFG", "AFG", "AFG", "AFG", "~
$ year         <dbl> 1980, 1981, 1982, 1983, 1984, 1985, 1986, 1987, 1988, 198~
$ new_sp_m014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_m1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_m2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_m3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_m4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_m5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_m65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_f014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_f1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_f2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_f3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_f4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_f5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_f65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_m014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_m1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_m2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_m3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_m4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_m5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_m65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_f014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
```

```
$ new_sn_f1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_f2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_f3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_f4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_f5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_f65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_m014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_m1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_m2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_m3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_m4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_m5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_m65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_f014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_f1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_f2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_f3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_f4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_f5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_f65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_m014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_m1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_m2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_m3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_m4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_m5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_m65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_f014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_f1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_f2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_f3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_f4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_f5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_f65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
```

Es por eso que necesitamos separar estas variables e introducir nombres más claros para ellas y para sus valores asociados.

```
# Empezamos creando la variable "who_tidy"
# que contiene el conjunto de datos de la OMS ya limpio
who_tidy <- who |>
  # Usamos pivot_longer para eliminar las variables que empiezan con "new"
  # y usarlas como valores en su lugar
  pivot_longer(
    cols = starts_with("new"),
    names_to = "key",
    values_to = "cases",
```

```
    values_drop_na = TRUE
  ) |>
  mutate(
    key = if_else(
      startsWith(key, "newrel"),
      sub("newrel", "new_rel", key),
      key
    ),
    cases = as.integer(cases)
  ) |>
  separate(key, into = c("new", "type", "sexage"), sep = "_") |>
  separate(sexage, into = c("sex", "age"), sep = 1) |>
  select(-new, -iso2, -iso3) %T>%
  # Finalmente, guardamos los datos ordenados en formato RDS y CSV
  saveRDS(here("data", "who_tidy.rds")) |>
  write_csv(here("data", "who_tidy.csv")) |>
  glimpse()
```

```
Rows: 76,046
Columns: 6
$ country <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", "A~
$ year    <dbl> 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 19~
$ type    <chr> "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "s~
$ sex     <chr> "m", "m", "m", "m", "m", "m", "m", "f", "f", "f", "f", "f", "f~
$ age     <chr> "014", "1524", "2534", "3544", "4554", "5564", "65", "014", "1~
$ cases   <int> 0, 10, 6, 3, 5, 2, 0, 5, 38, 36, 14, 8, 0, 1, 30, 129, 128, 90~
```

Ahora queremos convertir las columnas “type”, “sex” y “age” en factores. Empezamos construyendo las siguientes tablas de correspondencia:

```
who_type_lookup_tbl <- tibble(
  code = c("ep", "rel", "sn", "sp"),
  label = c(
    "TB extrapulmonar",
    "Caso de recaída",
    "TB pulmonar baciloscopía negativa",
    "TB pulmonar baciloscopía positiva"
  )
)

who_sex_lookup_tbl <- tibble(
  code = c("f", "m"),
  label = c("Mujer", "Hombre")
)

who_age_lookup_tbl <- tibble(
```

```
  code = c(
    "014",
    "1524",
    "2534",
    "3544",
    "4554",
    "5564",
    "65"
  ),
  label = c(
    "0-14",
    "15-24",
    "25-34",
    "35-44",
    "45-54",
    "55-64",
    "65+"
  )
)
```

Utilizando estas tablas, podemos agregar ahora estas columnas a nuestro conjunto de datos:

```
# Agregamos las columnas de factores como información adicional
who_factor <- who_tidy |>
  convert_codes_to_factor(
    code_col = type,
    lookup_tbl = who_type_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = Tipo
  ) |>
  convert_codes_to_factor(
    code_col = sex,
    lookup_tbl = who_sex_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = Sexo
  ) |>
  convert_codes_to_factor(
    code_col = age,
    lookup_tbl = who_age_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = Edad
  ) |>
  glimpse()
```

```
Rows: 76,046
Columns: 9
$ country <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", "A~
$ year    <dbl> 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 19~
$ type    <chr> "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "s~
$ sex     <chr> "m", "m", "m", "m", "m", "m", "m", "f", "f", "f", "f", "f", "f~
$ age     <chr> "014", "1524", "2534", "3544", "4554", "5564", "65", "014", "1~
$ cases   <int> 0, 10, 6, 3, 5, 2, 0, 5, 38, 36, 14, 8, 0, 1, 30, 129, 128, 90~
$ Tipo    <fct> TB pulmonar baciloscopía positiva, TB pulmonar baciloscopía po~
$ Sexo    <fct> Hombre, Hombre, Hombre, Hombre, Hombre, Hombre, Hombre, Mujer,~
$ Edad    <fct> 0-14, 15-24, 25-34, 35-44, 45-54, 55-64, 65+, 0-14, 15-24, 25-~
```

Finalmente, queremos usar esta información para analizar el número de casos de tuberculosis por tipo y sexo. Empezamos generando el tibble de frecuencias correspondiente:

```
who_type_sex_tbl <- who_factor |>
  count(Tipo, Sexo, wt = cases, name = "Casos")

kable(
  who_type_sex_tbl,
  booktabs = TRUE,
) |>
  kable_styling(latex_options = "hold_position")
```

Tabla 1.1: Número de casos de tuberculosis por tipo y sexo

Tipo	Sexo	Casos
TB extrapulmonar	Mujer	941880
TB extrapulmonar	Hombre	1044299
Caso de recaída	Mujer	1201596
Caso de recaída	Hombre	2018976
TB pulmonar baciloscopía negativa	Mujer	2439139
TB pulmonar baciloscopía negativa	Hombre	3840388
TB pulmonar baciloscopía positiva	Mujer	11324409
TB pulmonar baciloscopía positiva	Hombre	20586831

1.2. La Gramática de los Gráficos

El concepto de la *Gramática de los Gráficos* fue introducido por Leland Wilkinson en su libro "The Grammar of Graphics" [8], y proporciona un marco para entender cómo crear visualizaciones de forma sistemática ya que descompone el proceso de construir una gráfica en componentes como datos, estéticas, geometrías, estadísticas, coordenadas y temas. El paquete `ggplot2` [4] implementa esta gramática en R, permitiendo construir visualizaciones complejas por capas. Para más información sobre su historia y aplicaciones, consulte: [1, 2].

1.2.1. Capas en ggplot2

En `ggplot2`, las gráficas se construyen añadiendo múltiples *capas* que definen los componentes de la visualización. Cada capa corresponde a un aspecto específico de la gráfica y, en conjunto, forman la composición completa. Estas capas incluyen:

1. **Datos:** El conjunto a visualizar (usualmente un data frame o tibble). Es la fuente de información de la gráfica.
2. **Estéticas:** Mapeos que relacionan variables con propiedades visuales, por ejemplo:
   - Posición en los ejes x e y.
   - Color, relleno.
   - Tamaño, forma.
   - Transparencia.

   Estos mapeos definen *qué* datos se muestran y *cómo* se representan visualmente.
3. **Geometrías:** Objetos geométricos que muestran los datos; por ejemplo:
   - `geom_point()` para diagramas de dispersión.
   - `geom_line()` para gráficas de líneas.
   - `geom_col()` para barras.
   - `geom_histogram()` para histogramas.

   La geometría controla *cómo* se dibujan los datos.
4. **Transformaciones estadísticas:** Cálculos opcionales aplicados a los datos antes de graficar, como:
   - `stat_bin()` para agrupar en histogramas.
   - `stat_smooth()` para ajustar curvas suaves.

   Estos son pasos de preprocesamiento para la visualización.
5. **Escalas:** Definen cómo se traducen los valores a propiedades visuales; también controlan marcas de ejes, leyendas y guías.
6. **Coordenadas:** El sistema usado, como cartesiano (`coord_cartesian()`), polar (`coord_polar()`) o invertido (`coord_flip()`).
7. **Facetado:** Métodos para dividir los datos en subconjuntos y mostrar múltiples gráficas en una cuadrícula:
   - `facet_wrap()`
   - `facet_grid()`
8. **Etiquetas:** `labs()` agrega títulos, etiquetas de ejes y pies.
9. **Temas:** `theme()` controla la apariencia general (fuentes, fondo, rejillas).

Cada capa puede combinarse y personalizarse para construir gráficas complejas y elegantes. Esta *gramática por capas* invita a pensar en la gráfica como una composición de componentes independientes en lugar de un objeto monolítico.

1.2.2. Graficación con ggplot2

Ilustraremos estos conceptos graficando el número de casos de tuberculosis por tipo y sexo, usando el tibble de frecuencias que creamos antes.

```
# Generamos la gráfica usando ggplot2
# Cada capa corresponde a una componente de la gramática de los gráficos
who_type_sex_plot <- who_type_sex_tbl |>
  ggplot(aes(x = Tipo, y = Casos, fill = Sexo)) +
  geom_col(position = "dodge") +
  c_scale_fill("C rose", "C blue") +
  labs(
    title = "Casos de tuberculosis por tipo y sexo",
    x = "Tipo de tuberculosis",
    y = "Casos",
    fill = "Sexo"
  ) +
  theme_krul()

# Visualizamos la gráfica
who_type_sex_plot
```

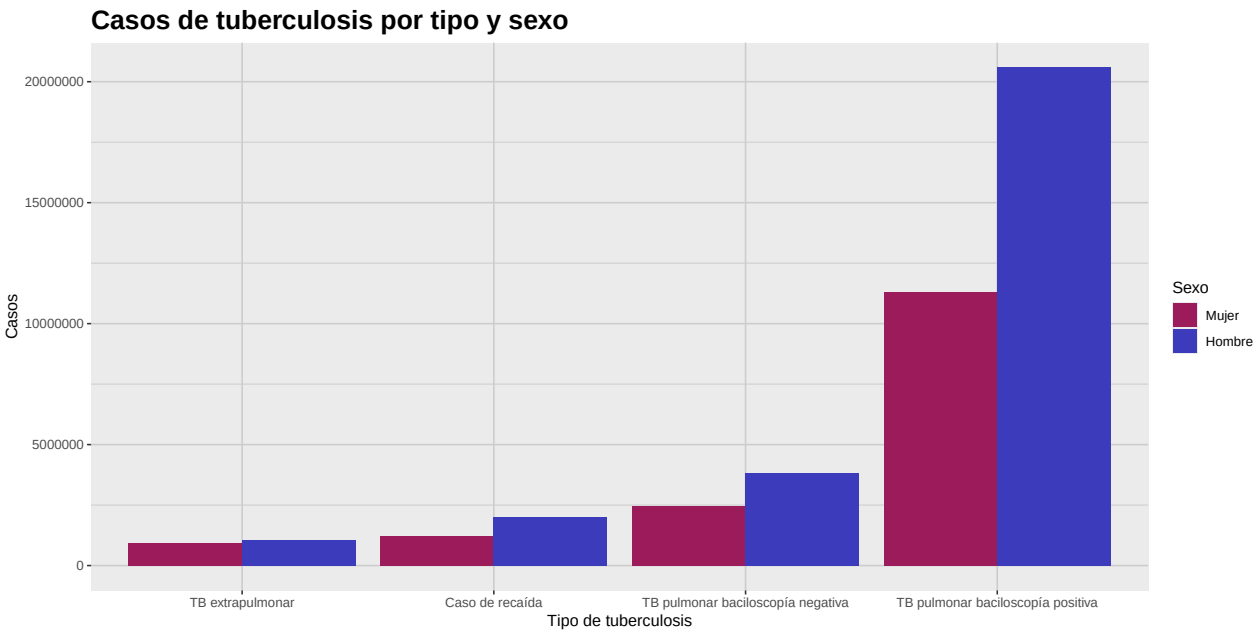


Figura 1.1: Esta gráfica muestra un análisis comparativo del número de casos de tuberculosis por tipo y sexo. Como se observa, el número de casos es consistentemente mayor en la población masculina en todos los tipos de tuberculosis.

1.3. Actividad: Limpieza de datos

1. En la sección anterior se mostró cómo usar el operador `|>` para encadenar múltiples operaciones en el Tidyverse. En este ejercicio, muestre el resultado de todos los pasos intermedios en el proceso de ordenamiento del conjunto de datos `who` usando el operador pipe.

2. El problema con el conjunto `relig_income`, es que los distintos niveles de ingreso están representados como columnas, lo que dificulta analizar los datos. Utilizando las técnicas vistas hasta ahora, ordene el conjunto para analizar la distribución del nivel de ingreso entre los distintos grupos religiosos en Estados Unidos, y genere una gráfica de barras que muestre esta información.

3. El problema con el conjunto `Billboard`, es que las semanas en que una canción apareció en la lista están representadas como columnas, lo que dificulta analizar los datos. Utilizando las técnicas vistas hasta ahora, ordene el conjunto para analizar la evolución del ranking de la canción "Who Let The Dogs Out" de "Baha Men" en el Billboard Hot 100. La visualización final debe lucir como la que se muestra abajo.

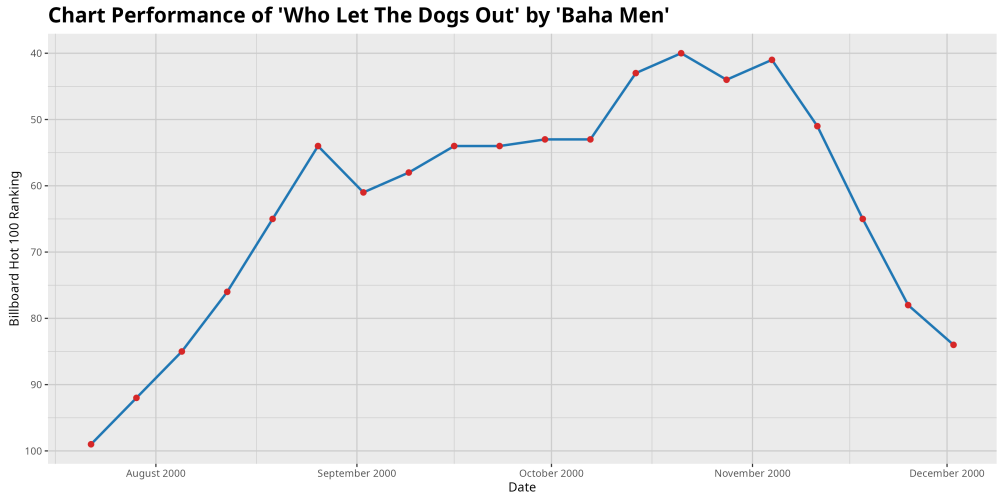


Figura 1.2: Evolución del ranking de la canción 'Who Let The Dogs Out' de 'Baha Men' en el Billboard Hot 100. La canción alcanzó el puesto 40 en la semana del 21 de octubre de 2000 y permaneció varias semanas en el top 100.

Bibliografía

[1] Cleveland, W. S. (1993) *The Elements of Graphing Data*. Hobart Press.

[2] Hyndman, R. J., and Athanasopoulos, G. (2021) *Forecasting: Principles and Practice* (3rd ed.). OTexts. https://otexts.com/fpp3/

[3] Wickham, H. (2014) Tidy data. *Journal of Statistical Software*, *59*(10), 1–23. https://doi.org/10.18637/jss.v059.i10

[4] Wickham, H. (2016) *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag.

[5] Wickham, H., and Grolemund, G. (2017) *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. O'Reilly Media.

[6] Wickham, H. (2019) Functional programming with purrr. *UseR! Conference*.

[7] Wickham, H., et al. (2019) Welcome to the tidyverse. *Journal of Open Source Software*, *4*(43), 1686. https://doi.org/10.21105/joss.01686

[8] Wilkinson, L. (1999) *The Grammar of Graphics*. Springer-Verlag.

Capítulo 2

Evaluación de normalidad univariada

En muchos métodos estadísticos univariados se asume que los datos siguen una distribución normal. Sin embargo, en la práctica, no todos nuestros conjuntos de datos satisfacen esta suposición. En estos casos no se justifica el uso de estos métodos y su uso indiscriminado puede llevar a conclusiones erróneas, que nos podrían llevar a tomar decisiones equivocadas bajo la ilusión de un respaldo estadístico que no existe. Es por eso que resulta fundamental contar con herramientas que nos permitan evaluar la normalidad de nuestros datos antes de aplicar cualquier método estadístico que dependa de esta suposición.

En este capítulo exploraremos algunas de estas herramientas, incluyendo las medidas de asimetría y curtosis, los gráficos QQ y algunas pruebas estadísticas para evaluar la normalidad. En el siguiente capítulo describiremos ciertas transformaciones que podemos aplicar a nuestros datos para hacerlos más cercanos a una distribución normal, en caso de que no lo sean. Pero antes de comenzar, cargaremos las librerías necesarias y definiremos algunas opciones globales para nuestro análisis.

```
library(tidyverse)
library(patchwork)
library(krulRutils)
library(e1071)
library(latex2exp)
library(greybox)
library(kableExtra)
library(car)
options(scipen = 999)
```

2.1. Los momentos estandarizados

Como es bien sabido, cualquier distribución normal queda completamente determinada por su media y su varianza. Se sigue que todos sus momentos estandarizados de orden superior quedan fijos y se pueden utilizar para comparar otras distribuciones contra la normal. En particular, el tercer y cuarto momentos estandarizados, conocidos como asimetría y curtosis respectivamente, son de gran utilidad para describir la forma de una distribución.

2.1.1. Asimetría

Definición 2.1.1. La *asimetría* de una variable aleatoria X se define como el tercer momento estandarizado, dado por

$$\gamma_1 = \frac{\mu_3}{\sigma^3} = \frac{\mathbb{E}[(X-\mu)^3]}{\sigma^3},$$

donde $\mu = \mathbb{E}[X]$ es la media y $\sigma = \sqrt{\mathbb{E}[(X-\mu)^2]}$ es la desviación estándar de X.

La asimetría, como su nombre lo indica, mide la falta de simetría de una distribución. Una distribución es *positivamente asimétrica* (*sesgo a la derecha*) si posee una cola más larga a la derecha, y *negativamente asimétrica* (*sesgo a la izquierda*) si la cola más larga está a la izquierda. Una distribución perfectamente simétrica tiene asimetría cero.

Para ilustrar estas ideas, introduciremos la distribución Gamma.

Definición 2.1.2. Decimos que una variable aleatoria continua X sigue una *distribución Gamma* con parámetro de forma $\alpha > 0$ y parámetro de tasa $\beta > 0$, si su función de densidad de probabilidad (PDF) está dada por

$$f_X(x;\alpha,\beta) = \begin{cases} \dfrac{\beta^\alpha}{\Gamma(\alpha)}\, x^{\alpha-1}e^{-\beta x}, & x > 0, \\ 0, & x \leq 0, \end{cases} \tag{2.1}$$

donde $\Gamma(\alpha)$ denota a la función Gamma, definida como

$$\Gamma(\alpha) = \int_0^\infty t^{\alpha-1}e^{-t}\,dt. \tag{2.2}$$

En este caso escribimos

$$X \sim \text{Gamma}(\alpha,\beta).$$

Observación 2.1.3. La distribución Gamma se usa frecuentemente en procesos de Poisson para modelar tiempos de espera hasta la ocurrencia de ciertos eventos. Por ejemplo, puede modelar el tiempo que pasa hasta que una compañía de seguros reciba un cierto número de reclamaciones.

La siguiente tabla resume las características poblacionales más importantes de la distribución Gamma:

```
# Crear la tabla de características de la distribución Gamma
gamma_tbl <- tibble(
  `Característica` = c("Media", "Varianza", "Asimetría"),
  `Valor` = c("$\\alpha/\\beta$", "$\\alpha/\\beta^2$", "$2/\\sqrt{\\alpha}$")
)

# Formatear y mostrar la tabla
kable(
  gamma_tbl,
  format = "latex",
  booktabs = TRUE,
  escape = FALSE,
  align = c("l", "c")
)
```

Tabla 2.1: Características poblacionales de la distribución Gamma con parámetros α y β.

Característica	Valor
Media	α/β
Varianza	α/β^2
Asimetría	$2/\sqrt{\alpha}$

Notemos que la asimetría siempre es positiva y tiende a cero conforme α aumenta. En otras palabras, la distribución se vuelve más simétrica conforme va aumentando el valor de α. Por otro lado, aunque no existe una fórmula cerrada para la mediana de la distribución Gamma, podemos calcularla numéricamente en R usando la función `qgamma()`. La mediana de una distribución Gamma siempre es menor que la media, lo cual es consistente con la asimetría positiva de esta distribución.

Observación 2.1.4. Más adelante definiremos la *curtosis excesiva* de una variable aleatoria, pero por el momento mencionaremos que la curtosis excesiva de la distribución Gamma está dada por $6/\alpha$. Notemos que, al igual que la asimetría, la curtosis excesiva es siempre positiva y tiende a cero conforme α aumenta, indicando que la distribución se vuelve más parecida a la normal conforme aumenta el valor de este parámetro.

Mostraremos ahora cómo graficar la PDF de una familia de distribuciones Gamma con $\beta = 1$ y distintos valores de α.

```
# Asignamos los parámetros
alpha_values <- c(1.5, 2, 3, 4, 6, 8)
beta <- 1

# Creamos una secuencia de valores para X
x_data <- seq(0, 15, length.out = 500)

# Calculamos la PDF de la familia de distribuciones Gamma
gamma_distribution_family_tbl <- expand_grid(
  x_data = x_data,
  alpha = alpha_values
) |>
  mutate(
    density_data = dgamma(x_data, shape = alpha, rate = beta),
    alpha = factor(alpha)
  )

# Generamos la gráfica de la familia
gamma_distribution_plot <- gamma_distribution_family_tbl |>
  ggplot(aes(x = x_data, y = density_data, color = alpha)) +
  geom_line(linewidth = 1) +
  scale_color_manual(
    values = unname(c_pal(
      "C red", "C yellow", "C orange",
```

```
      "C green", "C blue", "C magenta"
    )),
    labels = c(
      TeX("$\\alpha$ = 1.5"), TeX("$\\alpha$ = 2"), TeX("$\\alpha$ = 3"),
      TeX("$\\alpha$ = 4"), TeX("$\\alpha$ = 6"), TeX("$\\alpha$ = 8")
    )
  ) +
  labs(
    title = "Familia de distribuciones Gamma con beta = 1",
    x = TeX("$X$"),
    y = "Densidad",
    color = "Parámetro de forma"
  ) +
  theme_krul()

# Mostramos la gráfica
gamma_distribution_plot
```

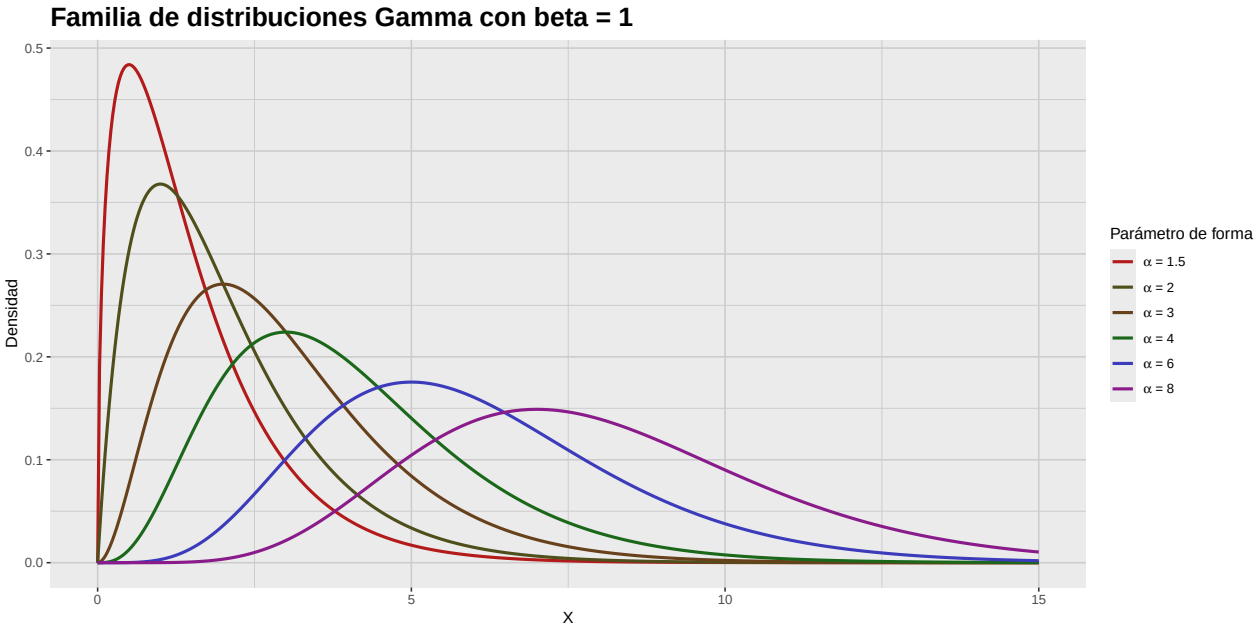


Figura 2.1: Funciones de densidad de una familia de distribuciones Gamma con $\beta = 1$ y distintos valores de α. Notemos cómo la forma de la distribución cambia conforme varía el parámetro de forma α, afectando la asimetría de la distribución. En particular, a medida que α aumenta, la distribución se vuelve más simétrica y el valor de la asimetría disminuye.

Finalmente, usaremos la distribución Gamma para generar una gráfica comparativa de una distribución con sesgo a la izquierda, una simétrica y una con sesgo a la derecha.

```r
# Fijamos un valor para alpha
alpha <- 2

# Calculamos la media, la mediana y la desviación estándar
gamma_mean <- alpha / beta
gamma_median <- qgamma(0.5, shape = alpha, rate = beta)
gamma_sd <- sqrt(alpha) / beta

# Creamos una secuencia de valores para X
x_data <- seq(0, 8, length.out = 1000)

# Calculamos la PDF de la distribución con sesgo negativo
left_skew_distribution_tbl <- tibble(
  x_data = -x_data,
  density_data = dgamma(-x_data, shape = alpha, rate = beta)
)

# Calculamos la PDF de la distribución simétrica
symmetric_distribution_tbl <- tibble(
  x_data = x_data - 4,
  density_data = dnorm(x_data)
)

# Calculamos la PDF de la distribución con sesgo positivo
right_skew_distribution_tbl <- tibble(
  x_data = x_data,
  density_data = dgamma(x_data, shape = alpha, rate = beta)
)

# Generamos la gráfica de la distribución con sesgo negativo
left_skew_distribution_plot <- left_skew_distribution_tbl |>
  ggplot(aes(x = x_data, y = density_data)) +
  geom_line(
    color = c_pal("C blue"),
    linewidth = 1
  ) +
  geom_vline(
    xintercept = -gamma_mean,
    color = c_pal("C red"),
    linetype = "dashed",
    linewidth = 1
  ) +
  geom_vline(
    xintercept = -gamma_median,
    color = c_pal("C green"),
```

```
    linetype = "dashed",
    linewidth = 1
  ) +
  annotate(
    "text",
    x = -gamma_mean,
    y = 0.5,
    label = "Media",
    color = c_pal("C red"),
    hjust = 1.1
  ) +
  annotate(
    "text",
    x = -gamma_median,
    y = 0.5,
    label = "Mediana",
    color = c_pal("C green"),
    hjust = -0.1
  ) +
  labs(
    title = "Sesgo a la izquierda",
    x = TeX("$X$"),
    y = "Densidad"
  ) +
  theme_krul()

# Generamos la gráfica de la distribución simétrica
symmetric_distribution_plot <- symmetric_distribution_tbl |>
  ggplot(aes(x = x_data, y = density_data)) +
  geom_line(
    color = c_pal("C blue"),
    linewidth = 1
  ) +
  geom_vline(
    xintercept = 0,
    color = c_pal("C red"),
    linetype = "dashed",
    linewidth = 1
  ) +
  geom_vline(
    xintercept = 0,
    color = c_pal("C green"),
    linetype = "dashed",
    linewidth = 1
  ) +
```

```
  annotate(
    "text",
    x = 0,
    y = 0.5,
    label = "Media y Mediana",
    color = c_pal("C red"),
    hjust = -0.1
  ) +
  labs(
    title = "Simétrica",
    x = TeX("$X$"),
    y = "Densidad"
  ) +
  theme_krul()

# Generamos la gráfica de la distribución con sesgo positivo
right_skew_distribution_plot <- right_skew_distribution_tbl |>
  ggplot(aes(x = x_data, y = density_data)) +
  geom_line(
    color = c_pal("C blue"),
    linewidth = 1
  ) +
  geom_vline(
    xintercept = gamma_mean,
    color = c_pal("C red"),
    linetype = "dashed",
    linewidth = 1
  ) +
  geom_vline(
    xintercept = gamma_median,
    color = c_pal("C green"),
    linetype = "dashed",
    linewidth = 1
  ) +
  annotate(
    "text",
    x = gamma_mean,
    y = 0.5,
    label = "Media",
    color = c_pal("C red"),
    hjust = -0.1
  ) +
  annotate(
    "text",
    x = gamma_median,
```

```
    y = 0.5,
    label = "Mediana",
    color = c_pal("C green"),
    hjust = 1.1
  ) +
  labs(
    title = "Sesgo a la derecha",
    x = TeX("$X$"),
    y = "Densidad"
  ) +
  theme_krul()

# Combinamos las gráficas
skew_comparison_plot <- left_skew_distribution_plot +
  symmetric_distribution_plot +
  right_skew_distribution_plot +
  plot_annotation(
    title = "Comparación de distribuciones con distintos niveles de asimetría",
    theme = theme(plot.title = element_text(size = 24, face = "bold"))
  )

# Mostramos la gráfica
skew_comparison_plot
```



Figura 2.2: Gráfica comparativa de distribuciones con sesgo a la izquierda, simétrica y con sesgo a la derecha. Notemos como la asimetría afecta la forma de la distribución y la relación entre la media y la mediana.

2.1.2. Curtosis

Definición 2.1.5. La *curtosis* de una variable aleatoria X se define como el cuarto momento estandarizado, dado por

$$\beta_2 = \frac{\mu_4}{\sigma^4} = \frac{\mathbb{E}[(X-\mu)^4]}{\sigma^4},$$

donde $\mu = \mathbb{E}[X]$ es la media y $\sigma = \sqrt{\mathbb{E}[(X-\mu)^2]}$ es la desviación estándar de X.

La curtosis mide el grosor de las colas de una distribución. Una distribución con alta curtosis (*leptocúrtica*) suele tener colas más pesadas y un pico más agudo que la normal. En cambio, una distribución con baja curtosis (*platicúrtica*) suele tener colas más ligeras y un pico más plano. La curtosis de la normal es 3, por lo cual es común calcular el *exceso de curtosis* de una distribución la cual está dada por:

$$\gamma_2 = \beta_2 - 3.$$

Para ilustrar estas ideas, introduciremos la distribución normal generalizada, también conocida como la distribución de Subbotin.

Definición 2.1.6. Decimos que una variable aleatoria X sigue una *distribución normal generalizada* con parámetros de localización $\mu \in \mathbb{R}$, escala $\alpha > 0$ y forma $\beta > 0$, si su función de densidad de probabilidad (PDF) está dada por

$$f_X(x; \mu, \alpha, \beta) = \frac{\beta}{2\alpha\Gamma(1/\beta)} e^{-\left(\frac{|x-\mu|}{\alpha}\right)^{\beta}}, \quad x \in \mathbb{R}, \tag{2.3}$$

donde Γ nuevamente denota a la función Gamma, que definimos en (2.2). En este caso escribimos

$$X \sim \text{NormalGeneralizada}(\mu, \alpha, \beta).$$

Las características poblacionales más importantes de la distribución normal generalizada se muestran en la siguiente tabla:

```
# Crear la tabla de características de la distribución Gamma
generalized_normal_tbl <- tibble(
  `Característica` = c(
    "Media",
    "Varianza",
    "Asimetría",
    "Curtosis excesiva"
  ),
  `Valor` = c(
    "$\\mu$",
    "$\\alpha^{2} \\frac{\\Gamma(3/\\beta)}{\\Gamma(1/\\beta)}$",
    "0",
    "$\\frac{\\Gamma(5/\\beta) \\Gamma(1/\\beta)}{\\Gamma(3/\\beta)^{2}} - 3$"
  )
)

# Formatear y mostrar la tabla
kable(
  generalized_normal_tbl,
  format = "latex",
  booktabs = TRUE,
  escape = FALSE,
  align = c("l", "c")
) |>
  kable_styling(latex_options = "hold_position")
```

Tabla 2.2: Características de la distribución normal generalizada con parámetros μ, α y β.

Característica	Valor
Media	μ
Varianza	$\alpha^2 \frac{\Gamma(3/\beta)}{\Gamma(1/\beta)}$
Asimetría	0
Curtosis excesiva	$\frac{\Gamma(5/\beta)\Gamma(1/\beta)}{\Gamma(3/\beta)^2} - 3$

Observación 2.1.7. Notemos que para $\beta = 1$ la distribución normal generalizada coincide con la distribución de Laplace cuya PDF está dada por

$$f_X(x; \mu, b) = \frac{1}{2b} e^{-\frac{|x-\mu|}{b}}, \quad x \in \mathbb{R}, \tag{2.4}$$

donde $b > 0$ es un parámetro de escala (que correspnde al valor de α en la normal generalizada). En este caso, la curtosis excesiva es 3, indicando colas más pesadas que la normal. Por otro lado, cuando $\beta = 2$ obtenemos la distribución normal con media μ y varianza $\alpha^2/2$. Finalmente, a medida que β tiende a infinito, la distribución normal generalizada converge a una distribución uniforme en el intervalo $(\mu - \alpha, \mu + \alpha)$, la cual tiene curtosis excesiva de -1.2, indicando colas más ligeras que la normal.

Mostraremos ahora la gráfica de la PDF de la distribución normal generalizada para $\mu = 0$, $\alpha = 1$ y distintos valores de β, destacando sus colas y la forma del pico central.

```
# Creamos una secuencia de valores para X
x_data <- seq(-3.5, 3.5, length.out = 500)

# Seleccionamos los valores de beta a graficar
beta_values <- c(0.5, 1, 1.5, 2, 4, 8)

# Calculamos las densidades para cada valor de beta
dgnorm_distribution_family_tbl <- expand_grid(
  x_data = x_data,
  beta = beta_values
) |>
  mutate(
    density_data = dgnorm(x_data, mu = 0, scale = 1, shape = beta),
    beta = factor(beta)
  )

# Generamos la gráfica
dgnorm_distribution_family_plot <- dgnorm_distribution_family_tbl |>
  ggplot(aes(x = x_data, y = density_data, color = beta)) +
  geom_line(linewidth = 1) +
  scale_color_manual(
    values = unname(c_pal(
```

```
      "C red", "C yellow", "C orange",
      "C green", "C blue", "C magenta"
    )),
    labels = c(
      TeX("$\\beta$ = 0.5 (Colas muy pesadas)"),
      TeX("$\\beta$ = 1 (Laplace)"),
      TeX("$\\beta$ = 1.5 (Colas pesadas)"),
      TeX("$\\beta$ = 2 (Normal)"),
      TeX("$\\beta$ = 4 (Colas ligeras)"),
      TeX("$\\beta$ = 8 (Colas muy ligeras)")
    )
  ) +
  labs(
    title = "Distribución normal generalizada para distintos valores de beta",
    x = TeX("$X$"),
    y = "Densidad",
    color = "Parámetro de forma"
  ) +
  theme_krul()

# Mostramos la gráfica
dgnorm_distribution_family_plot
```

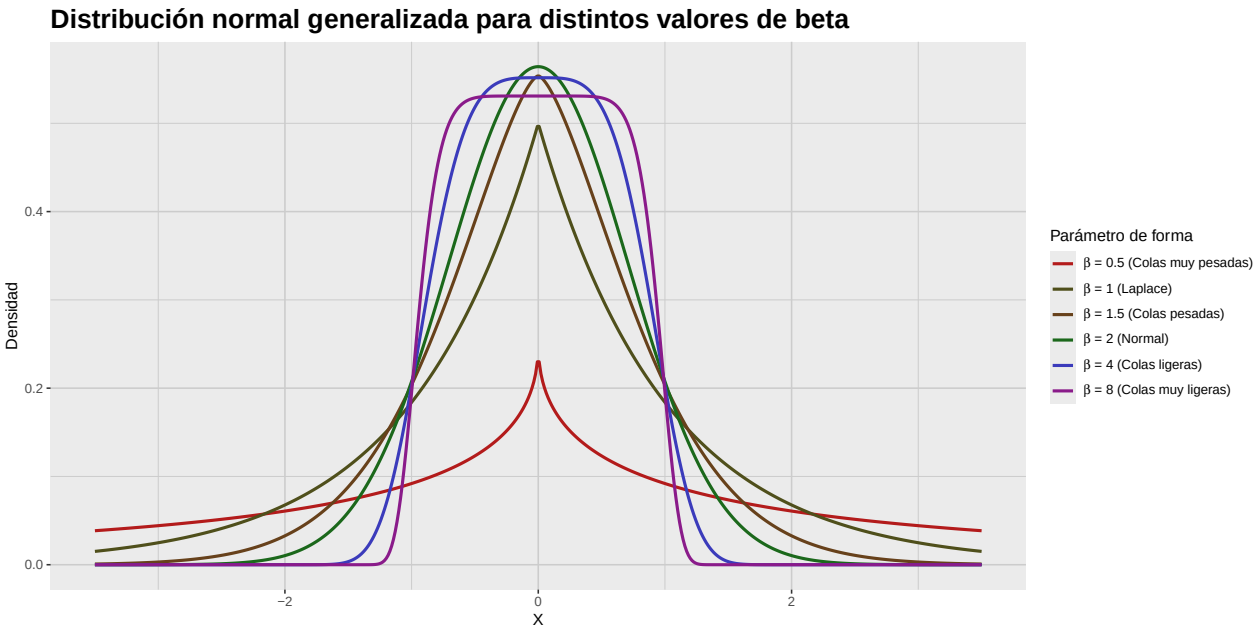


Figura 2.3: Funciones de densidad de la distribución normal generalizada para $\mu = 0$, $\alpha = 1$ y distintos valores de β. Notemos cómo cambia la forma de la distribución conforme varía el parámetro de forma β, afectando el grosor de las colas y la forma del pico central.

Finalmente, utilizaremos la distribución normal generalizada, con parámetros $\mu = 0$, $\alpha = 1$ y $\beta = 1, 2, 4$, para generar una gráfica comparativa de una distribución leptocúrtica, una mesocúrtica y una platicúrtica.

```
# Creamos una secuencia de valores para X
x_data <- seq(-2, 2, length.out = 500)

# Calculamos la PDF de la distribución leptocúrtica
leptokurtic_distribution_tbl <- tibble(
  x_data = x_data,
  density_data = dgnorm(x_data, mu = 0, scale = 1, shape = 1)
)

# Calculamos la PDF de la distribución mesocúrtica
mesokurtic_distribution_tbl <- tibble(
  x_data = x_data,
  density_data = dgnorm(x_data, mu = 0, scale = 1, shape = 2)
)

# Calculamos la PDF de la distribución platicúrtica
platykurtic_distribution_tbl <- tibble(
  x_data = x_data,
  density_data = dgnorm(x_data, mu = 0, scale = 1, shape = 4)
)

# Generamos la gráfica de la distribución leptocúrtica
leptokurtic_distribution_plot <- leptokurtic_distribution_tbl |>
  ggplot(aes(x = x_data, y = density_data)) +
  geom_line(
    color = c_pal("C blue"),
    linewidth = 1
  ) +
  labs(
    title = "Leptocúrtica",
    x = TeX("$X$"),
    y = "Densidad"
  ) +
  theme_krul()

# Generamos la gráfica de la distribución mesocúrtica
mesokurtic_distribution_plot <- mesokurtic_distribution_tbl |>
  ggplot(aes(x = x_data, y = density_data)) +
  geom_line(
    color = c_pal("C blue"),
    linewidth = 1
  ) +
```

```
  labs(
    title = "Mesocúrtica",
    x = TeX("$X$"),
    y = "Densidad"
  ) +
  theme_krul()

# Generamos la gráfica de la distribución platicúrtica
platykurtic_distribution_plot <- platykurtic_distribution_tbl |>
  ggplot(aes(x = x_data, y = density_data)) +
  geom_line(
    color = c_pal("C blue"),
    linewidth = 1
  ) +
  labs(
    title = "Platicúrtica",
    x = TeX("$X$"),
    y = "Densidad"
  ) +
  theme_krul()

# Combinamos las gráficas
kurtosis_comparison_plot <- leptokurtic_distribution_plot +
  mesokurtic_distribution_plot +
  platykurtic_distribution_plot +
  plot_annotation(
    title = "Comparación de distribuciones con distintos niveles de curtosis",
    theme = theme(plot.title = element_text(size = 24, face = "bold"))
  )

# Mostramos la gráfica
kurtosis_comparison_plot
```



Figura 2.4: Gráfica comparativa de distribuciones leptocúrtica, mesocúrtica y platicúrtica. Notemos cómo la curtosis afecta la forma de la distribución, especialmente en las colas y el pico central.

2.1.3. Asimetría y curtosis muestrales

Hasta ahora hemos definido la asimetría y curtosis para variables aleatorias. Sin embargo, en la práctica trabajamos con muestras y no con poblaciones completas, por lo que necesitamos definir la asimetría y curtosis *muestrales*. Empecemos por recordar que, dado un *vector muestral*

$$X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_n \end{bmatrix},$$

definimos su *media muestral* como

$$\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i.$$

Para la varianza muestral, tenemos dos versiones, la *varianza muestral sesgada*

$$\widetilde{S}^2 = \frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^2,$$

y la *varianza muestral insesgada*

$$S^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2.$$

En estadística, es más común utilizar la varianza muestral insesgada ya que, como su nombre lo indica, esta resulta ser un estimador insesgado de la varianza poblacional. Para definir la asimetría muestral, definiremos primero el tercer momento muestral. Igual que con la varianza muestral, existen dos versiones de esta: el *tercer momento muestral sesgado*

$$\widetilde{M}_3 = \frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^3,$$

y el *tercer momento muestral insesgado*

$$M_3 = \frac{n}{(n-1)(n-2)}\sum_{i=1}^{n}(X_i - \bar{X})^3.$$

A partir de aquí, hay varias posibles definiciones de la asimetría muestral que podemos considerar. Las más comunes son la *asimetría muestral de tipo 1*

$$\widetilde{G}_1 = \frac{\widetilde{M}_3}{\widetilde{S}^3} = \frac{\frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^3}{\left(\frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^2\right)^{3/2}} = \sqrt{n}\frac{\sum_{i=1}^{n}(X_i - \bar{X})^3}{\left[\sum_{i=1}^{n}(X_i - \bar{X})^2\right]^{3/2}},$$

y la *asimetría muestral de tipo 2*

$$G_1 = \frac{M_3}{S^3} = \frac{\frac{n}{(n-1)(n-2)}\sum_{i=1}^{n}(X_i - \bar{X})^3}{\left(\frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2\right)^{3/2}} = \frac{n\sqrt{n-1}}{n-2}\frac{\sum_{i=1}^{n}(X_i - \bar{X})^3}{\left[\sum_{i=1}^{n}(X_i - \bar{X})^2\right]^{3/2}} = \frac{\sqrt{n(n-1)}}{n-2}\widetilde{G}_1.$$

Aunque menos común, también podemos definir la *asimetría muestral de tipo 3* como

$$G_1^* = \frac{\widetilde{M}_3}{S^3} = \frac{\frac{1}{n}\sum_{i=1}^n (X_i - \bar{X})^3}{\left(\frac{1}{n-1}\sum_{i=1}^n (X_i - \bar{X})^2\right)^{3/2}} = \frac{(n-1)^{3/2}}{n} \frac{\sum_{i=1}^n (X_i - \bar{X})^3}{\left[\sum_{i=1}^n (X_i - \bar{X})^2\right]^{3/2}} = \left(1 - \frac{1}{n}\right)^{3/2} \widetilde{G}_1.$$

Observación 2.1.8. Es importante destacar que no existe un estimador insesgado de la asimetría poblacional. Sin embargo, todas estas versiones de la asimetría muestral son consistentes, es decir, convergen en probabilidad a la asimetría poblacional conforme el tamaño muestral n tiende a infinito. En particular, para muestras grandes, las tres versiones de la asimetría muestral son aproximadamente iguales.

En R, podemos calcular las tres versiones de la asimetría muestral utilizando la función `skewness()`, del paquete `e1071`, especificando el argumento `type` como 1, 2 o 3 según la versión que deseemos calcular. (Con la opción `type = 3`, especificada como default.) El paquete `moments` también ofrece una función similar llamada `skewness()`, pero esta solo calcula la asimetría muestral de tipo 1.

Mostraremos ahora un ejemplo de una muestra que viene de una distribución con sesgo a la izquierda, una simétrica y una con sesgo a la derecha.

```
# Para asegurar la reproducibilidad de los resultados
set.seed(123)

# Tamaño de la muestra
n <- 3000

# Parámetros de la muestra
alpha <- 2 # forma
beta <- 1 # tasa

# Generamos las muestras
left_skew_sample <- -rgamma(n, shape = alpha, rate = beta)
symmetric_sample <- rnorm(n)
right_skew_sample <- rgamma(n, shape = alpha, rate = beta)


# Generamos un tibble para la muestra con sesgo a la izquierda
left_skew_sample_tbl <- tibble(
  sample = left_skew_sample
)

# Generamos un tibble para la muestra simétrica
symmetric_sample_tbl <- tibble(
  sample = symmetric_sample
)

# Generamos un tibble para la muestra con sesgo a la derecha
right_skew_sample_tbl <- tibble(
```

```
  sample = right_skew_sample
)

# Generamos la gráfica de la muestra con sesgo a la izquierda
left_skew_sample_plot <- left_skew_sample_tbl |>
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Sesgo a la izquierda",
    x = "Valores muestrales",
    y = "Frecuencia absoluta"
  ) +
  theme_krul()

# Generamos la gráfica de la muestra simétrica
symmetric_sample_plot <- symmetric_sample_tbl |>
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Simétrica",
    x = "Valores muestrales",
    y = "Frecuencia absoluta"
  ) +
  theme_krul()

# Generamos la gráfica de la muestra con sesgo a la derecha
right_skew_sample_plot <- right_skew_sample_tbl |>
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
  ) +
  labs(
```

```
    title = "Sesgo a la derecha",
    x = "Valores muestrales",
    y = "Frecuencia absoluta"
  ) +
  theme_krul()

# Combinamos las gráficas
skew_sample_comparison_plot <- left_skew_sample_plot +
  symmetric_sample_plot +
  right_skew_sample_plot +
  plot_annotation(
    title = "Comparación de muestras con distintos niveles de asimetría",
    theme = theme(plot.title = element_text(size = 24, face = "bold"))
  )

# Mostramos la gráfica
skew_sample_comparison_plot
```



Figura 2.5: Gráfica comparativa de muestras con sesgo a la izquierda, simétrica y con sesgo a la derecha. Notemos como la asimetría nos alerta de la existencia de valores atípicos en las colas de la muestra en la dirección del sesgo.

Construiremos ahora una tabla que nos muestre los valores de la media, la mediana y los tres tipos de asimetría muestral para cada una de las muestras generadas anteriormente.

```
# Calcular media, mediana
# y los tres tipos de asimetría muestral para cada muestra
skew_sample_summary_tbl <- tibble(
  Estadística = c(
    "Media",
    "Mediana",
    "Asimetría Tipo 1",
    "Asimetría Tipo 2",
    "Asimetría Tipo 3"
  ),
```

```
  `Sesgo a la izquierda` = c(
    mean(left_skew_sample),
    median(left_skew_sample),
    skewness(left_skew_sample, type = 1),
    skewness(left_skew_sample, type = 2),
    skewness(left_skew_sample, type = 3)
  ),
  `Simétrica` = c(
    mean(symmetric_sample),
    median(symmetric_sample),
    skewness(symmetric_sample, type = 1),
    skewness(symmetric_sample, type = 2),
    skewness(symmetric_sample, type = 3)
  ),
  `Sesgo a la derecha` = c(
    mean(right_skew_sample),
    median(right_skew_sample),
    skewness(right_skew_sample, type = 1),
    skewness(right_skew_sample, type = 2),
    skewness(right_skew_sample, type = 3)
  )
)

# Formatear y mostrar la tabla redondeando a 4 decimales
kable(
  skew_sample_summary_tbl,
  booktabs = TRUE,
  digits = 4,
  align = c("l", "c", "c", "c")
)
```

Tabla 2.3: Tabla de media, mediana y asimetría muestral para las muestras con sesgo a la izquierda, simétrica y con sesgo a la derecha que generamos.

Estadística	Sesgo a la izquierda	Simétrica	Sesgo a la derecha
Media	-1.9375	-0.0140	1.9721
Mediana	-1.6610	-0.0452	1.6604
Asimetría Tipo 1	-1.3414	-0.0004	1.3377
Asimetría Tipo 2	-1.3421	-0.0004	1.3384
Asimetría Tipo 3	-1.3407	-0.0004	1.3370

Ahora, como sabemos que estas muestras vienen de distribuciones con media, mediana y asimetría conocida, podemos comparar los valores obtenidos para estos estadísticos muestrales con los valores de las características poblacionales que estamos estimando.

```r
# Valores poblacionales conocidos
population_values_tbl <- tibble(
  `Caraterística` = c(
    "Media",
    "Mediana",
    "Asimetría"
  ),
  `Sesgo a la izquierda` = c(
    -alpha / beta,
    -qgamma(0.5, shape = alpha, rate = beta),
    -2 / sqrt(alpha)
  ),
  `Simétrica` = c(
    0,
    0,
    0
  ),
  `Sesgo a la derecha` = c(
    alpha / beta,
    qgamma(0.5, shape = alpha, rate = beta),
    2 / sqrt(alpha)
  )
)

# Formatear y mostrar la tabla redondeando a 4 decimales
kable(
  population_values_tbl,
  booktabs = TRUE,
  digits = 4,
  align = c("l", "c", "c", "c")
)
```

Tabla 2.4: Tabla de valores poblacionales conocidos para la media, mediana y asimetría de las distribuciones consideradas con sesgo a la izquierda, simétrica y con sesgo a la derecha.

Caraterística	Sesgo a la izquierda	Simétrica	Sesgo a la derecha
Media	-2.0000	0	2.0000
Mediana	-1.6783	0	1.6783
Asimetría	-1.4142	0	1.4142

Observación 2.1.9. Comparando estas tablas, notamos lo siguiente:

- Los valores de los distintos tipos de asimetría muestral son ligeramente distintos entre sí, pero todos indican correctamente la dirección del sesgo de la muestra. En particular, para muestras lo suficientemente grandes, todos proporcionan esencialmente la misma información.

- Los valores de estos estadísticos son ligeramente diferentes a los valores poblacionales conocidos, pero se acercan más conforme el tamaño de la muestra aumenta. Esto ilustra la propiedad de consistencia de estos estimadores, pero también ilustra la diferencia entre los valores de los estadísticos de una muestra (que suelen cambiar de una muestra a otra) y los parámetros poblacionales (que son fijos).

Para definir la curtosis muestral, empezaremos por definir el *cuarto momento muestral sesgado* como

$$\widetilde{M}_4 = \frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^4.$$

El paquete `e1071`, a través de la función `kurtosis()`, nos permite calcular tres tipos de curtosis a partir de esta definición: la *curtosis muestral de tipo 1*

$$\widetilde{G}_2 = \frac{\widetilde{M}_4}{\widetilde{S}^4} - 3 = \frac{\frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^4}{\left(\frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^2\right)^2} - 3 = n\frac{\sum_{i=1}^{n}(X_i - \bar{X})^4}{\left[\sum_{i=1}^{n}(X_i - \bar{X})^2\right]^2} - 3,$$

la *curtosis muestral de tipo 2*

$$\begin{aligned} G_2 =& \frac{(n-1)(n+1)}{(n-2)(n-3)}\left[\widetilde{G}_2 + \frac{6}{n+1}\right] = \frac{n-1}{(n-2)(n-3)}\left[(n+1)\widetilde{G}_2 + 6\right] \\ =& \frac{(n-1)n(n+1)}{(n-2)(n-3)}\frac{\sum_{i=1}^{n}(X_i - \bar{X})^4}{\left[\sum_{i=1}^{n}(X_i - \bar{X})^2\right]^2} - \frac{3(n-1)^2}{(n-2)(n-3)}, \end{aligned}$$

y la *curtosis muestral de tipo 3* (que es la opción por default de la función `kurtosis()` del paquete `e1071`)

$$\begin{aligned} G_2^* =& \frac{\widetilde{M}_4}{S^4} - 3 = \left(1 - \frac{1}{n}\right)^2(\widetilde{G}_2 + 3) - 3 \\ =& \frac{\frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^4}{\left(\frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2\right)^2} - 3 = \frac{(n-1)^2}{n}\frac{\sum_{i=1}^{n}(X_i - \bar{X})^4}{\left[\sum_{i=1}^{n}(X_i - \bar{X})^2\right]^2} - 3. \end{aligned}$$

Notemos que todas estas definiciones estiman el exceso de curtosis. Al igual que con la asimetría muestral, no existe un estimador insesgado de la curtosis poblacional, pero todas estas versiones de la curtosis muestral son estimadores consistentes de este valor.

Mostraremos ahora un ejemplo de una muestra que viene de una distribución leptocúrtica, una mesocúrtica y una platicúrtica.

```
# Para asegurar la reproducibilidad de los resultados
set.seed(123)

# Tamaño de la muestra
n <- 3000

# Parámetros de la muestra
mu <- 0 # localización
```

```r
alpha <- 1 # escala
beta_leptokurtic <- 1 # forma leptocúrtica
beta_mesokurtic <- 2 # forma mesocúrtica
beta_platykurtic <- 4 # forma platicúrtica

# Generamos la muestra leptocúrtica
leptokurtic_sample <- rgnorm(
  n = n,
  mu = mu,
  scale = alpha,
  shape = beta_leptokurtic
)

# Generamos la muestra mesocúrtica
mesokurtic_sample <- rgnorm(
  n = n,
  mu = mu,
  scale = alpha,
  shape = beta_mesokurtic
)

# Generamos la muestra platicúrtica
platykurtic_sample <- rgnorm(
  n = n,
  mu = mu,
  scale = alpha,
  shape = beta_platykurtic
)

# Generamos un tibble para la muestra leptocúrtica
leptokurtic_sample_tbl <- tibble(
  sample = leptokurtic_sample
)

# Generamos un tibble para la muestra mesocúrtica
mesokurtic_sample_tbl <- tibble(
  sample = mesokurtic_sample
)

# Generamos un tibble para la muestra platicúrtica
platykurtic_sample_tbl <- tibble(
  sample = platykurtic_sample
)

# Generamos la gráfica de la muestra leptocúrtica
```

```
leptokurtic_sample_plot <- leptokurtic_sample_tbl |>
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Leptocúrtica",
    x = "Valores muestrales",
    y = "Frecuencia absoluta"
  ) +
  theme_krul()

# Generamos la gráfica de la muestra mesocúrtica
mesokurtic_sample_plot <- mesokurtic_sample_tbl |>
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Mesocúrtica",
    x = "Valores muestrales",
    y = "Frecuencia absoluta"
  ) +
  theme_krul()

# Generamos la gráfica de la muestra platicúrtica
platykurtic_sample_plot <- platykurtic_sample_tbl |>
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Platicúrtica",
    x = "Valores muestrales",
    y = "Frecuencia absoluta"
  ) +
```

```
  theme_krul()

# Combinamos las gráficas
kurtosis_sample_comparison_plot <- leptokurtic_sample_plot +
  mesokurtic_sample_plot +
  platykurtic_sample_plot +
  plot_annotation(
    title = "Comparación de muestras con distintos niveles de curtosis",
    theme = theme(plot.title = element_text(size = 24, face = "bold"))
  )

# Mostramos la gráfica
kurtosis_sample_comparison_plot
```



Figura 2.6: Gráfica comparativa de muestras leptocúrtica, mesocúrtica y platicúrtica. Notemos como las muestras con mayor curtosis tienden a tener más valores atípicos en las colas, mientras que las muestras con menor curtosis tienen colas más ligeras.

Construiremos ahora una tabla que nos muestre los valores de la media, la varianza y los tres tipos de curtosis muestral para cada una de las muestras generadas anteriormente.

```
# Calcular media, varianza
# y los tres tipos de curtosis muestral para cada muestra
kurtosis_sample_summary_tbl <- tibble(
  Estadística = c(
    "Media",
    "Varianza",
    "Curtosis Tipo 1",
    "Curtosis Tipo 2",
    "Curtosis Tipo 3"
  ),
  `Leptocúrtica` = c(
    mean(leptokurtic_sample),
    var(leptokurtic_sample),
    kurtosis(leptokurtic_sample, type = 1),
```

```
    kurtosis(leptokurtic_sample, type = 2),
    kurtosis(leptokurtic_sample, type = 3)
  ),
  `Mesocúrtica` = c(
    mean(mesokurtic_sample),
    var(mesokurtic_sample),
    kurtosis(mesokurtic_sample, type = 1),
    kurtosis(mesokurtic_sample, type = 2),
    kurtosis(mesokurtic_sample, type = 3)
  ),
  `Platicúrtica` = c(
    mean(platykurtic_sample),
    var(platykurtic_sample),
    kurtosis(platykurtic_sample, type = 1),
    kurtosis(platykurtic_sample, type = 2),
    kurtosis(platykurtic_sample, type = 3)
  )
)

# Formatear y mostrar la tabla redondeando a 4 decimales
kable(
  kurtosis_sample_summary_tbl,
  booktabs = TRUE,
  digits = 4,
  align = c("l", "c", "c", "c")
)
```

Tabla 2.5: Tabla de media, varianza y curtosis muestral para las muestras leptocúrtica, mesocúrtica y platicúrtica que generamos.

Estadística	Leptocúrtica	Mesocúrtica	Platicúrtica
Media	-0.0094	-0.0027	-0.0111
Varianza	1.9775	0.4935	0.3281
Curtosis Tipo 1	2.5569	0.0291	-0.7679
Curtosis Tipo 2	2.5631	0.0311	-0.7672
Curtosis Tipo 3	2.5532	0.0271	-0.7694

Ahora, como sabemos que estas muestras vienen de distribuciones con media, varianza y curtosis conocida, podemos comparar los valores obtenidos para estos estadísticos muestrales con los valores de las características poblacionales que estamos estimando.

```
# Valores poblacionales conocidos
population_kurtosis_values_tbl <- tibble(
  `Caraterística` = c(
    "Media",
```

```
    "Varianza",
    "Curtosis excesiva"
  ),
  `Leptocúrtica` = c(
    mu,
    alpha^2 * gamma(3 / beta_leptokurtic) / gamma(1 / beta_leptokurtic),
    gamma(5 / beta_leptokurtic) * gamma(1 / beta_leptokurtic) /
      (gamma(3 / beta_leptokurtic))^2 - 3
  ),
  `Mesocúrtica` = c(
    mu,
    alpha^2 * gamma(3 / beta_mesokurtic) / gamma(1 / beta_mesokurtic),
    gamma(5 / beta_mesokurtic) * gamma(1 / beta_mesokurtic) /
      (gamma(3 / beta_mesokurtic))^2 - 3
  ),
  `Platicúrtica` = c(
    mu,
    alpha^2 * gamma(3 / beta_platykurtic) / gamma(1 / beta_platykurtic),
    gamma(5 / beta_platykurtic) * gamma(1 / beta_platykurtic) /
      (gamma(3 / beta_platykurtic))^2 - 3
  )
)

# Formatear y mostrar la tabla redondeando a 4 decimales
kable(
  population_kurtosis_values_tbl,
  booktabs = TRUE,
  digits = 4,
  align = c("l", "c", "c", "c")
)
```

Tabla 2.6: Tabla de valores poblacionales conocidos para la media, varianza y curtosis de las distribuciones leptocúrtica, mesocúrtica y platicúrtica consideradas.

Caraterística	Leptocúrtica	Mesocúrtica	Platicúrtica
Media	0	0.0	0.0000
Varianza	2	0.5	0.3380
Curtosis excesiva	3	0.0	-0.8116

Nuevamente notamos que los valores de los distintos tipos de curtosis muestral son ligeramente diferentes entre sí, pero todos indican correctamente el nivel de curtosis de la distribución de la cual se extrajo la muestra. Además, los valores de estos estadísticos son ligeramente diferentes a los valores poblacionales conocidos, pero se acercan más conforme el tamaño de la muestra aumenta, ilustrando que estos estimadores son, efectivamente, consistentes.

2.2. Gráficos QQ

Los gráficos QQ (quantile–quantile) son una herramienta que nos permite comparar la distribución de un conjunto de datos dado contra una distribución teórica. Cuando tomamos esta distribución teórica como la normal estándar, estos gráficos nos permiten evaluar de manera visual la normalidad de nuestros datos.

Para construir un gráfico QQ, debemos seguir los siguientes pasos:

1. **Ordenar los datos muestrales.** Coloque los datos en orden creciente:

$$X_{(1)} \leq X_{(2)} \leq \cdots \leq X_{(n)}.$$

2. **Calcular las probabilidades a considerar.** Para cada valor ordenado $X_{(i)}$, calcule la probabilidad correspondiente

$$P_i = \frac{2i-1}{2n}, \quad i = 1, 2, \ldots, n.$$

3. **Obtener los cuantiles teóricos de la normal.** Usando la función cuantil de la normal estándar, podemos obtener los cuantiles teóricos correspondientes a las probabilidades calculadas en el paso anterior:

$$Q_i = \Phi^{-1}(P_i),$$

donde Φ^{-1} es la función cuantil de la normal estándar.

4. **Generar el gráfico.** Una vez obtenidos estos valores, debemos graficar los pares $(Q_i, X_{(i)})$ para $i = 1, 2, \ldots, n$, en un mismo plano cartesiano. También es común agregar una recta de referencia que pase por el primer y tercer cuartil de los datos muestrales.

En general, entre más cerca se encuentren los puntos del gráfico QQ a la recta de referencia, más evidencia tendremos de que los datos muestrales provienen de una distribución normal. Por el contrario, si los puntos se alejan significativamente de esta recta, tendremos evidencia en contra de la normalidad de nuestros datos.

Para ilustrar estas ideas, consideraremos los histogramas y los gráficos QQ de las muestras con sesgo a la izquierda, simétrica y con sesgo a la derecha que generamos en la sección anterior.

```
# Generamos el gráfico QQ para la muestra con sesgo a la izquierda
left_skew_qq_plot <- left_skew_sample_tbl |>
  ggplot(aes(sample = sample)) +
  geom_qq_line(
    color = c_pal("C magenta"),
    linewidth = 0.5
  ) +
  geom_qq(
    color = c_pal("C blue"),
    size = 0.3
  ) +
  labs(
```

```
    x = "Cuantiles teóricos",
    y = "Cuantiles muestrales"
  ) +
  theme_krul()

# Generamos el gráfico QQ para la muestra simétrica
symmetric_qq_plot <- symmetric_sample_tbl |>
  ggplot(aes(sample = sample)) +
  geom_qq_line(
    color = c_pal("C magenta"),
    linewidth = 0.5
  ) +
  geom_qq(
    color = c_pal("C blue"),
    size = 0.3
  ) +
  labs(
    x = "Cuantiles teóricos",
    y = "Cuantiles muestrales"
  ) +
  theme_krul()

# Generamos el gráfico QQ para la muestra con sesgo a la derecha
right_skew_qq_plot <- right_skew_sample_tbl |>
  ggplot(aes(sample = sample)) +
  geom_qq_line(
    color = c_pal("C magenta"),
    linewidth = 0.5
  ) +
  geom_qq(
    color = c_pal("C blue"),
    size = 0.3
  ) +
  labs(
    x = "Cuantiles teóricos",
    y = "Cuantiles muestrales"
  ) +
  theme_krul()

# Combinamos los gráficos QQ
skew_sample_qq_comparison_plot <- left_skew_qq_plot +
  symmetric_qq_plot +
  right_skew_qq_plot

# Combinamos los gráficos QQ con los histogramas
```

```
skew_hist_qq_comparison_plot <- skew_sample_comparison_plot /
  skew_sample_qq_comparison_plot +
  plot_annotation(
    title = paste0(
      "Comparación de histogramas y gráficos QQ\n",
      "de muestras con distintos niveles de asimetría"
    ),
    theme = theme(plot.title = element_text(size = 24, face = "bold"))
  )

# Mostramos la gráfica
skew_hist_qq_comparison_plot
```

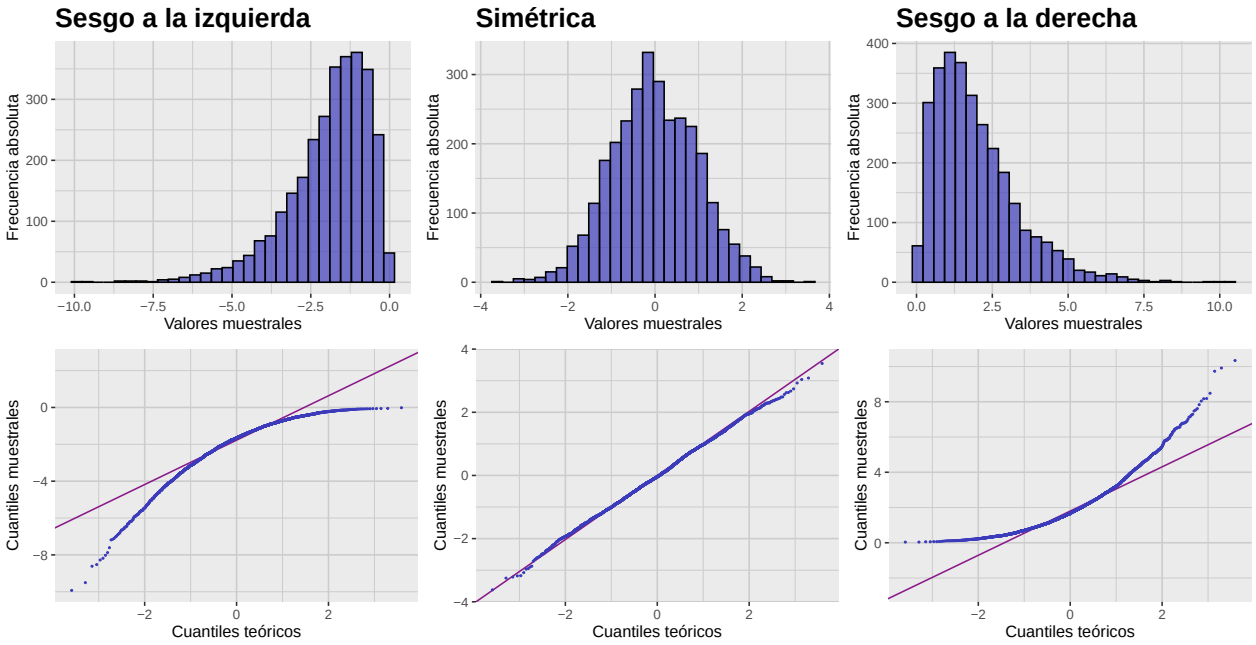


Figura 2.7: Gráfica comparativa de histogramas y gráficos QQ de muestras con sesgo a la izquierda, simétrica y con sesgo a la derecha. Notemos cómo los gráficos QQ nos permiten evaluar visualmente la normalidad de las muestras, mostrando desviaciones significativas de la línea de referencia en las muestras sesgadas.

Consideraremos también los histogramas y los gráficos QQ de las muestras leptocúrtica, mesocúrtica y platicúrtica que generamos en la sección anterior.

```
# Generamos el gráfico QQ para la muestra leptocúrtica
leptokurtic_qq_plot <- leptokurtic_sample_tbl |>
  ggplot(aes(sample = sample)) +
  geom_qq_line(
```

```
    color = c_pal("C magenta"),
    linewidth = 0.5
  ) +
  geom_qq(
    color = c_pal("C blue"),
    size = 0.3
  ) +
  labs(
    x = "Cuantiles teóricos",
    y = "Cuantiles muestrales"
  ) +
  theme_krul()

# Generamos el gráfico QQ para la muestra mesocúrtica
mesokurtic_qq_plot <- mesokurtic_sample_tbl |>
  ggplot(aes(sample = sample)) +
  geom_qq_line(
    color = c_pal("C magenta"),
    linewidth = 0.5
  ) +
  geom_qq(
    color = c_pal("C blue"),
    size = 0.3
  ) +
  labs(
    x = "Cuantiles teóricos",
    y = "Cuantiles muestrales"
  ) +
  theme_krul()

# Generamos el gráfico QQ para la muestra platicúrtica
platykurtic_qq_plot <- platykurtic_sample_tbl |>
  ggplot(aes(sample = sample)) +
  geom_qq_line(
    color = c_pal("C magenta"),
    linewidth = 0.5
  ) +
  geom_qq(
    color = c_pal("C blue"),
    size = 0.3
  ) +
  labs(
    x = "Cuantiles teóricos",
    y = "Cuantiles muestrales"
  ) +
```

```
  theme_krul()

# Combinamos los gráficos QQ
kurtosis_sample_qq_comparison_plot <- leptokurtic_qq_plot +
  mesokurtic_qq_plot +
  platykurtic_qq_plot

# Combinamos los gráficos QQ con los histogramas
kurtosis_hist_qq_comparison_plot <- kurtosis_sample_comparison_plot /
  kurtosis_sample_qq_comparison_plot +
  plot_annotation(
    title = paste0(
      "Comparación de histogramas y gráficos QQ\n",
      "de muestras con distintos niveles de curtosis"
    ),
    theme = theme(plot.title = element_text(size = 24, face = "bold"))
  )

# Mostramos la gráfica
kurtosis_hist_qq_comparison_plot
```

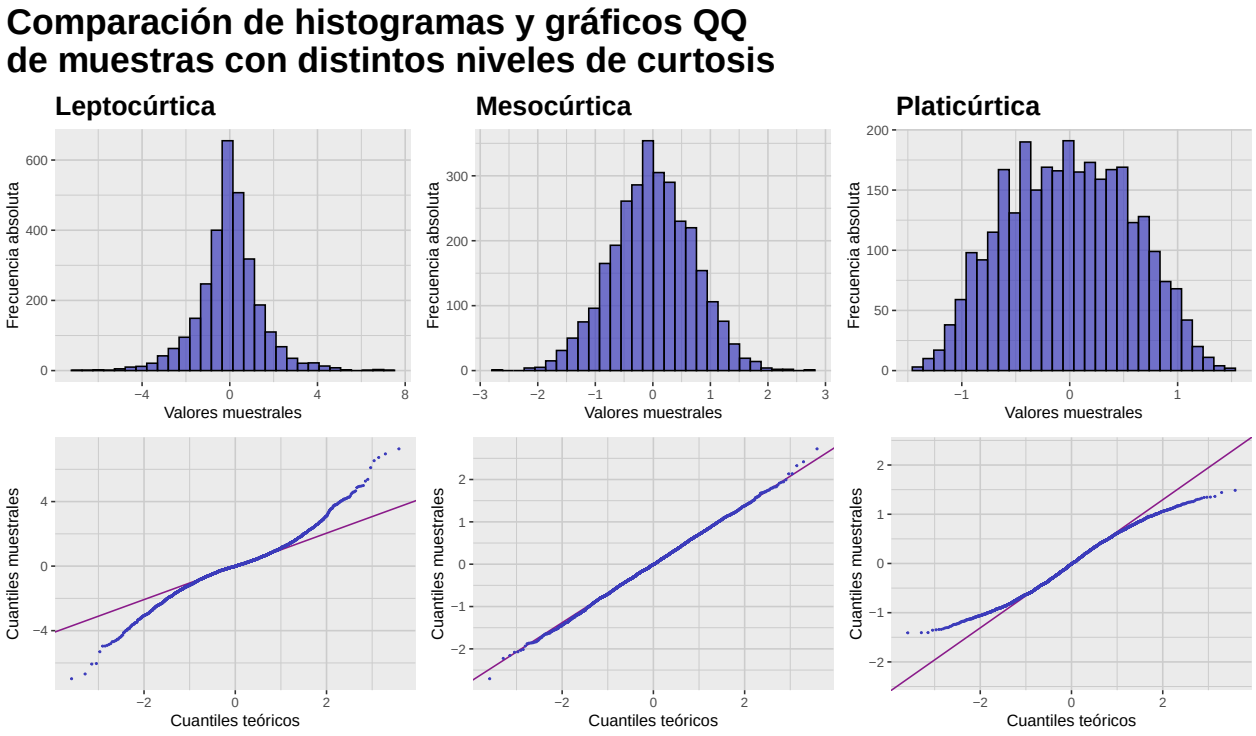


Figura 2.8: Gráfica comparativa de histogramas y gráficos QQ de muestras leptocúrtica, mesocúrtica y platicúrtica. Observemos cómo los gráficos QQ reflejan las diferencias en la curtosis de las muestras, con desviaciones notables de la línea de referencia en las muestras leptocúrtica y platicúrtica.

2.3. Pruebas de normalidad

Aunque los gráficos QQ son una buena forma de evaluar visualmente la normalidad de nuestros datos, muchas veces es necesario contar con pruebas estadísticas más objetivas, y que nos permitan determinar de una manera cuantitativa si estos datos siguen o no una distribución normal. Algunas de las pruebas más usadas para este propósito son las siguientes:

1. **Shapiro–Wilk.** Evalúa que tan bien los datos ordenados de la muestra se ajustan a los cuantiles esperados de una distribución normal. Es muy usada para muestras pequeñas (menores a 50) pero puede ser demasiado sensible en muestras grandes. En R, se puede llamar utilizando la función `shapiro.test()`.
2. **Kolmogorov–Smirnov.** Compara la función de distribución empírica con la acumulada de la normal. Es no paramétrica y aplicable a cualquier tamaño, pero menos potente que Shapiro–Wilk en muestras pequeñas. Para llamar a esta función en R, se puede usar el comando `ks.test()`.
3. **Anderson–Darling.** Similar a Kolmogorov–Smirnov, pero da mayor peso a las colas. Potente en muestras pequeñas y ampliamente utilizada. Para utilizar esta prueba en R, se puede usar la función `ad.test()` del paquete `nortest`.
4. **Jarque–Bera.** Basada en la asimetría y curtosis de los datos. Es potente en muestras grandes, pero en muestras pequeñas puede ser demasiado sensible a leves desvíos. En R, se puede llamar utilizando la función `jarque.bera.test()` del paquete `tseries`.

Como ilustración, aplicaremos estas pruebas a las muestras con sesgo a la izquierda, simétrica y con sesgo a la derecha que generamos anteriormente.

```
# Generamos la tabla de resultados de las pruebas de normalidad
skew_normality_results_tbl <- list(
  compute_normality_pvalues(
    data = left_skew_sample_tbl,
    value_col = sample,
    sample_name = "Sesgo a la izquierda"
  ),
  compute_normality_pvalues(
    data = symmetric_sample_tbl,
    value_col = sample,
    sample_name = "Simétrica"
  ),
  compute_normality_pvalues(
    data = right_skew_sample_tbl,
    value_col = sample,
    sample_name = "Sesgo a la derecha"
  )
) |>
  reduce(full_join, by = "Prueba")

# Formatear y mostrar la tabla redondeando a 4 decimales
```

```
kable(
  skew_normality_results_tbl,
  format = "latex",
  booktabs = TRUE,
  digits = 4,
  align = c("l", rep("c", ncol(skew_normality_results_tbl) - 1))
) |>
  kable_styling(latex_options = "hold_position")
```

Tabla 2.7: Tabla de valores p de las pruebas de normalidad aplicadas a las muestras con sesgo a la izquierda, simétrica y con sesgo a la derecha que generamos.

Prueba	Sesgo a la izquierda	Simétrica	Sesgo a la derecha
Shapiro-Wilk	0	0.3335	0
Kolmogorov-Smirnov	0	0.5069	0
Anderson-Darling	0	0.0894	0
Jarque-Bera	0	0.8762	0

Notemos que todas estas pruebas rechazan la hipótesis de normalidad de manera contundente para las muestras con sesgo a la izquierda y con sesgo a la derecha, mientras que no rechazan esta hipótesis para la muestra simétrica. En general, todas estas pruebas suelen rechazar la hipótesis nula para muestras muy grandes, incluso si la desviación de la normalidad es mínima. Es por esto que es importante complementar los resultados de estas pruebas con gráficos QQ y análisis adicionales para tomar decisiones informadas sobre la viabilidad de nuestros datos. Como ejemplo, consideraremos ahora estas mismas pruebas aplicadas a muestras mucho más pequeñas extraídas de las mismas distribuciones consideradas anteriormente.

```
# Para asegurar la reproducibilidad de los resultados
set.seed(173)

# Tamaño de la muestra
n <- 30

# Parámetros de la muestra
alpha <- 2 # forma
beta <- 1 # tasa

# Generamos las muestras
left_skew_sample <- -rgamma(n, shape = alpha, rate = beta)
symmetric_sample <- rnorm(n)
right_skew_sample <- rgamma(n, shape = alpha, rate = beta)

# Generamos los tibbles para las muestras
left_skew_sample_tbl <- tibble(sample = left_skew_sample)
symmetric_sample_tbl <- tibble(sample = symmetric_sample)
```

```
right_skew_sample_tbl <- tibble(sample = right_skew_sample)

# Generamos la tabla de resultados de las pruebas de normalidad
skew_normality_results_tbl <- list(
  compute_normality_pvalues(
    data = left_skew_sample_tbl,
    value_col = sample,
    sample_name = "Sesgo a la izquierda"
  ),
  compute_normality_pvalues(
    data = symmetric_sample_tbl,
    value_col = sample,
    sample_name = "Simétrica"
  ),
  compute_normality_pvalues(
    data = right_skew_sample_tbl,
    value_col = sample,
    sample_name = "Sesgo a la derecha"
  )
) |>
  reduce(full_join, by = "Prueba")

# Formatear y mostrar la tabla redondeando a 4 decimales
kable(
  skew_normality_results_tbl,
  format = "latex",
  booktabs = TRUE,
  digits = 4,
  align = c("l", rep("c", ncol(skew_normality_results_tbl) - 1))
) |>
  kable_styling(latex_options = "hold_position")
```

Tabla 2.8: Tabla de valores p de las pruebas de normalidad aplicadas a las muestras pequeñas generadas a partir de distribuciones con sesgo a la izquierda, simétrica y con sesgo a la derecha.

Prueba	Sesgo a la izquierda	Simétrica	Sesgo a la derecha
Shapiro-Wilk	0.3277	0.7246	0.0554
Kolmogorov-Smirnov	0.5882	0.9962	0.6770
Anderson-Darling	0.2040	0.8696	0.1075
Jarque-Bera	0.4466	0.8289	0.1883

Notemos que en este caso, las pruebas de normalidad no logran rechazar la hipótesis de normalidad para ninguna de las muestras consideradas. Esto ilustra cómo el tamaño de la muestra puede influir significativamente en los resultados de estas pruebas, y refuerza la importancia de considerar el con-

texto del análisis estadístico al interpretar estos resultados. Como ejemplo adicional, consideraremos ahora estas mismas pruebas aplicadas a las muestras leptocúrtica, mesocúrtica y platicúrtica que generamos anteriormente.

```
# Generamos la tabla de resultados de las pruebas de normalidad
kurtosis_normality_results_tbl <- list(
  compute_normality_pvalues(
    data = leptokurtic_sample_tbl,
    value_col = sample,
    sample_name = "Leptocúrtica"
  ),
  compute_normality_pvalues(
    data = mesokurtic_sample_tbl,
    value_col = sample,
    sample_name = "Mesocúrtica"
  ),
  compute_normality_pvalues(
    data = platykurtic_sample_tbl,
    value_col = sample,
    sample_name = "Platicúrtica"
  )
) |>
  reduce(full_join, by = "Prueba")

# Formatear y mostrar la tabla redondeando a 4 decimales
kable(
  kurtosis_normality_results_tbl,
  format = "latex",
  booktabs = TRUE,
  digits = 4,
  align = c("l", rep("c", ncol(kurtosis_normality_results_tbl) - 1))
) |>
  kable_styling(latex_options = "hold_position")
```

Tabla 2.9: Tabla de valores p de las pruebas de normalidad aplicadas a las muestras leptocúrtica, mesocúrtica y platicúrtica que generamos.

Prueba	Leptocúrtica	Mesocúrtica	Platicúrtica
Shapiro-Wilk	0	0.9409	0.0000
Kolmogorov-Smirnov	0	0.9673	0.0006
Anderson-Darling	0	0.7535	0.0000
Jarque-Bera	0	0.8298	0.0000

Nuevamente notamos que todas estas pruebas son muy sensibles a desviaciones de la normalidad para muestras grandes. En general, es difícil esperar que una muestra obtenida a partir de datos

reales siga exactamente una distribución normal, por lo que hay que ser cautelosos al interpretar los resultados de estas pruebas. Como ejemplo final, consideremos ahora estas mismas pruebas aplicadas a muestras mucho más pequeñas extraídas de estas mismas distribuciones.

```
# Para asegurar la reproducibilidad de los resultados
set.seed(136)

# Tamaño de la muestra
n <- 30

# Parámetros de la muestra
mu <- 0 # localización
alpha <- 1 # escala
beta_leptokurtic <- 1 # forma leptocúrtica
beta_mesokurtic <- 2 # forma mesocúrtica
beta_platykurtic <- 4 # forma platicúrtica

# Generamos la muestra leptocúrtica
leptokurtic_sample <- rgnorm(
  n = n,
  mu = mu,
  scale = alpha,
  shape = beta_leptokurtic
)

# Generamos la muestra mesocúrtica
mesokurtic_sample <- rgnorm(
  n = n,
  mu = mu,
  scale = alpha,
  shape = beta_mesokurtic
)

# Generamos la muestra platicúrtica
platykurtic_sample <- rgnorm(
  n = n,
  mu = mu,
  scale = alpha,
  shape = beta_platykurtic
)

# Generamos los tibbles para las muestras
leptokurtic_sample_tbl <- tibble(sample = leptokurtic_sample)
mesokurtic_sample_tbl <- tibble(sample = mesokurtic_sample)
platykurtic_sample_tbl <- tibble(sample = platykurtic_sample)
```

```r
# Generamos la tabla de resultados de las pruebas de normalidad
kurtosis_normality_results_tbl <- list(
  compute_normality_pvalues(
    data = leptokurtic_sample_tbl,
    value_col = sample,
    sample_name = "Leptocúrtica"
  ),
  compute_normality_pvalues(
    data = mesokurtic_sample_tbl,
    value_col = sample,
    sample_name = "Mesocúrtica"
  ),
  compute_normality_pvalues(
    data = platykurtic_sample_tbl,
    value_col = sample,
    sample_name = "Platicúrtica"
  )
) |>
  reduce(full_join, by = "Prueba")

# Formatear y mostrar la tabla redondeando a 4 decimales
kable(
  kurtosis_normality_results_tbl,
  format = "latex",
  booktabs = TRUE,
  digits = 4,
  align = c("l", rep("c", ncol(kurtosis_normality_results_tbl) - 1))
) |>
  kable_styling(latex_options = "hold_position")
```

Tabla 2.10: Tabla de valores p de las pruebas de normalidad aplicadas a las muestras pequeñas generadas a partir de distribuciones leptocúrtica, mesocúrtica y platicúrtica.

Prueba	Leptocúrtica	Mesocúrtica	Platicúrtica
Shapiro-Wilk	0.3948	0.5283	0.7410
Kolmogorov-Smirnov	0.8412	0.9165	0.9926
Anderson-Darling	0.4862	0.6399	0.9001
Jarque-Bera	0.7170	0.3866	0.6199

Como podemos notar, en este caso las pruebas de normalidad tampoco lograron rechazar la hipótesis nula de normalidad para ninguna de las muestras pequeñas consideradas. En conclusión, aunque estas pruebas son herramientas útiles para evaluar la normalidad de los datos, es crucial interpretar siempre sus resultados en el contexto del análisis estadístico que se está realizando y considerar otros factores como el tamaño de la muestra y el propósito del estudio.

Capítulo 3

Escalamiento y transformaciones

El escalamiento y las transformaciones de variables aleatorias y muestras son dos maneras en las que podemos cambiar la manera en la que representamos la información contenida en un modelo estadístico. Estas técnicas son útiles en diversas situaciones, como cuando queremos comparar variables con diferentes unidades de medida o cuando buscamos mejorar la normalidad de los datos para cumplir con los supuestos de ciertos análisis estadísticos. La principal diferencia entre ambas es que el escalamiento preserva la forma de la distribución original, mientras que las transformaciones pueden alterar dicha forma así como sus propiedades estadísticas.

3.1. Escalamiento de variables aleatorias

Un problema común en el análisis multivariado es que las variables involucradas pueden tener diferentes unidades de medida o rangos de valores. Esto provoca que algunas variables dominen el análisis simplemente por su escala, lo que puede llevar a interpretaciones erróneas. En esta sección discutiremos los métodos más comunes para escalar y estandarizar variables y muestras, lo que ayuda a mitigar este tipo de problemas.

Definición 3.1.1. Una *transformación afín* de una variable aleatoria X es una transformación de la forma

$$Y = aX + b$$

donde a y b son constantes.

Comentario 3.1.2. En estadística, a menudo se habla de *escalamientos* (aunque incluyan traslaciones) para referirse a transformaciones afines.

Suponga que X tiene media μ y desviación estándar σ. Entonces la variable

$$Y = aX + b$$

tendrá media $a\mu + b$ y desviación estándar $|a|\sigma$. En particular, si elegimos $a = 1/\sigma$ y $b = -\mu/\sigma$, entonces la variable

$$Z = \frac{X - \mu}{\sigma}$$

tendrá media 0 y desviación estándar 1. Esta transformación se conoce como la *estandarización* de la variable X y, en este caso, decimos que Z es la versión *estandarizada* de X (también conocida como su *puntaje z*).

Comentario 3.1.3. Si X no es normal, entonces Z *no* será normal estándar. La estandarización no transforma una distribución no normal en una distribución normal.

Observación 3.1.4. Por definición,

$$\gamma_1(Z) = \mathbb{E}[Z^3] = \frac{\mathbb{E}[(X-\mu)^3]}{\sigma^3} = \gamma_1(X)$$

y

$$\gamma_2(Z) = \mathbb{E}[Z^4] - 3 = \frac{\mathbb{E}[(X-\mu)^4]}{\sigma^4} - 3 = \gamma_2(X).$$

En otras palabras, la asimetría y la curtosis en exceso no cambian al estandarizar una variable aleatoria.

Como ejemplo, consideremos de nuevo la distribución Gamma con parámetros $\alpha = 2$ y $\beta = 1$. En este caso la media y la desviación estándar están dadas por:

$$\mu = \frac{\alpha}{\beta} = 2$$
$$\sigma = \frac{\sqrt{\alpha}}{\beta} = \sqrt{2}.$$

Usando esta información, podemos generar la gráfica de la versión estandarizada de la distribución Gamma y compararla con la original.

```
# Cargamos las librerías necesarias
library(tidyverse)
library(patchwork)
library(krulRutils)
library(car)
library(latex2exp)

# Parámetros de la distribución Gamma
alpha <- 2
beta <- 1

# Calculamos la media, la mediana y la desviación estándar
gamma_mean <- alpha / beta
gamma_median <- qgamma(0.5, shape = alpha, rate = beta)
gamma_sd <- sqrt(alpha) / beta

# Creamos una secuencia de valores para X
x_data <- seq(0, 8, length.out = 1000)

# Calculamos la PDF de la distribución Gamma original
gamma_distribution_tbl <- tibble(
```

```
  x_data = x_data,
  density_data = dgamma(x_data, shape = alpha, rate = beta)
)

# Calculamos la PDF de la distribución Gamma estandarizada
z_gamma_distribution_tbl <- gamma_distribution_tbl |>
  mutate(
    z_data = (x_data - gamma_mean) / gamma_sd,
    z_density = dgamma(x_data, shape = alpha, rate = beta) * gamma_sd
  ) |>
  select(z_data, z_density)

# Calculamos la media y mediana estandarizadas
z_gamma_mean <- 0
z_gamma_median <- (gamma_median - gamma_mean) / gamma_sd

# Generamos la gráfica de la distribución Gamma original
gamma_distribution_plot <- gamma_distribution_tbl |>
  ggplot(aes(x = x_data, y = density_data)) +
  geom_line(
    color = c_pal("C blue"),
    linewidth = 1
  ) +
  geom_vline(
    xintercept = gamma_mean,
    color = c_pal("C red"),
    linetype = "dashed",
    linewidth = 1
  ) +
  geom_vline(
    xintercept = gamma_median,
    color = c_pal("C green"),
    linetype = "dashed",
    linewidth = 1
  ) +
  annotate(
    "text",
    x = gamma_mean,
    y = 0.6,
    label = "Media",
    color = c_pal("C red"),
    hjust = -0.1
  ) +
  annotate(
    "text",
```

```
    x = gamma_median,
    y = 0.6,
    label = "Mediana",
    color = c_pal("C green"),
    hjust = 1.1
  ) +
  coord_cartesian(ylim = c(0, 0.65)) +
  labs(
    title = "Original",
    x = TeX("$X$"),
    y = "Densidad"
  ) +
  theme_krul()

# Generamos la gráfica de la distribución Gamma estandarizada
z_gamma_distribution_plot <- z_gamma_distribution_tbl |>
  ggplot(aes(x = z_data, y = z_density)) +
  geom_line(
    color = c_pal("C blue"),
    linewidth = 1
  ) +
  geom_vline(
    xintercept = z_gamma_mean,
    color = c_pal("C red"),
    linetype = "dashed",
    linewidth = 1
  ) +
  geom_vline(
    xintercept = z_gamma_median,
    color = c_pal("C green"),
    linetype = "dashed",
    linewidth = 1
  ) +
  annotate(
    "text",
    x = z_gamma_mean,
    y = 0.6,
    label = "Media",
    color = c_pal("C red"),
    hjust = -0.1
  ) +
  annotate(
    "text",
    x = z_gamma_median,
    y = 0.6,
```

```
    label = "Mediana",
    color = c_pal("C green"),
    hjust = 1.1
  ) +
  coord_cartesian(ylim = c(0, 0.65)) +
  labs(
    title = "Estandarizada",
    x = TeX("$X$"),
    y = "Densidad"
  ) +
  theme_krul()

# Combinamos ambas gráficas
combined_gamma_plot <- gamma_distribution_plot + z_gamma_distribution_plot +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Distribución Gamma: original y estandarizada",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

# Mostramos la gráfica combinada
combined_gamma_plot
```



Figura 3.1: Comparación de la función de densidad (PDF) de la distribución Gamma original y su versión estandarizada. Notemos que aunque la media y la desviación estándar cambian, la forma de la distribución permanece igual. En particular, se puede observar que la asimetría de la distribución no cambia.

3.2. Escalamiento de muestras

Para una muestra dada, no siempre se conoce la distribución poblacional subyacente. En particular, pueden desconocerse la media y la desviación estándar poblacional. Aun así, existen varios métodos comunes para escalar nuestros datos. Algunos de los más utilizados son:

1. **Estandarización.** Usa la media y la desviación estándar muestrales. En otras palabras, la muestra estandarizada está dada por

$$T = \frac{X - \bar{X}}{S},$$

donde $\bar{X}$ es la media muestral y S es la desviación estándar muestral.

2. **Escalamiento Min–Max.** En este caso la muestra escalada está dada por

$$T = \frac{X - \text{mín}(X)}{\text{máx}(X) - \text{mín}(X)},$$

donde $\text{mín}(X)$ y $\text{máx}(X)$ son el mínimo y máximo de la muestra. Este método de escalamiento es muy sensible a valores atípicos, ya que utiliza los valores extremos de la muestra.

3. **Escalamiento robusto.** Usa la mediana y el rango intercuartil (IQR). La muestra escalada es

$$T = \frac{X - \text{mediana}(X)}{\text{IQR}(X)},$$

Este método es más robusto frente a valores atípicos en comparación con el escalamiento Min–Max.

4. **Normalización.** Divide cada dato por la norma del vector. La muestra normalizada es

$$T = \frac{X}{\|X\|},$$

donde $\|X\|$ es la norma. Se usa con menor frecuencia que los anteriores.

Como ejemplo, consideraremos los distintos métodos de escalado aplicados a una muestra proveniente de una distribución Gamma.

```
# Para asegurar la reproducibilidad de los resultados
set.seed(123)

scale_lookup_tbl <- tibble(
  code = c(
    "sample",
    "sample_standard",
    "sample_min_max",
    "sample_robust",
    "sample_normal"
  ),
  label = c(
```

```
    "Muestra Original",
    "Estandarización",
    "Escalamiento Min-Max",
    "Escalamiento Robusto",
    "Normalización"
  )
)

# Tamaño de la muestra
n <- 3000

gamma_sample_tbl <- tibble(
  sample = rgamma(n, shape = alpha, rate = beta)
)

scaled_gamma_sample_tbl <- gamma_sample_tbl |>
  mutate(
    sample_standard = standard_scale(sample),
    sample_min_max = min_max_scale(sample),
    sample_robust = robust_scale(sample),
    sample_normal = normal_scale(sample)
  ) |>
  pivot_longer(
    cols = everything(),
    names_to = "scaling_method",
    values_to = "sample_scaled"
  ) |>
  convert_codes_to_factor(
    code_col = scaling_method,
    lookup_tbl = scale_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = scaling_method_factor
  )

scaled_gamma_qq_plot <- scaled_gamma_sample_tbl |>
  ggplot(aes(sample = sample_scaled)) +
  facet_wrap(
    ~scaling_method_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_qq(
    color = c_pal("C blue"),
    size = 0.3
```

```
  ) +
  geom_qq_line(
    color = c_pal("C magenta"),
    linewidth = 0.5
  ) +
  labs(
    title = "Gráficos QQ",
    x = "Cuantiles teóricos",
    y = "Cuantiles muestrales"
  ) +
  theme_krul()

scaled_gamma_histogram <- scaled_gamma_sample_tbl |>
  ggplot(aes(x = sample_scaled)) +
  facet_wrap(
    ~scaling_method_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Histogramas",
    x = "Valores muestrales",
    y = "Frecuencia absoluta"
  ) +
  theme_krul()

scaled_gamma_plot <- scaled_gamma_qq_plot + scaled_gamma_histogram +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Comparación de métodos de escalamiento aplicados a la muestra Gamma",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

scaled_gamma_plot
```

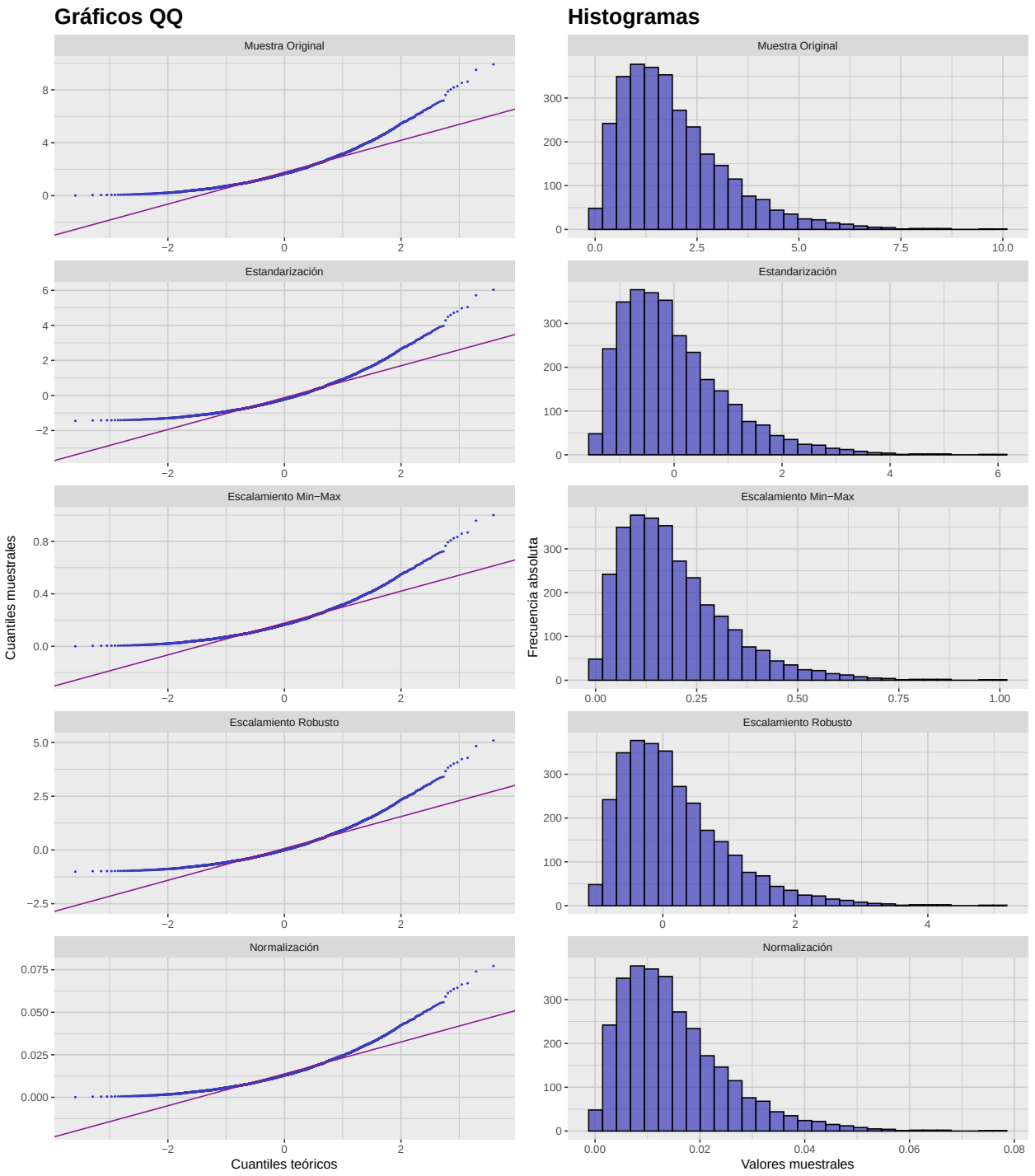


Figura 3.2: Comparación de distintos métodos de escalamiento aplicados a una muestra generada a partir de la distribución Gamma. Notemos que todos los gráficos QQ son versiones reescaladas del gráfico QQ original y, por tanto, tienen la misma forma. En particular, no es posible alterar la normalidad de una muestra al aplicar un método de escalamiento.

3.3. Transformaciones

Las transformaciones son otro método que nos permite modificar la manera en la que representamos la información contenida en una muestra. Sin embargo, a diferencia del escalamiento, las transformaciones si pueden alterar la forma de la distribución original y sus propiedades estadísticas. Una aplicación común de estas transformaciones es mejorar la normalidad de nuestros datos, lo cual es útil para cumplir con los supuestos de ciertos análisis estadísticos.

3.3.1. Familias de transformaciones

Usualmente las transformaciones que aplicamos a una muestra pertenecen a alguna familia de transformaciones que dependen de ciertos parámetros. Dos de las familias más comunes son:

1. **Transformaciones de Box–Cox.** Esta es una familia de transformaciones que se utiliza comúnmente para corregir asimetrías, pero solo es aplicable a datos positivos. Está dada por:

$$\tilde{X} = \begin{cases} \frac{X^\lambda - 1}{\lambda} & \text{si } \lambda \neq 0 \\ \log(X) & \text{si } \lambda = 0 \end{cases}$$

   donde λ es un parámetro.

2. **Transformaciones de Yeo–Johnson.** Similar a las transformaciones de Box–Cox, pero también son aplicables a datos negativos. Están dadas por:

$$\tilde{X} = \begin{cases} \frac{((X+1)^\lambda - 1)}{\lambda} & \text{si } X \geq 0, \lambda \neq 0 \\ \log(X+1) & \text{si } X \geq 0, \lambda = 0 \\ \frac{-((-X+1)^{2-\lambda} - 1)}{2-\lambda} & \text{si } X < 0, \lambda \neq 2 \\ \log(-X+1) & \text{si } X < 0, \lambda = 2 \end{cases}$$

   donde λ nuevamente es nuestro parámetro.

Como ilustración, aplicaremos una transformación de Box–Cox a la muestra Gamma que habíamos considerado anteriormente, utilizando los valores del parámetro $\lambda = 0$, 0.25, 0.5, 0.75 y 1:

```
# Generamos una tabla de búsqueda para los valores de lambda
# Esta tabla nos ayudará a etiquetar las facetas de las gráficas
# que generaremos más adelante
box_cox_lookup_tbl <- tibble(
  code = c(
    "box_cox_0",
    "box_cox_025",
    "box_cox_05",
    "box_cox_075",
    "box_cox_1"
  ),
  label = c(
    "lambda = 0",
    "lambda = 0.25",
```

```
    "lambda = 0.5",
    "lambda = 0.75",
    "lambda = 1"
  ),
)

# Generamos un tibble con las transformaciones de Box-Cox
# aplicadas a la muestra Gamma que ya teníamos
box_cox_gamma_sample_tbl <- gamma_sample_tbl |>
  mutate(
    box_cox_0 = bcPower(sample, lambda = 0),
    box_cox_025 = bcPower(sample, lambda = 0.25),
    box_cox_05 = bcPower(sample, lambda = 0.5),
    box_cox_075 = bcPower(sample, lambda = 0.75),
    box_cox_1 = bcPower(sample, lambda = 1)
  ) |>
  pivot_longer(
    starts_with("box_cox_"),
    names_to = "lambda",
    values_to = "transformed_sample"
  ) |>
  convert_codes_to_factor(
    code_col = lambda,
    lookup_tbl = box_cox_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = lambda_factor
  )

# Generamos las gráficas QQ. Notemos que estamos utilizando
# los diversos valores de lambda para etiquetar las facetas
box_cox_gamma_qq_plot <- box_cox_gamma_sample_tbl |>
  ggplot(aes(sample = transformed_sample)) +
  facet_wrap(
    ~lambda_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_qq(
    color = c_pal("C blue"),
    size = 0.3
  ) +
  geom_qq_line(
    color = c_pal("C magenta"),
    linewidth = 0.5
```

```
  ) +
  labs(
    title = "Gráficos QQ",
    x = "Cuantiles teóricos",
    y = "Cuantiles  muestrales"
  ) +
  theme_krul()

# Generamos los histogramas. Nuevamente estamos utilizando
# los diversos valores de lambda para etiquetar las facetas
box_cox_gamma_histogram <- box_cox_gamma_sample_tbl |>
  ggplot(aes(x = transformed_sample)) +
  facet_wrap(
    ~lambda_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Histogramas",
    x = "Valores muestrales",
    y = "Frecuencia absoluta"
  ) +
  theme_krul()

# Combinamos las gráficas
box_cox_gamma_plot <- (box_cox_gamma_qq_plot + box_cox_gamma_histogram) +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Transformaciones Box-Cox para distintos valores de lambda",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

# Mostramos la gráfica combinada
box_cox_gamma_plot
```

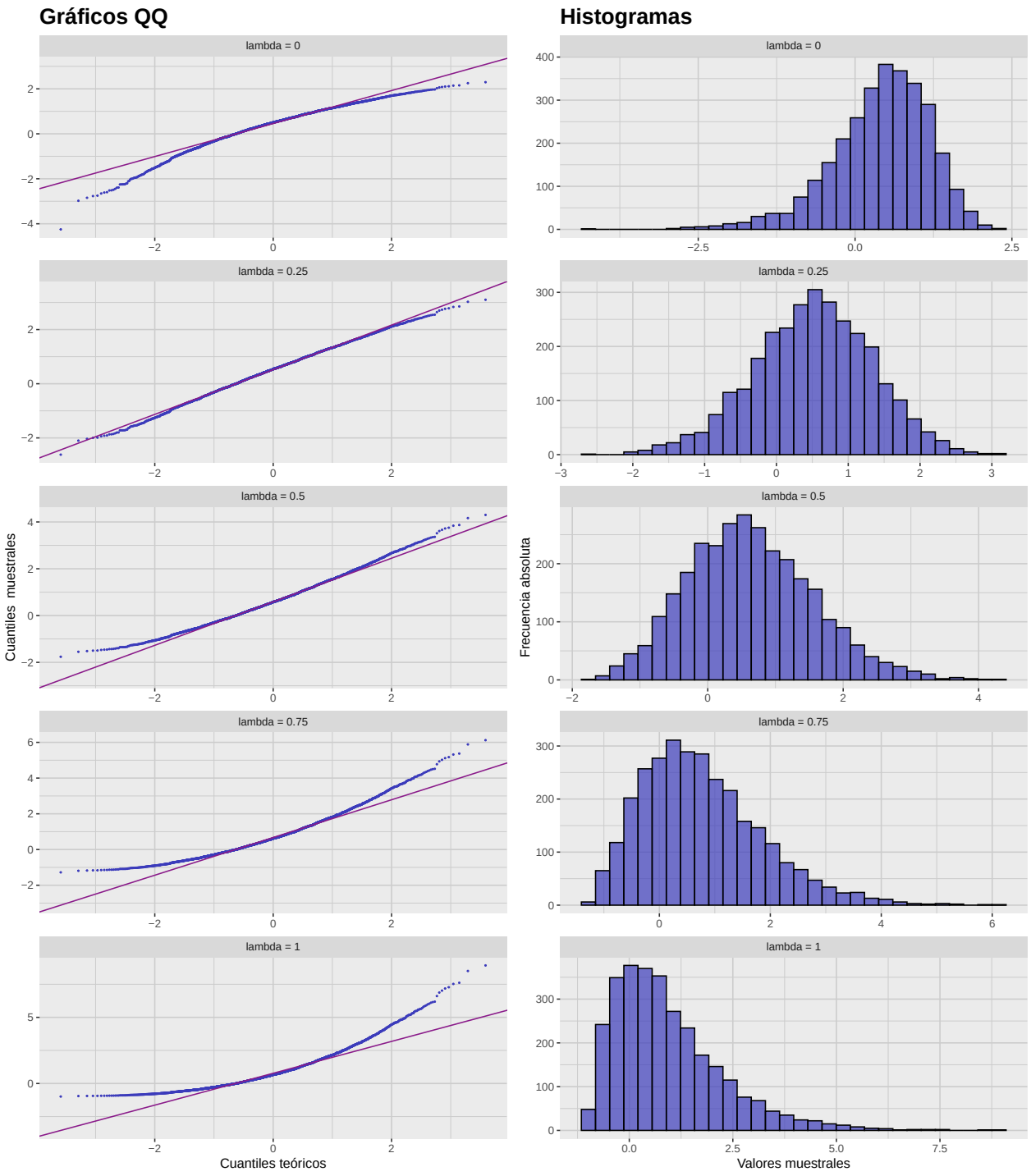


Figura 3.3: Comparación de distintas transformaciones de Box–Cox aplicadas a la muestra Gamma, con parámetro lambda = 0, 0.25, 0.5, 0.75 y 1. Notemos como la forma de los gráficos QQ e histogramas cambia significativamente con cada transformación, lo que indica que la normalidad de la muestra puede mejorarse o empeorarse dependiendo del valor de lambda seleccionado.

3.3.2. Perfil de log-verosimilitud y transformación óptima

Una manera de cuantificar la normalidad de una muestra es mediante el cálculo de su máxima verosimilitud bajo el supuesto de normalidad. (O, de manera equivalente, el cálculo de su máxima log-verosimilitud.) Recordemos que dada una muestra $X_1, X_2, \ldots, X_n$ proveniente de una distribución normal $\mathcal{N}(\mu, \sigma^2)$, su log-verosimilitud está dada por:

$$\ell(\mu, \sigma^2) = -\frac{n}{2}\log(2\pi) - n\log(\sigma) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(X_i - \mu)^2.$$

Notemos que este valor depende de los parámetros μ y σ^2, los cuales son desconocidos cuando trabajamos solamente con una muestra. Sin embargo, podemos estimar estos parámetros usando los estimadores de máxima verosimilitud:

$$\begin{aligned}\hat{\mu} =& \bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i,\\ \hat{\sigma}^2 =& \widetilde{S}^2 = \frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^2.\end{aligned}$$

Utilizando estos estimadores, podemos asignar a cada muestra el valor máximo de su log-verosimilitud:

$$\text{MLE} = \ell(\hat{\mu}, \hat{\sigma}^2) = -\frac{n}{2}\log(2\pi) - n\log(\hat{\sigma}) - \frac{n}{2}.$$

Entre mayor sea este valor, mejor será el ajuste de nuestra muestra a una distribución normal. Por otro lado, hemos visto que las transformaciones de Box–Cox pueden mejorar o empeorar la normalidad de una muestra dependiendo del valor del parámetro λ utilizado. Se sigue que, para sacar el mayor provecho posible a estas transformaciones, debemos buscar el valor de λ que maximice la log-verosimilitud de la muestra transformada, obteniendo así la *transformación óptima* de Box–Cox para nuestros datos.

Para ilustrar este proceso, en la siguiente figura mostraremos el perfil de log-verosimilitud para distintos valores de λ aplicados a la muestra Gamma que hemos estado utilizando, resaltando el valor óptimo de este parámetro:

```
# Las transformaciones de Box-Cox pueden verse como parte
# de un modelo lineal generalizado. En nuestro caso
# no hay variables explicativas, por lo que ajustamos
# un modelo con solo el intercepto
gamma_sample_lm_fit <- lm(
  sample ~ 1,
  data = gamma_sample_tbl
)

# Para completar el ajuste, estimamos el valor óptimo de lambda.
gamma_sample_bc_estimate <- powerTransform(
  gamma_sample_lm_fit,
  family = "bcPower"
)
```

```
# Extraemos el valor óptimo de lambda
gamma_sample_optimal_lambda <- gamma_sample_bc_estimate$lambda

# Extraemos el intervalo de confianza al 95% para lambda
gamma_sample_lambda_confint <- as.numeric(confint(
  gamma_sample_bc_estimate,
  level = 0.95
))

# Calculamos el perfil de log-verosimilitud
gamma_sample_bc_profile <- boxCox(
  gamma_sample_lm_fit,
  lambda = seq(-2, 2, 0.1),
  plotit = FALSE
)

# Generamos el tibble con los datos del perfil de log-verosimilitud
gamma_sample_bc_tbl <- tibble(
  lambda = gamma_sample_bc_profile$x,
  log_likehood = gamma_sample_bc_profile$y
)

# Generamos la gráfica del perfil de log-verosimilitud
gamma_sample_bc_plot <- gamma_sample_bc_tbl |>
  ggplot(aes(x = lambda, y = log_likehood)) +
  geom_line(
    color = c_pal("C blue"),
    linewidth = 1
  ) +
  geom_vline(
    xintercept = gamma_sample_optimal_lambda,
    linetype = "dashed",
    color = c_pal("C green"),
    linewidth = 1
  ) +
  geom_vline(
    xintercept = gamma_sample_lambda_confint,
    linetype = "dashed",
    color = c_pal("C red"),
    linewidth = 1
  ) +
  labs(
    title = "Perfil de log-verosimilitud para la transformación Box-Cox",
    x = TeX("$\\lambda$"),
```

```
    y = "MLE"
  ) +
  theme_krul()

# Mostramos la gráfica
gamma_sample_bc_plot
```

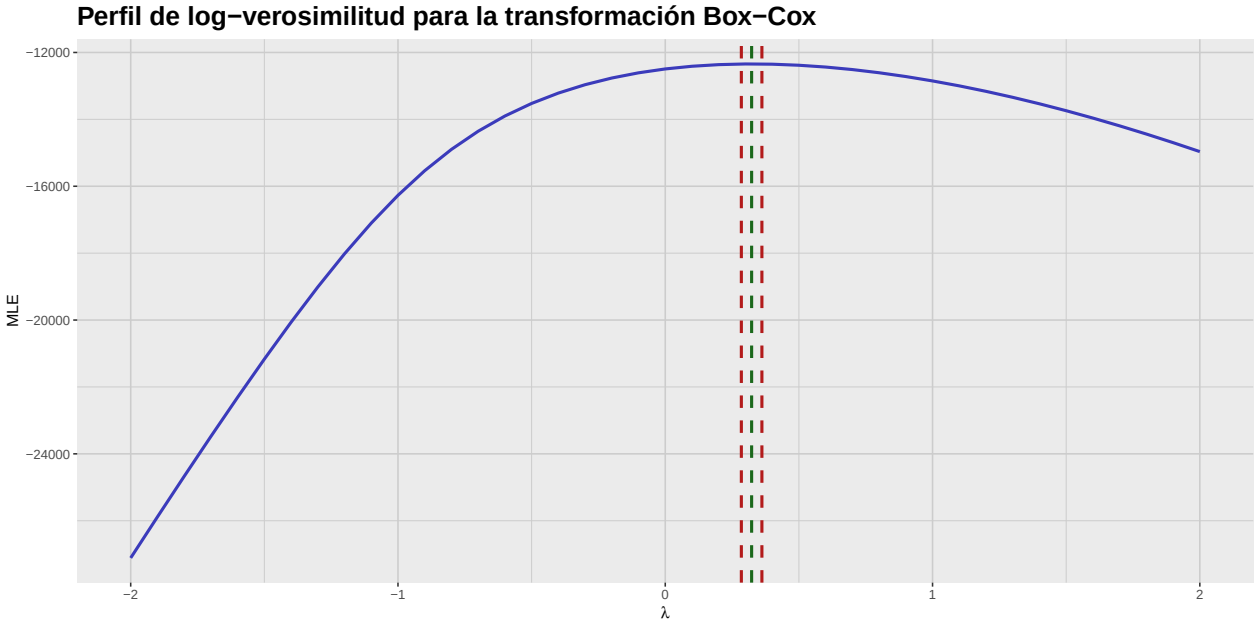


Figura 3.4: Perfil de log-verosimilitud para la transformación de Box–Cox aplicada a la muestra Gamma. El valor óptimo de lambda se indica con una línea discontinua verde, mientras que las líneas discontinuas rojas indican el intervalo de confianza al 95 % para lambda.

Con esta información, podemos comparar la muestra original con su transformación de Box–Cox utilizando el valor óptimo de λ:

```
# Generamos la tabla de búsqueda
box_cox_lookup_tbl <- tibble(
  code = c(
    "box_cox_1",
    "box_cox_optimal"
  ),
  label = c(
    "Muestra original",
    "Transformación óptima"
  )
)

# Generamos el tibble con la muestra original y transformada
```

```
box_cox_gamma_sample_tbl <- gamma_sample_tbl |>
  mutate(
    box_cox_1 = sample,
    box_cox_optimal = bcPower(sample, lambda = gamma_sample_optimal_lambda)
  ) |>
  pivot_longer(
    starts_with("box_cox_"),
    names_to = "lambda",
    values_to = "transformed_sample"
  ) |>
  convert_codes_to_factor(
    code_col = lambda,
    lookup_tbl = box_cox_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = lambda_factor
  )

# Generamos las gráficas QQ.
box_cox_gamma_qq_plot <- box_cox_gamma_sample_tbl |>
  ggplot(aes(sample = transformed_sample)) +
  facet_wrap(~lambda_factor, nrow = 2, scales = "free") +
  geom_qq(
    color = c_pal("C blue"),
    size = 0.3
  ) +
  geom_qq_line(
    color = c_pal("C magenta"),
    linewidth = 0.5
  ) +
  labs(
    title = "Gráficos QQ",
    x = "Cuantiles teóricos",
    y = "Cuantiles  muestrales"
  ) +
  theme_krul()

# Generamos los histogramas.
box_cox_gamma_histogram <- box_cox_gamma_sample_tbl |>
  ggplot(aes(x = transformed_sample)) +
  facet_wrap(~lambda_factor, nrow = 2, scales = "free") +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
```

```
    alpha = 0.7
  ) +
  labs(
    title = "Histogramas",
    x = "Valores muestrales",
    y = "Frecuencia absoluta"
  ) +
  theme_krul()

# Combinamos las gráficas
box_cox_gamma_plot <- (box_cox_gamma_qq_plot + box_cox_gamma_histogram) +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Comparación de la muestra original y su transformación óptima",
    theme = theme(plot.title = element_text(size = 24, face = "bold"))
  )

# Mostramos la gráfica combinada
box_cox_gamma_plot
```

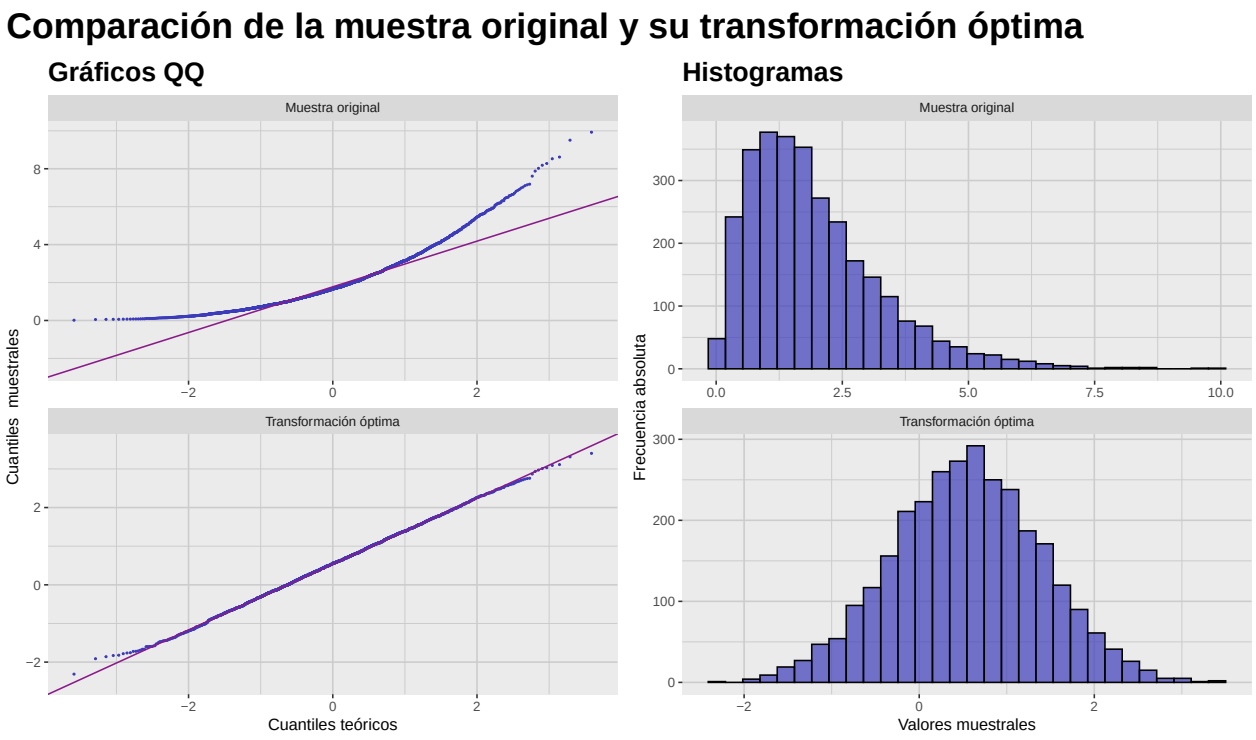


Figura 3.5: Comparación de la muestra Gamma original y su transformación de Box–Cox con el valor óptimo de lambda. Notemos la mejora significativa en la normalidad de los datos tras aplicar la transformación.

Esta figura muestra gráficamente la mejora significativa en la normalidad de la muestra tras aplicar la transformación de Box–Cox con el valor óptimo de λ. Esta misma mejora se puede cuantificar también aplicando las pruebas de normalidad que conocemos a la muestra original y a la transformada:

```
# Generamos el tibble con la muestra transformada
optimal_gamma_sample_tbl <- gamma_sample_tbl |>
  mutate(
    transformed_sample = bcPower(sample, lambda = gamma_sample_optimal_lambda)
  )

# Generamos la tabla de resultados de las pruebas de normalidad
# tanto para la muestra original como para la transformada
comparative_normality_results_tbl <- list(
  compute_normality_pvalues(
    data = gamma_sample_tbl,
    value_col = sample,
    sample_name = "Muestra original"
  ),
  compute_normality_pvalues(
    data = optimal_gamma_sample_tbl,
    value_col = transformed_sample,
    sample_name = "Transformación óptima"
  )
) |>
  reduce(full_join, by = "Prueba")

# Formateamos y mostramos la tabla redondeando a 4 decimales
kable(
  comparative_normality_results_tbl,
  format = "latex",
  booktabs = TRUE,
  digits = 4,
  align = c("l", rep("c", ncol(comparative_normality_results_tbl) - 1))
) |>
  kable_styling(latex_options = "hold_position")
```

Tabla 3.1: Tabla de valores p de las pruebas de normalidad aplicadas a la muestra original y a su transformación de Box–Cox con el valor óptimo de lambda. Notemos la mejora significativa en los valores p tras aplicar la transformación, lo que indica un mejor ajuste a la normalidad.

Prueba	Muestra original	Transformación óptima
Shapiro-Wilk	0	0.8248
Kolmogorov-Smirnov	0	0.9978
Anderson-Darling	0	0.9709
Jarque-Bera	0	0.5816

3.4. Actividad: Visualizando la no normalidad de muestras

Para cada una de las siguientes distribuciones, construya una muestra de tamaño 300 y calcule la asimetría y curtosis muestrales. Después, construya y compare sus gráficos QQ e histogramas.

1. Distribución exponencial con tasa $\lambda = 1$.
2. Distribución uniforme en $[0, 1]$.
3. Distribución beta con $\alpha = 2$ y $\beta = 5$.
4. Distribución de Cauchy con localización $x_0 = 0$ y escala $\gamma = 1$.
5. Distribución χ^2_k con $k = 3$ grados de libertad.

Parte II

Introducción al análisis multivariado

Capítulo 4

Introducción al análisis multivariante

En este capítulo introducimos conceptos básicos del análisis multivariante, enfocándonos en el vector de medias y la matriz de covarianzas de un vector aleatorio. Estos conceptos son fundamentales para entender relaciones entre múltiples variables en un conjunto de datos.

4.1. El vector de medias y la matriz de covarianzas

Definición 4.1.1. Sea Π una población y sea

$$X : \Pi \to \mathbb{R}^p$$

un vector aleatorio (fila). El *vector de medias* de X se define como

$$\mu = \mathbb{E}[X] = [\mathbb{E}[X_1], \mathbb{E}[X_2], \ldots, \mathbb{E}[X_p]] \tag{4.1}$$

donde X_i es la i-ésima componente del vector aleatorio X. La *matriz de covarianzas* de X se define como

$$\Sigma = \mathrm{Cov}[X] = \mathbb{E}[(X-\mu)^T(X-\mu)] = [\mathrm{Cov}[X_i, X_j]]_{i,j=1}^{p} \tag{4.2}$$

donde $\mathrm{Cov}[X_i, X_j] = \mathbb{E}[(X_i - \mathbb{E}[X_i])(X_j - \mathbb{E}[X_j])]$ es la covarianza entre las componentes i-ésima y j-ésima del vector aleatorio X.

Comentario 4.1.2. Recuerde que $\mathrm{Cov}[X_i, X_j] = \mathrm{Cov}[X_j, X_i]$, por lo que la matriz es simétrica. Además, $\mathrm{Cov}[X_i, X_i] = \mathrm{Var}[X_i]$, la varianza de la i-ésima componente de X. Por ello también se le llama *matriz varianza–covarianza*.

Ahora, dada una muestra de tamaño n de Π, su *matriz muestral* es de tamaño $n \times p$ y cada renglón contiene una observación del vector X. En este contexto, el vector de medias muestral y la matriz de covarianzas muestral se definen así:

Definición 4.1.3. Dada una muestra de tamaño n de la población Π, con matriz muestral asociada x, definimos su *vector de medias muestral* como

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i = \left[\frac{1}{n}\sum_{i=1}^{n} x_{i,1}, \frac{1}{n}\sum_{i=1}^{n} x_{i,2}, \ldots, \frac{1}{n}\sum_{i=1}^{n} x_{i,p}\right] \tag{4.3}$$

donde x_i es la fila i-ésima de la matriz muestral x y $x_{i,j}$ es el componente j-ésimo de la fila i-ésima. La *matriz de covarianzas muestral* se define como

$$\hat{\Sigma} = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^T(x_i - \bar{x}) \tag{4.4}$$

donde $\hat{\Sigma}$ es una matriz $p \times p$ y $\hat{\Sigma}i, j = \frac{1}{n-1}\sum k = 1^n(x_{k,i} - \bar{x}i)(xk, j - \bar{x}_j)$ es la covarianza muestral entre las componentes i-ésima y j-ésima del vector aleatorio X.

Comentario 4.1.4. La matriz de covarianzas muestral es un estimador insesgado de la matriz de covarianzas de X.

Ejemplo 4.1.5. Considere la siguiente matriz muestral de tamaño $n = 5$ de una población Π:

$$\begin{bmatrix} 2 & 5 & 7 \\ 4 & 3 & 6 \\ 6 & 4 & 5 \\ 8 & 2 & 4 \\ 10 & 1 & 3 \end{bmatrix}$$

Para obtener el vector de medias muestral, calcule:

$$\bar{x}_1 = \frac{2+4+6+8+10}{5} = 6$$
$$\bar{x}_2 = \frac{5+3+4+2+1}{5} = 3$$
$$\bar{x}_3 = \frac{7+6+5+4+3}{5} = 5$$

En otras palabras,

$$\bar{x} = [6, 3, 5]$$

La varianza muestral de X_j es:

$$\hat{\Sigma}_{X_j} = \frac{1}{n-1}\sum_{i=1}^{n}(x_{ij} - \bar{x_j})^2$$

Por lo tanto,

$$\hat{\Sigma}_{X_1} = \frac{(2-6)^2 + (4-6)^2 + (6-6)^2 + (8-6)^2 + (10-6)^2}{4} = 10$$
$$\hat{\Sigma}_{X_2} = \frac{(5-3)^2 + (3-3)^2 + (4-3)^2 + (2-3)^2 + (1-3)^2}{4} = 2.5$$
$$\hat{\Sigma}_{X_3} = \frac{(7-5)^2 + (6-5)^2 + (5-5)^2 + (4-5)^2 + (3-5)^2}{4} = 2.5$$

La covarianza muestral entre X_j y X_k es:

$$\hat{\Sigma}_{X_j,X_k} = \frac{1}{n-1}\sum_{i=1}^{n}(x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$$

Realizando los cálculos correspondientes, obtenemos:

$$\begin{aligned}
\hat{\Sigma}_{X_1,X_2} &= \frac{(2-6)(5-3)+(4-6)(3-3)+(6-6)(4-3)+(8-6)(2-3)+(10-6)(1-3)}{4} \\
&= \frac{(-8)+0+0+(-2)+(-8)}{4} = -4.5 \\
\hat{\Sigma}_{X_1,X_3} &= \frac{(2-6)(7-5)+(4-6)(6-5)+(6-6)(5-5)+(8-6)(4-5)+(10-6)(3-5)}{4} \\
&= \frac{(-8)+(-2)+0+(-2)+(-8)}{4} = -5 \\
\hat{\Sigma}_{X_2,X_3} &= \frac{(5-3)(7-5)+(3-3)(6-5)+(4-3)(5-5)+(2-3)(4-5)+(1-3)(3-5)}{4} \\
&= \frac{4+0+0+1+4}{4} = 2.25
\end{aligned}$$

La matriz de covarianzas resultante es:

$$\hat{\Sigma} = \begin{bmatrix} 10 & -4.5 & -5 \\ -4.5 & 2.5 & 2.25 \\ -5 & 2.25 & 2.5 \end{bmatrix}$$

4.2. La distancia de Mahalanobis

Definición 4.2.1. Supongamos que tenemos una población Π con un vector aleatorio X que tiene vector de medias μ y matriz de covarianzas Σ. Si Σ es invertible, entonces la *distancia de Mahalanobis* entre dos puntos x e y en $\mathbb{R}^p$ se define como:

$$d_M(x,y) = \sqrt{(x-y)\Sigma^{-1}(x-y)^T} \tag{4.5}$$

En el contexto de datos muestrales, la distancia de Mahalanobis puede calcularse usando la matriz de covarianzas muestral $\hat{\Sigma}$:

Definición 4.2.2. Dada una muestra de tamaño n de la población Π, con matriz muestral asociada x y matriz de covarianzas muestral $\hat{\Sigma}$, la *distancia de Mahalanobis muestral* entre dos puntos x_i y x_j se define como:

$$d_M(x_i,x_j) = \sqrt{(x_i-x_j)\hat{\Sigma}^{-1}(x_i-x_j)^T} \tag{4.6}$$

Comentario 4.2.3. La distancia de Mahalanobis mide la separación entre dos puntos en un espacio multivariante, tomando en cuenta las correlaciones entre variables. Es particularmente útil para identificar atípicos en datos multivariantes.

Para ilustrar, consideremos el siguiente conjunto de datos. Iniciamos con una muestra aleatoria de tamaño $n = 300$ de una uniforme en $[0, 10]$.

```
set.seed(123)
n <- 300
x_data <- runif(n, min = 0, max = 10)
```

Construimos ahora un vector de error ϵ normal con media 0 y desviación 1:

```
epsilon_data <- rnorm(n, mean = 0, sd = 1)
```

Podemos entonces formar el vector

```
y_data <- x_data + epsilon_data
```

y crear el tibble:

```
sample_data_tbl <- tibble(
  x = x_data,
  y = y_data
)
```

La dispersión asociada se muestra a continuación:

```
library(ggforce)
library(mvtnorm)
library(ellipse)
```

```
Attaching package: 'ellipse'

The following object is masked from 'package:car':

    ellipse

The following object is masked from 'package:graphics':

    pairs
```

```
sample_data_plot <- sample_data_tbl %>%
  ggplot(aes(x = x_data, y = y_data)) +
  geom_point(
    color = "black",
    size = 0.2,
    alpha = 1
  ) +
  labs(
    title = "Datos de muestra",
    x = "X",
    y = "Y"
  ) +
  coord_fixed() +
  theme_krul()

sample_data_plot
```

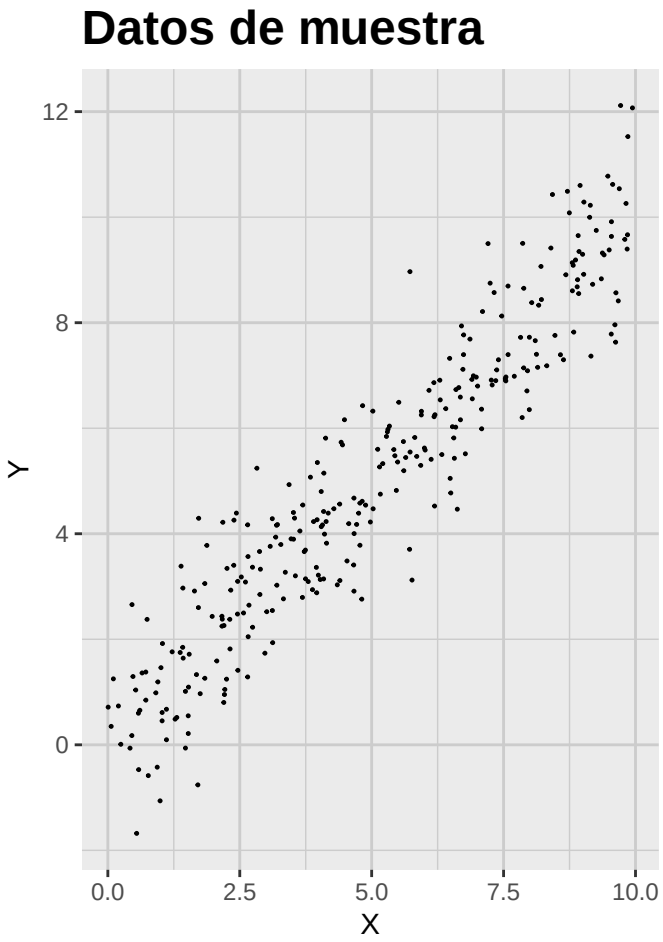


Figura 4.1: Diagrama de dispersión de los datos de muestra. Nótese la alta correlación entre las variables X e Y.

Ahora añadimos dos puntos extra a esta gráfica:

```
extra_points_tbl <- tibble(
  x_extra = c(1, 5),
  y_extra = c(1, 2)
)

extra_sample_data_plot <- sample_data_plot +
  geom_point(
    data = extra_points_tbl,
    aes(x = x_extra, y = y_extra),
    color = c_pal("C red"),
    size = 1.5
  )

extra_sample_data_plot
```

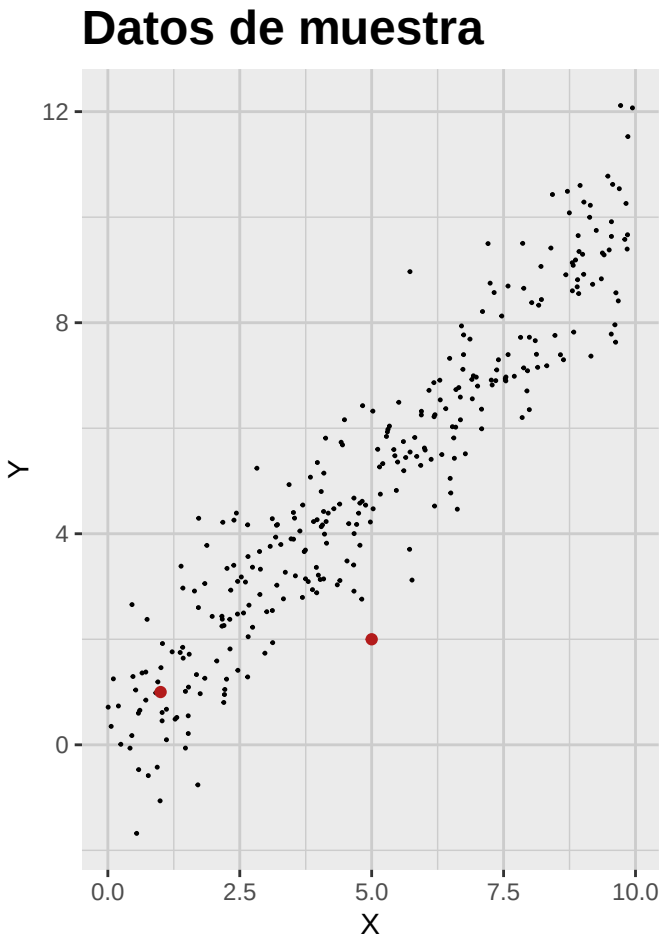


Figura 4.2: Dispersión con puntos adicionales. Nótese que uno de ellos es claramente atípico.

Nótese que uno de estos puntos es claramente un atípico. Nos gustaría tener una medida concreta para mostrarlo, así que calculamos el vector de medias muestral (centro de los datos). Para referencia futura, también calculamos la matriz de covarianzas muestral.

```
sample_mean_vector <- colMeans(sample_data_tbl)
sample_covariance_matrix <- cov(sample_data_tbl)

sample_mean_tbl <- tibble(
  x_bar = sample_mean_vector[1],
  y_bar = sample_mean_vector[2]
)
```

Si usamos la distancia euclídea, obtenemos que el punto atípico parece más cercano a la media muestral:

```
euclidean_sample_data_plot <- extra_sample_data_plot +
  geom_point(
    data = sample_mean_tbl,
```

```
    aes(x = x_bar, y = y_bar),
    color = c_pal("C red"),
    size = 1.5
  ) +
  geom_circle(
    aes(x0 = sample_mean_vector[1], y0 = sample_mean_vector[2], r = 4),
    color = c_pal("C blue"),
    linewidth = 0.6,
    fill = NA
  )

euclidean_sample_data_plot
```

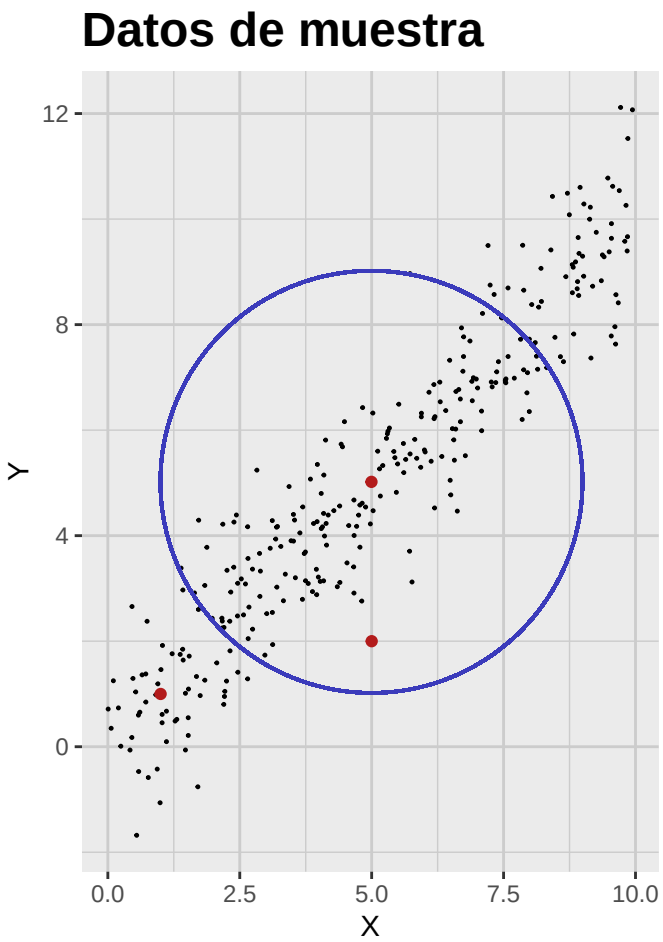


Figura 4.3: Dispersión con vector de media muestral y un círculo de distancia euclidiana constante al centro de los datos.

Por otra parte, si calculamos la distancia de Mahalanobis, obtenemos que el punto atípico está en realidad más alejado del vector de medias muestral:

```
annotated_sample_data_plot <- extra_sample_data_plot +
  geom_point(
    data = sample_mean_tbl,
    aes(x = x_bar, y = y_bar),
    color = c_pal("C red"),
    size = 1.5
  ) +
  stat_ellipse(
    type = "norm",
    level = 0.8647,
    color = c_pal("C blue"),
    linewidth = 0.6
  )

annotated_sample_data_plot
```

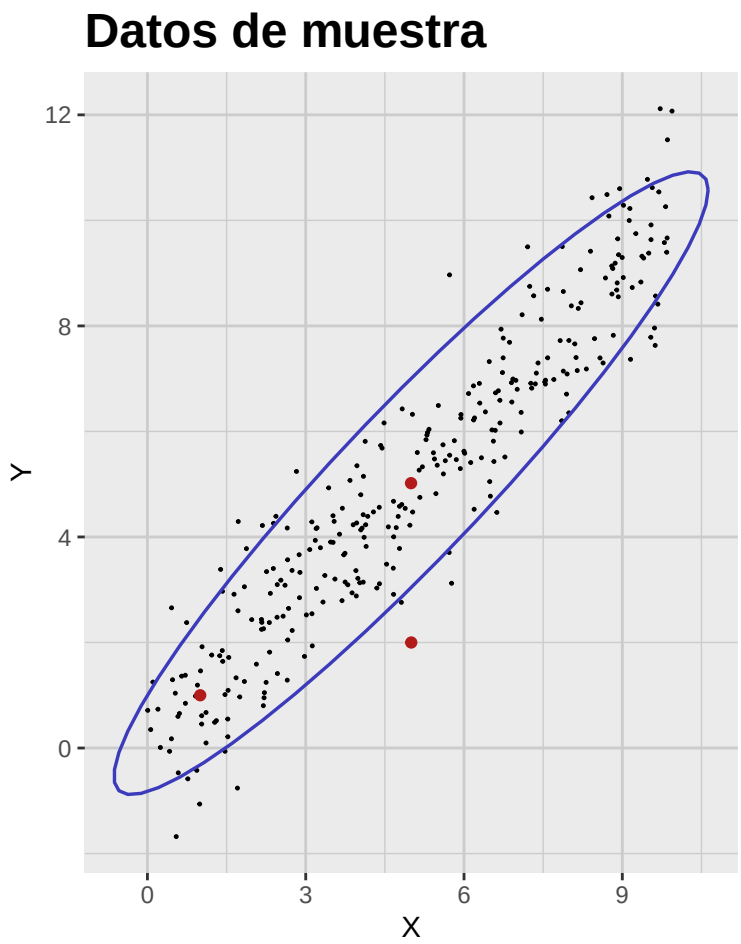


Figura 4.4: Dispersión con vector de media muestral y una elipse de distancia de Mahalanobis constante al centro de los datos.

4.3. Escalamiento y estandarización

Suponga ahora que tenemos un vector aleatorio X con media μ y covarianzas Σ. Definimos un nuevo vector Y como:

$$Y^T = AX^T + b$$

donde A es una matriz $p \times p$ y b un vector de dimensión p. El vector de medias y la matriz de covarianzas de Y se obtienen como:

$$\mu_Y^T = \mathbb{E}[Y]^T = A\mu^T + b$$

y

$$\Sigma_Y = \text{Cov}[Y] = A\Sigma A^T$$

Más adelante mostraremos que siempre es posible encontrar A y b tales que $\mu_Y = 0$ y $\Sigma_Y = I_p$ (identidad $p \times p$). A este proceso se le conoce como *estandarización* de X.

4.4. Actividad: La geometría de los datos multivariantes

Parte A: Ejercicios teóricos

1. Demuestre que la distancia de Mahalanobis se reduce a la euclídea si la covarianza es la identidad.
2. Muestre que la distancia de Mahalanobis es invariante bajo transformaciones afines.
3. Muestre que al estandarizar cada variable, la matriz de covarianzas pasa a ser la matriz de correlaciones.
4. Si X_1 y X_2 son normales estándar independientes, muestre que $d_M(X, 0)^2$ sigue una ji-cuadrada con 2 g.l., donde d_M es la distancia al origen.

Parte B: Ejercicios prácticos

1. Genere un conjunto 2D de 50 puntos (dos variables correlacionadas).
2. Calcule el vector de medias muestral y la matriz de covarianzas.
3. Escriba una función en R que calcule la distancia de Mahalanobis de cada punto a la media.
4. Identifique el punto con mayor distancia de Mahalanobis.
5. Grafique el conjunto con `ggplot2`.
6. Dibuje elipses de distancia de Mahalanobis 1, 2 y 3.
7. Coloree puntos según si su distancia supera 1, 2 o 3.
8. Cree un conjunto de 3 variables.
9. Calcule distancias de Mahalanobis en 3D.
10. Grafique pares de variables con elipses proyectadas.
11. Compare distancias euclídeas a la media con distancias de Mahalanobis.

II. Introducción al análisis multivariado

Capítulo 5

Distribución normal multivariante

5.1. Definición y propiedades básicas

Definición 5.1.1. Sea μ un vector (renglón) y Σ una matriz simétrica definida positiva. Decimos que un vector aleatorio (renglón) X tiene *distribución normal multivariante* con media μ y matriz de covarianzas Σ, en símbolos $X \sim \mathcal{N}_p(\mu, \Sigma)$, si su densidad es

$$\frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(X-\mu)\Sigma^{-1}(X-\mu)^T\right) dX$$

Comentario 5.1.2. Algunas observaciones sobre la definición:

1. Al igual que en el caso univariado, la normal multivariante queda determinada por μ y Σ.
2. Σ debe ser simétrica y definida positiva, garantizando que la distribución esté bien definida y el exponente sea negativo definido.
3. El término
$$|\Sigma| = \det(\Sigma)$$
es la *varianza generalizada*. Puede interpretarse como una medida de la "dispersión" en el espacio de dimensión p.
4. El término $(X-\mu)\Sigma^{-1}(X-\mu)^T$ es el cuadrado de la distancia de Mahalanobis entre X y la media μ.

Como en el caso univariado, la normal multivariante se destaca por el Teorema Central del Límite (TCL), que en términos generales afirma que la suma de muchas variables i.i.d. (debidamente estandarizadas) converge a una normal. El enunciado preciso es:

Teorema 5.1.3 (Teorema central del límite multivariante)**.** *Sea $\{X_i\}_{i=1}^{\infty}$ una sucesión de vectores aleatorios independientes e idénticamente distribuídos en $\mathbb{R}^p$ con vector de medias μ y matriz de covarianzas Σ. Definimos la media muestral como*

$$\bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i.$$

Entonces, cuando $n \to \infty$*, tenemos que*

$$\sqrt{n}\,(\bar{X}_n - \mu) \xrightarrow{d} \mathcal{N}_p(0, \Sigma),$$

donde $\xrightarrow{d}$ *denota convergencia en distribución y* $\mathcal{N}_p(0, \Sigma)$ *es la normal multivariante p-dimensional con media* 0 *y covarianzas* Σ*.*

Comentario 5.1.4. Aun cuando el TCL destaca a la normal multivariante, no es la única relevante en estadística multivariante. De hecho, muchos datos reales no siguen normalidad, y existen métodos para manejarla; además, técnicas como regresión lineal, PCA y clustering no arrojan buenos resultados aplicadas a una normal multivariante.

5.2. Contornos de la normal multivariante

Observe que la densidad de la normal multivariante es gaussiana en p dimensiones. Por ello, sus contornos son elipsoides centrados en μ y su forma está controlada por Σ. La probabilidad de estos contornos está gobernada por la χ^2 así: sea $0 \leq \varpi \leq 1$ y c tal que

$$\mathbb{P}(\chi_p^2 \leq c) = \varpi$$

Entonces, el contorno que contiene proporción ϖ de masa de probabilidad es el conjunto de X que satisfacen

$$(X - \mu)\Sigma^{-1}(X - \mu)^T = c$$

En la figura siguiente se muestran contornos de normales bivariantes con distintas covarianzas. Son elipses centradas en $\mu = (0, 0)$; el gradiente de color representa la densidad (más oscuro, mayor densidad).

```r
# Add seed for reproducibility
set.seed(123)

# Parameters
mu <- c(0, 0)
Sigma_1 <- matrix(c(1, 0, 0, 1), nrow = 2)
Sigma_2 <- matrix(c(0.3, 0, 0, 1.5), nrow = 2)
Sigma_3 <- matrix(c(1, 0.6, 0.6, 1), nrow = 2)
Sigma_4 <- matrix(c(1, -0.6, -0.6, 1), nrow = 2)



# Function to generate contour + scatter plot
plot_bivariate_normal <- function(mu, Sigma, n_samples = 1000,
                                  xlim = c(-3, 3), ylim = c(-3, 3),
                                  grid_length = 100) {
  # Create grid
  x <- seq(xlim[1], xlim[2], length.out = grid_length)
  y <- seq(ylim[1], ylim[2], length.out = grid_length)
  grid <- expand.grid(x = x, y = y)
```

```
  grid$z <- dmvnorm(grid, mean = mu, sigma = Sigma)

  # Sample points
  samples <- rmvnorm(n_samples, mean = mu, sigma = Sigma)
  colnames(samples) <- c("x", "y")
  samples <- as_tibble(samples)

  # Plot
  normal_plot <- ggplot() +
    geom_contour(data = grid, aes(x, y, z = z, color = after_stat(level))) +
    geom_point(
      data = samples,
      aes(x, y),
      color = c_pal("C red"),
      alpha = 0.5,
      size = 0.1
    ) +
    scale_color_viridis_c(option = "mako") +
    coord_cartesian(xlim = xlim, ylim = ylim) +
    labs(
      title = "",
      x = expression(X[1]), y = expression(X[2]),
      color = "Densidad"
    ) +
    theme_krul()

  return(normal_plot)
}

normal_plot_1 <- plot_bivariate_normal(mu, Sigma_1)
normal_plot_2 <- plot_bivariate_normal(mu, Sigma_2)
normal_plot_3 <- plot_bivariate_normal(mu, Sigma_3)
normal_plot_4 <- plot_bivariate_normal(mu, Sigma_4)

normal_plot <- (normal_plot_1 + normal_plot_2) /
  (normal_plot_3 + normal_plot_4) +
  plot_layout(widths = c(1, 1), heights = c(1, 1)) +
  plot_annotation(
    title = "Contornos de la normal bivariante ",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
```

```
  )

normal_plot
```

Contornos de la normal bivariante

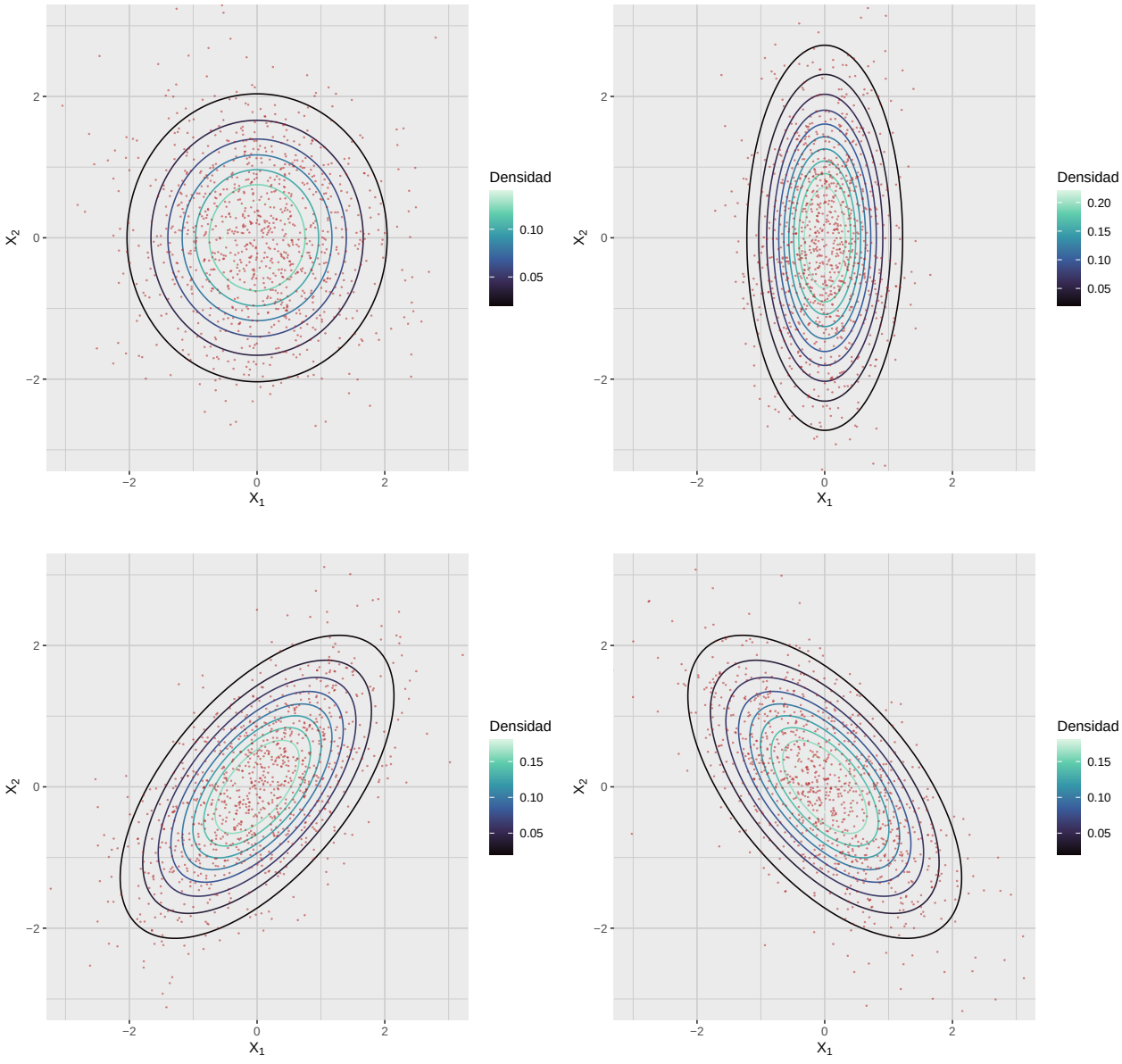


Figura 5.1: Distribución normal bivariante con distintas matrices de covarianzas

5.3. Estandarización de la normal multivariante

Al igual que en el caso univariado, si $X \sim \mathcal{N}_p(\mu, \Sigma)$ podemos estandarizar para obtener $Z \sim \mathcal{N}_p(0, I_p)$, donde I_p es la identidad de tamaño p. Para describirlo, recordemos algunos resultados de álgebra lineal.

Teorema 5.3.1 (Descomposición en valores singulares)**.** *Sea Σ una matriz simétrica definida positiva de dimensión $p \times p$. Entonces, existe una matriz ortogonal Q y una matriz diagonal Λ con entradas positivas tal que*

$$\Sigma = Q\Lambda Q^T$$

Nota 5.3.2. Recordemos que una matriz Q es ortogonal si $Q^TQ = I_p$; sus columnas son ortonormales y forman una base de $\mathbb{R}^p$.

Comentario 5.3.3. La descomposición en valores singulares (SVD) permite descomponer una matriz en valores (diagonal de Λ, también *autovalores*) y vectores singulares (columnas de Q, *autovectores*). Es útil para entender la geometría de la normal multivariante.

Como Σ es definida positiva, las entradas diagonales de

$$\Lambda = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_p \end{bmatrix}$$

son todas positivas. En particular, podemos definir una nueva matriz

$$D = \begin{bmatrix} \sqrt{\lambda_1} & 0 & \cdots & 0 \\ 0 & \sqrt{\lambda_2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sqrt{\lambda_p} \end{bmatrix}$$

que satisface la condición $D^2 = \Lambda$. Si ahora tomamos

$$\sigma = DQ^T$$

entonces

$$\Sigma = \sigma^T\sigma$$

En particular, podemos definir la *standarización* del vector aleatorio X como

$$Z = (X - \mu)\sigma^{-1} = (X - \mu)QD^{-1} = (X - \mu)Q\Lambda^{-1/2}$$

en cuyo caso $Z \sim \mathcal{N}_p(0, I_p)$.

5.4. Interpretación Geométrica de la SVD

Anteriormente en este capítulo consideramos la distribución normal multivariante con media

$$\mu = [0, 0]$$

y matriz de covarianzas

$$\Sigma = \begin{bmatrix} 1 & 0.6 \\ 0.6 & 1 \end{bmatrix}$$

De acuerdo con la SVD, podemos escribir esta matriz de covarianzas como

$$\Sigma = Q\Lambda Q^T$$

para alguna matriz ortogonal Q y diagonal Λ. En este caso, la SVD de Σ se calcula así:

```
# Example mean and covariance
mu <- c(0, 0)
Sigma <- matrix(c(
  1, 0.6,
  0.6, 1
), 2, 2)


# Compute eigenvectors and eigenvalues
eig <- eigen(Sigma)
vectors <- eig$vectors
values <- eig$values
```

Obtenemos:

$$Q = \begin{bmatrix} 0.7071068 & -0.7071068 \\ 0.7071068 & 0.7071068 \end{bmatrix}$$

y

$$\Lambda = \begin{bmatrix} 1.6 & 0 \\ 0 & 0.4 \end{bmatrix}$$

donde los vectores singulares son las columnas de Q y los valores singulares son las entradas diagonales de Λ. Geométricamente, los vectores singulares representan las direcciones de los ejes del elipsoide que define los contornos de la normal multivariante, y las raíces cuadradas de los valores singulares representan las longitudes de dichos ejes.

```
# Scale eigenvectors by sqrt(eigenvalues) for plotting
vec_data <- tibble(
  x0 = mu[1],
  y0 = mu[2],
  x1 = mu[1] + sqrt(values) * vectors[1, ],
  y1 = mu[2] + sqrt(values) * vectors[2, ]
)

# Generate points of the 1-Mahalanobis-distance ellipse
ellipse_points <- as_tibble(ellipse(Sigma, centre = mu, level = 0.393))
# level = 0.393 corresponds to ~1 standard deviation for 2D
```

```
# Plot
ggplot() +
  geom_path(
    data = ellipse_points,
    aes(x = x, y = y),
    color = c_pal("C blue"),
    linewidth = 1
  ) +
  geom_segment(
    data = vec_data,
    aes(x = x0, y = y0, xend = x1, yend = y1),
    arrow = arrow(length = unit(0.2, "cm")),
    color = c_pal("C green"),
    linewidth = 1
  ) +
  geom_point(
    aes(x = mu[1], y = mu[2]),
    color = c_pal("C red"),
    size = 3
  ) +
  coord_equal() +
  labs(
    title = "Interpretación geométrica de la SVD",
    x = expression(X[1]),
    y = expression(X[2])
  ) +
  theme_krul()
```

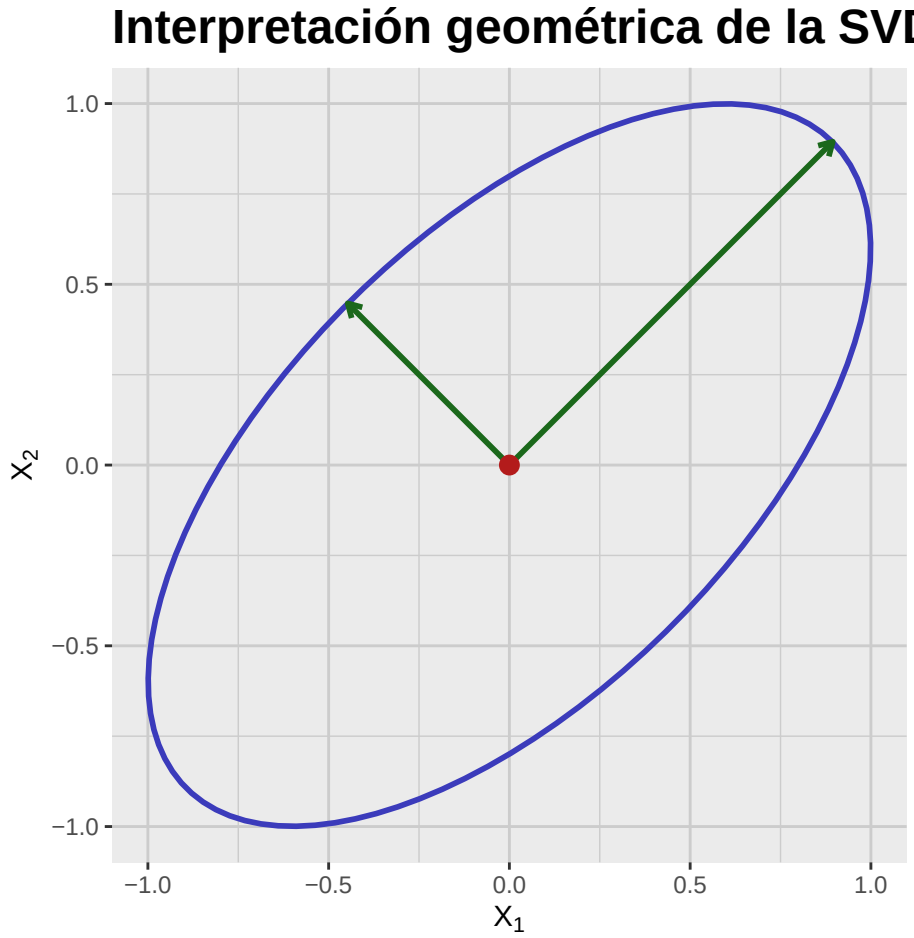


Figura 5.2: Interpretación geométrica de la SVD de una normal bivariante. El punto rojo es la media, las flechas verdes son los eigenvectores escalados por la raíz de los eigenvalores y la elipse azul es el contorno de distancia de Mahalanobis igual a 1.

5.5. Clausura bajo transformaciones lineales

Teorema 5.5.1 (Clausura bajo transformaciones lineales)**.** *Sea* $X \sim \mathcal{N}_p(\mu, \Sigma)$ *un vector con distribución normal multivariante. Para cualquier matriz* A *de tamaño* $m \times p$ *y vector* b *de tamaño* m*, el vector aleatorio*

$$Y = AX + b$$

también tiene distribución normal multivariante: $Y \sim \mathcal{N}_m(A\mu + b, A\Sigma A^T)$.

Comentario 5.5.2. Este teorema afirma que si aplicamos una transformación lineal a un vector con distribución normal multivariante, el resultado también es normal multivariante. Las expresiones de media y covarianza se dan en el enunciado. Es especialmente útil en aplicaciones como regresión (manteniendo normalidad de residuos) y para derivar distribuciones de combinaciones lineales.

Ejemplo 5.5.3. Sea $X \sim \mathcal{N}_2(\mu, \Sigma)$ un vector aleatorio con media $\mu = (1, 2)$ y matriz de covarianzas

$$\Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 2 \end{bmatrix}$$

Considere la transformación lineal

$$Y = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} X + \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Es decir,

$$\begin{aligned} Y_1 &= 2X_1 + X_2 \\ Y_2 &= 3X_2 + 1 \end{aligned}$$

Usando el teorema de clausura bajo transformaciones lineales, obtenemos la media y la covarianza de Y:

$$\text{Mean: } A\mu + b = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 7 \end{bmatrix}$$

$$\text{Covariance: } A\Sigma A^T = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 & 0.5 \\ 0.5 & 2 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 8 & 9 \\ 9 & 18 \end{bmatrix}$$

Así,

$$Y \sim \mathcal{N}_2 \left(\begin{bmatrix} 4 \\ 7 \end{bmatrix}, \begin{bmatrix} 8 & 9 \\ 9 & 18 \end{bmatrix} \right)$$

Comentario 5.5.4. Nótese que la proyección a un subespacio es lineal, por lo que el teorema de clausura implica que cualquier subconjunto de componentes de una normal multivariante también es normal multivariante. Esto permite analizar subconjuntos de variables manteniendo la suposición de normalidad.

5.6. Pruebas de normalidad multivariante

Existen varias pruebas para verificar si un conjunto sigue normalidad multivariante. Son esenciales, pues muchos métodos la asumen. Algunas de las más usadas y su implementación en R:

1. **Prueba de Mardia:** Basada en asimetría y curtosis multivariantes. Para normalidad multivariante, la asimetría debe ser cercana a 0 y la curtosis a $p(p+2)$, donde p es el número de variables. En R: `MVN::mvn(datos, mvn_test = "mardia")`.
2. **Prueba de Henze–Zirkler (HZ):** Usa la función característica empírica para medir desviaciones de la normalidad. Tiene buen poder global y es adecuada en dimensiones moderadas. En R: `MVN::mvn(datos, mvn_test = "hz")`.
3. **Prueba de Royston:** Extensión multivariante de Shapiro–Wilk, basada en el producto de estadísticas univariantes aplicadas a componentes principales. Fácil de interpretar; el poder disminuye en alta dimensión. En R: `MVN::mvn(datos, mvn_test = "royston")`.

4. **Prueba de Doornik–Hansen:** Combina asimetría y curtosis en una prueba ómnibus, alternativa robusta a Mardia. En R: `MVN::mvn(datos, mvn_test = "dh")`.
5. **Métodos gráficos (Q–Q chi-cuadrado):** Graficar las distancias de Mahalanobis contra cuantiles de una chi-cuadrada; desviaciones de la recta indican no normalidad multivariante. En R: `multivariate_diagnostic_plot(data = datos, type = "qq")`.

Capítulo 6

Análisis de componentes principales

El Análisis de Componentes Principales (PCA) es una técnica potente para reducir la dimensionalidad preservando la mayor información posible (medida por la varianza). Transforma las variables originales en un nuevo conjunto de variables no correlacionadas llamadas componentes principales, ordenadas por la varianza que explican.

6.1. Entendiendo la varianza total

Sea Σ la matriz de covarianzas del conjunto. La *varianza total* se da por la traza:

$$\text{Total Variance} = \operatorname{Tr}\Sigma = \sum_{i=1}^{p} \Sigma_{ii},$$

donde Σ_{ii} son las varianzas muestrales de cada variable. Usando la descomposición en valores/vectores propios (SVD) de Σ, existen matrices Q y Λ tales que:

$$\Sigma = Q\Lambda Q^T,$$

siendo Λ diagonal con los valores propios $\lambda_1, \ldots, \lambda_p$. Entonces la varianza total puede escribirse como:

$$\text{Total Variance} = \operatorname{Tr}\Sigma = \operatorname{Tr} Q\Lambda Q^T = \operatorname{Tr}\Lambda = \lambda_1 + \cdots + \lambda_p.$$

Las columnas de Q son las *direcciones de los componentes principales*; los valores de Λ indican cuánta varianza explica cada componente. El primero (mayor valor propio) capta la mayor varianza, el segundo la siguiente, y así sucesivamente. Estos componentes inducen un nuevo sistema de coordenadas; cada punto puede representarse por sus *puntuaciones* de componentes, relacionadas con las variables por:

$$X = Q\,(PC) + \bar{x}$$

y

$$PC = Q^T(X - \bar{x}) \tag{6.1}$$

Aquí X es el vector en coordenadas originales, PC el de puntuaciones, y $\bar{x}$ el vector de medias.

Para ilustrar estas ideas, consideramos el conjunto de la figura:

```
sample_data_plot
```

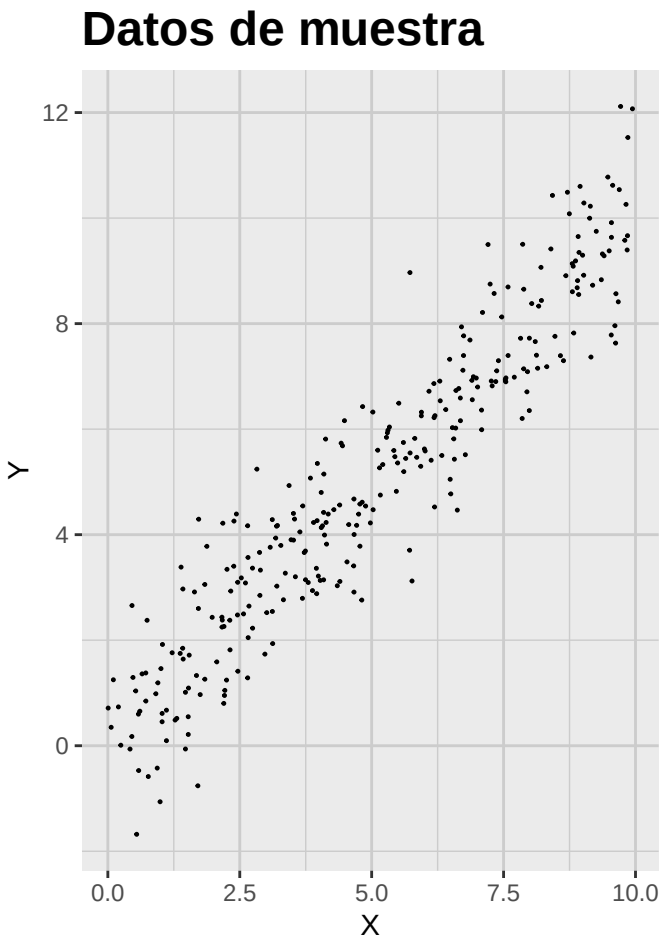


Figura 6.1: Datos de muestra con dos variables altamente correlacionadas.

Este conjunto consta de 300 observaciones de dos variables, X y Y: la primera es uniforme en $[0, 10]$ y la segunda es $Y = X + \epsilon$, con ϵ normal de media 0 y desviación 1. La matriz de covarianzas es:

```
Sigma <- cov(sample_data_tbl)
```

$$\Sigma = \begin{bmatrix} 7.874751 & 7.7745959 \\ 7.7745959 & 8.6510962 \end{bmatrix}$$

La varianza total es:

$$\text{Total Variance} = \operatorname{Tr}\Sigma = \Sigma_{11} + \Sigma_{22} = 16.5258472$$

Consideramos ahora la descomposición espectral de Σ para obtener:

```
# Compute singular vectors and values
eig <- eigen(Sigma)
vectors <- eig$vectors
values <- eig$values
```

$$Q = \begin{bmatrix} 0.689251 & -0.7245227 \\ 0.7245227 & 0.689251 \end{bmatrix}$$

y

$$\Lambda = \begin{bmatrix} 16.0472039 & 0 \\ 0 & 0.4786433 \end{bmatrix}$$

Las direcciones de componentes principales son las columnas de Q y la varianza explicada por cada componente es el valor propio correspondiente. En particular, como el vector de medias es:

```
sample_mean_vector <- colMeans(sample_data_tbl)
```

$$\bar{x} = 4.9961185$$
$$\bar{y} = 5.019733$$

tenemos que la relación entre variables originales y coordenadas de componentes principales es:

$$X = 0.689251PC_1 + -0.7245227PC_2 + 4.9961185$$
$$Y = 0.7245227PC_1 + 0.689251PC_2 + 5.019733$$

y

$$PC_1 = 0.689251(X - 4.9961185) + 0.7245227(Y - 5.019733)$$
$$PC_2 = -0.7245227(X - 4.9961185) + 0.689251(Y - 5.019733)$$

Con estas fórmulas, obtenemos la proyección de los datos sobre el primer y segundo componente. Los resultados se muestran a continuación:

```
sample_data_matrix <- as.matrix(sample_data_tbl)

# Center the data
centered_sample_data <- scale(sample_data_matrix, center = TRUE, scale = FALSE)

# SVD
svd_result <- svd(centered_sample_data)
v1 <- svd_result$v[, 1]
v2 <- svd_result$v[, 2]

# Projection onto PC, in centered coordinates
proj_PC1 <- (centered_sample_data %*% v1) %*% t(v1)
proj_PC2 <- (centered_sample_data %*% v2) %*% t(v2)
```

```
# Shift back to original location
proj_coords1 <- proj_PC1 + attr(centered_sample_data, "scaled:center")
proj_coords2 <- proj_PC2 + attr(centered_sample_data, "scaled:center")

# Provide unique names before converting to tibble
colnames(proj_coords1) <- c("x_proj1", "y_proj1")
colnames(proj_coords2) <- c("x_proj2", "y_proj2")

proj_coords1_tbl <- as_tibble(proj_coords1)
proj_coords2_tbl <- as_tibble(proj_coords2)

# Bind with original data
compare_proj1_tbl <- bind_cols(sample_data_tbl, proj_coords1_tbl)
compare_proj2_tbl <- bind_cols(sample_data_tbl, proj_coords2_tbl)

# Generate plot corresponding to the first projection
compare_plot1 <- ggplot(compare_proj1_tbl) +
  geom_point(
    aes(x = x, y = y),
    color = "black",
    size = 0.2,
    alpha = 1
  ) +
  geom_point(
    aes(x = x_proj1, y = y_proj1),
    color = c_pal("C red"),
    size = 0.2,
    alpha = 1
  ) +
  labs(
    title = "Proyección en el primer componente principal",
    x = "X",
    y = "Y"
  ) +
  coord_fixed() +
  theme_krul()

# Generate plot corresponding to the second projection
compare_plot2 <- ggplot(compare_proj2_tbl) +
  geom_point(
    aes(x = x, y = y),
    color = "black",
    size = 0.2,
    alpha = 1
```

```
  ) +
  geom_point(
    aes(x = x_proj2, y = y_proj2),
    color = c_pal("C red"),
    size = 0.2,
    alpha = 1
  ) +
  labs(
    title = "Proyección en el segundo componente principal",
    x = "X",
    y = "Y"
  ) +
  coord_fixed() +
  theme_krul()

# Combine plots
compare_plot1 + compare_plot2 +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Proyecciones de componentes principales",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )
```

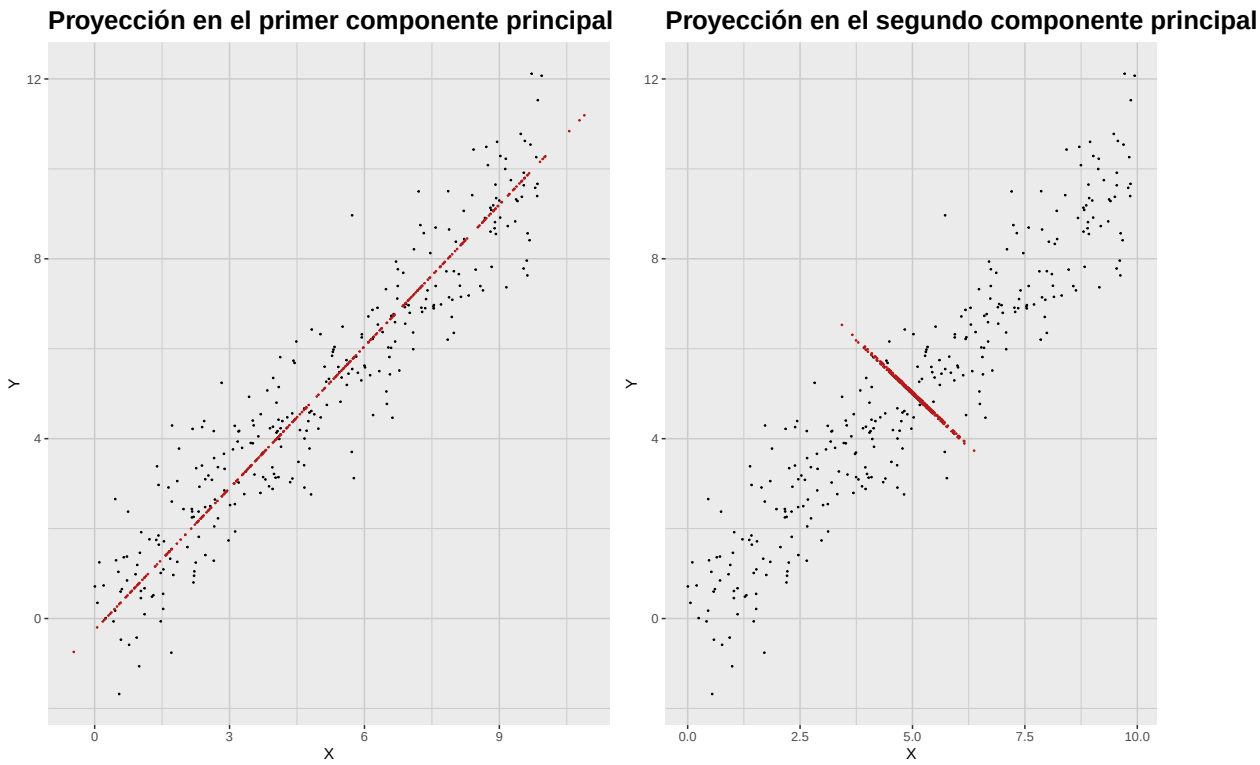


Figura 6.2: Proyecciones sobre las direcciones de los componentes principales. Se pierde mucha más información al proyectar sobre el segundo componente.

6.2. Varianza acumulada y gráficos scree

Como se indicó, el PCA se usa para reducción de dimensión. Buscamos reducir variables reteniendo la mayor información. Para decidir cuántos componentes conservar, examinamos la varianza explicada por cada uno. Es común usar un *gráfico scree*, que muestra valores propios en y y el índice del componente en x; ayuda a identificar el "codo". Otro gráfico útil es el de *varianza acumulada*, que muestra la proporción acumulada explicada. La proporción explicada por el componente k es:

$$\frac{\lambda_k}{\sum_{i=1}^{p} \lambda_i}$$

Este gráfico ayuda a determinar cuántos componentes se requieren para explicar, por ejemplo, 70 % o 90 % de la varianza total.

Para ilustrar, usaremos el conocido conjunto "iris", con cuatro mediciones (largo/ancho de sépalo y pétalo) para tres especies (setosa, versicolor, virginica). Realizaremos PCA y construiremos el scree para visualizar la varianza explicada por cada componente. Primero, cargamos los datos y calculamos covarianzas por especie:

```
# Load the iris dataset
data(iris)
```

```r
iris_setosa_tbl <- as_tibble(iris) %>%
  filter(Species == "setosa") %>%
  select(-Species)

iris_versicolor_tbl <- as_tibble(iris) %>%
  filter(Species == "versicolor") %>%
  select(-Species)

iris_virginica_tbl <- as_tibble(iris) %>%
  filter(Species == "virginica") %>%
  select(-Species)

# Compute covariance matrix
cov_matrix_setosa <- cov(iris_setosa_tbl)
cov_matrix_versicolor <- cov(iris_versicolor_tbl)
cov_matrix_virginica <- cov(iris_virginica_tbl)

# Singular value decomposition
eig_setosa <- eigen(cov_matrix_setosa)
eig_versicolor <- eigen(cov_matrix_versicolor)
eig_virginica <- eigen(cov_matrix_virginica)
```

La matriz de covarianza de cada especie es:

$$\Sigma_{setosa} = \begin{bmatrix} 0.12 & 0.1 & 0.02 & 0.01 \\ 0.1 & 0.14 & 0.01 & 0.01 \\ 0.02 & 0.01 & 0.03 & 0.01 \\ 0.01 & 0.01 & 0.01 & 0.01 \end{bmatrix}$$

$$\Sigma_{versicolor} = \begin{bmatrix} 0.27 & 0.09 & 0.18 & 0.06 \\ 0.09 & 0.1 & 0.08 & 0.04 \\ 0.18 & 0.08 & 0.22 & 0.07 \\ 0.06 & 0.04 & 0.07 & 0.04 \end{bmatrix}$$

$$\Sigma_{virginica} = \begin{bmatrix} 0.4 & 0.09 & 0.3 & 0.05 \\ 0.09 & 0.1 & 0.07 & 0.05 \\ 0.3 & 0.07 & 0.3 & 0.05 \\ 0.05 & 0.05 & 0.05 & 0.08 \end{bmatrix}$$

Los valores singulares de cada especie son:

$$\Lambda_{setosa} = \begin{bmatrix} 0.24 & 0 & 0 & 0 \\ 0 & 0.04 & 0 & 0 \\ 0 & 0 & 0.03 & 0 \\ 0 & 0 & 0 & 0.01 \end{bmatrix}$$

$$\Lambda_{versicolor} = \begin{bmatrix} 0.49 & 0 & 0 & 0 \\ 0 & 0.07 & 0 & 0 \\ 0 & 0 & 0.05 & 0 \\ 0 & 0 & 0 & 0.01 \end{bmatrix}$$

$$\Lambda_{virginica} = \begin{bmatrix} 0.7 & 0 & 0 & 0 \\ 0 & 0.11 & 0 & 0 \\ 0 & 0 & 0.05 & 0 \\ 0 & 0 & 0 & 0.03 \end{bmatrix}$$

Las direcciones asociadas a las componentes principales son las columnas de las matrices

$$Q_{setosa} = \begin{bmatrix} -0.67 & 0.6 & 0.44 & -0.04 \\ -0.73 & -0.62 & -0.27 & -0.02 \\ -0.1 & 0.49 & -0.83 & -0.24 \\ -0.06 & 0.13 & -0.2 & 0.97 \end{bmatrix}$$

$$Q_{versicolor} = \begin{bmatrix} -0.69 & 0.67 & -0.27 & 0.1 \\ -0.31 & -0.57 & -0.73 & -0.23 \\ -0.62 & -0.34 & 0.63 & -0.32 \\ -0.21 & -0.34 & 0.06 & 0.92 \end{bmatrix}$$

$$Q_{virginica} = \begin{bmatrix} -0.74 & -0.17 & -0.53 & 0.37 \\ -0.2 & 0.75 & -0.33 & -0.54 \\ -0.63 & -0.17 & 0.65 & -0.39 \\ -0.12 & 0.62 & 0.43 & 0.65 \end{bmatrix}$$

El diagrama de valores singulares (scree plot) y el de varianza acumulada para las tres especies se muestran en la figura siguiente:

```
# Scree plot data
scree_data <- tibble(
  Component = 1:4,
  Setosa = eig_setosa$values,
  Versicolor = eig_versicolor$values,
  Virginica = eig_virginica$values
) %>%
  pivot_longer(
    cols = -Component,
    names_to = "Species",
    values_to = "SingularValue"
  )
# Scree plot
```

```r
scree_plot <- scree_data %>%
  ggplot(aes(x = Component, y = SingularValue, color = Species)) +
  geom_point(size = 3) +
  geom_line() +
  c_scale_color("C red", "C blue", "C green") +
  scale_x_continuous(breaks = 1:4) +
  labs(
    title = "Gráfico scree del conjunto Iris",
    x = "Componente principal",
    y = "Valor singular"
  ) +
  theme_krul() +
  theme(legend.position = "top")

# Cumulative variance data
cumulative_variance_data <- scree_data %>%
  group_by(Species) %>%
  arrange(Component) %>%
  mutate(CumulativeVariance = cumsum(SingularValue) / sum(SingularValue))
# Cumulative variance plot
cumulative_variance_plot <- cumulative_variance_data %>%
  ggplot(aes(x = Component, y = CumulativeVariance, color = Species)) +
  geom_point(size = 3) +
  geom_line() +
  c_scale_color("C red", "C blue", "C green") +
  scale_x_continuous(breaks = 1:4) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +
  labs(
    title = "Varianza acumulada explicada por los componentes principales",
    x = "Componente principal",
    y = "Varianza acumulada explicada"
  ) +
  theme_krul() +
  theme(legend.position = "top")

# Combine scree plot and cumulative variance plot
scree_plot + cumulative_variance_plot +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Gráfico scree y varianza acumulada para el conjunto Iris",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
```

```
    )
  )
```

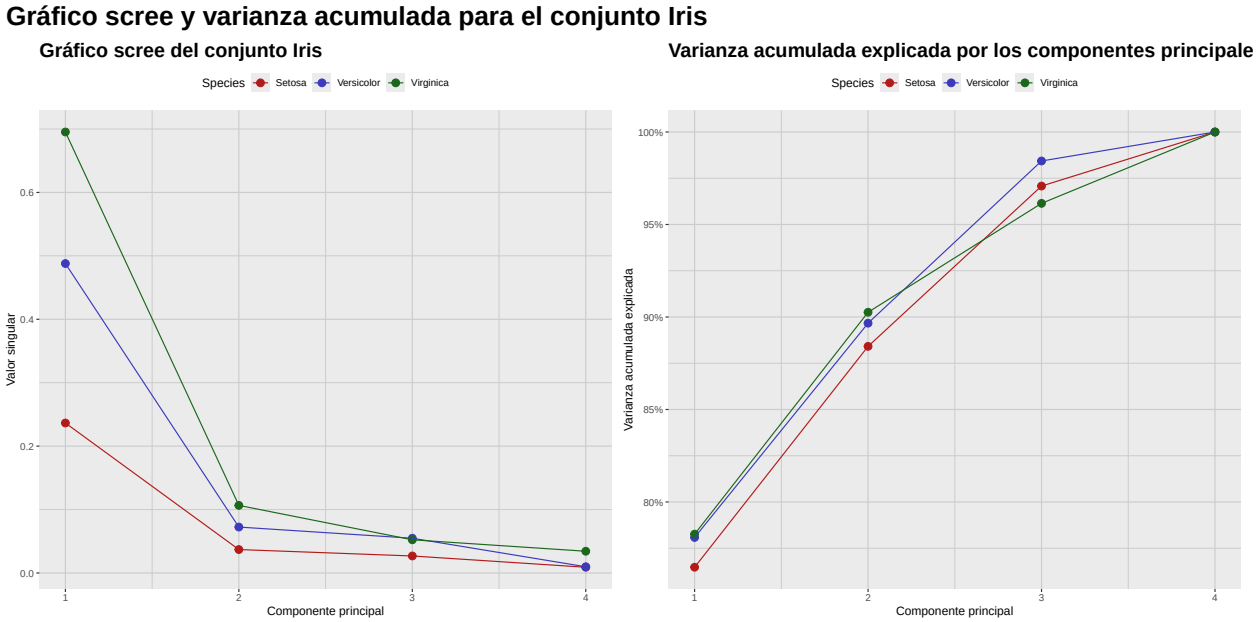


Figura 6.3: Análisis de Componentes Principales (ACP) del conjunto de datos Iris por especie. Para decidir cuántos componentes conservar, podemos usar el gráfico de sedimentación (scree plot, izquierda) y buscar el "codo" donde la varianza explicada se estabiliza. Alternativamente, podemos usar el gráfico de varianza acumulada (derecha) para determinar cuántos componentes se necesitan para explicar una cantidad deseada de varianza total, como 70 % o 90 %.

6.3. Cargas (loadings) y biplots

Recuerde que la ecuación (6.1) da la relación entre las variables originales y las puntuaciones de componentes principales. Con ella podemos calcular rápidamente las puntuaciones de cada observación. Los coeficientes de esta ecuación son las *cargas* de los componentes y cuantifican la contribución de cada variable. Un *biplot* muestra simultáneamente las puntuaciones de las observaciones y las cargas de las variables, permitiendo visualizar contribuciones y distribución en el espacio de los componentes.

Para ilustrar, crearemos biplots de los dos primeros componentes del conjunto "iris" por especie. Empezamos obteniendo las cargas correspondientes a cada especie, tomando las dos primeras

columnas de las matrices Q_{setosa}, $Q_{versicolor}$ y $Q_{virginica}$ mostradas arriba.

$$\text{Loadings}_{setosa} = \begin{bmatrix} -0.67 & 0.6 \\ -0.73 & -0.62 \\ -0.1 & 0.49 \\ -0.06 & 0.13 \end{bmatrix}$$

$$\text{Loadings}_{versicolor} = \begin{bmatrix} -0.69 & 0.67 \\ -0.31 & -0.57 \\ -0.62 & -0.34 \\ -0.21 & -0.34 \end{bmatrix}$$

$$\text{Loadings}_{virginica} = \begin{bmatrix} -0.74 & -0.17 \\ -0.2 & 0.75 \\ -0.63 & -0.17 \\ -0.12 & 0.62 \end{bmatrix}$$

Luego calculamos las puntuaciones multiplicando los datos centrados por estas matrices. Los biplots resultantes para cada especie se muestran a continuación:

```
# Function to prepare biplot data from precomputed singular vectors
prepare_biplot_from_eigen <- function(tbl, eig) {
  data_matrix <- as.matrix(tbl)
  centered_data_matrix <- scale(data_matrix, center = TRUE, scale = FALSE)


  # Scores: project observations onto PC1 and PC2
  scores <- centered_data_matrix %*% eig$vectors[, 1:2]
  colnames(scores) <- c("PC1", "PC2")
  scores_tbl <- as_tibble(scores) %>%
    mutate(Obs = row_number())


  # Loadings: first two singular vectors
  loadings <- eig$vectors[, 1:2]
  colnames(loadings) <- c("PC1", "PC2")
  loadings <- as_tibble(loadings) %>%
    mutate(Variable = colnames(tbl))

  list(scores = scores_tbl, loadings = loadings)
}

# Prepare data for each species
setosa_data <- prepare_biplot_from_eigen(
  iris_setosa_tbl,
  eig_setosa
)
```

```r
versicolor_data <- prepare_biplot_from_eigen(
  iris_versicolor_tbl,
  eig_versicolor
)

virginica_data <- prepare_biplot_from_eigen(
  iris_virginica_tbl,
  eig_virginica
)

# Function to create biplot
create_biplot <- function(scores_tbl, loadings_tbl, species_name) {
  ggplot() +
    geom_point(
      data = scores_tbl,
      aes(x = PC1, y = PC2),
      color = c_pal("C red"),
      alpha = 1,
      size = 0.3
    ) +
    geom_segment(
      data = loadings_tbl,
      aes(
        x = 0,
        y = 0,
        xend = PC1 * max(scores_tbl$PC1),
        yend = PC2 * max(scores_tbl$PC2)
      ),
      arrow = arrow(length = unit(0.3, "cm")),
      color = c_pal("C blue"),
    ) +
    geom_text(
      data = loadings_tbl,
      aes(
        x = PC1 * max(scores_tbl$PC1) * 1.1,
        y = PC2 * max(scores_tbl$PC2) * 1.1,
        label = Variable
      ),
      color = "black",
      size = 5
    ) +
    labs(
      title = paste("Biplot para", species_name),
      x = "PC1",
      y = "PC2"
```

```
    ) +
    theme_minimal()
}

# Create biplots
biplot_setosa <- create_biplot(
  setosa_data$scores,
  setosa_data$loadings,
  "Setosa"
)

biplot_versicolor <- create_biplot(
  versicolor_data$scores,
  versicolor_data$loadings,
  "Versicolor"
)

biplot_virginica <- create_biplot(
  virginica_data$scores,
  virginica_data$loadings,
  "Virginica"
)

# Combine biplots
biplot_setosa + biplot_versicolor + biplot_virginica +
  plot_layout(ncol = 3) +
  plot_annotation(
    title = "Biplots de especies de Iris (PC1 vs PC2)",
    theme = theme(
      plot.title = element_text(size = 24, face = "bold")
    )
  )
```

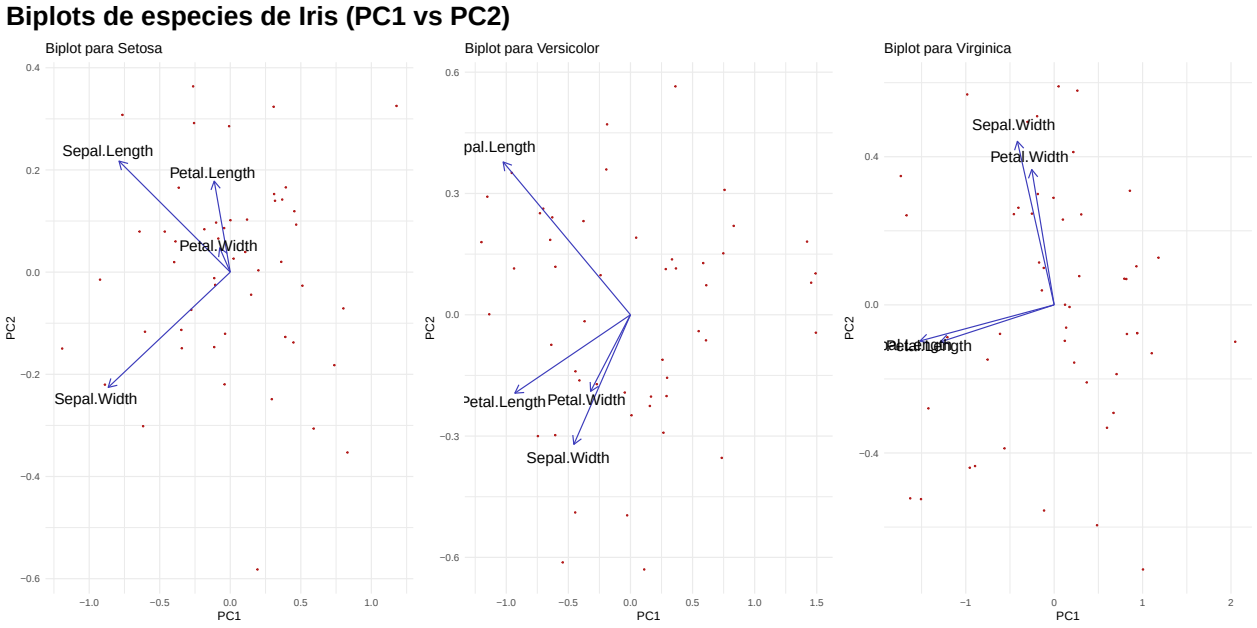


Figura 6.4: Cargas y puntuaciones para los dos primeros componentes principales del conjunto Iris por especie.

6.4. Resumen del proceso de PCA

En general, al realizar PCA se recomienda seguir estos pasos:

1. **Estandarice los datos (si es necesario).** No siempre es obvio si conviene estandarizar. Algunas pautas:
   - Si las variables están en escalas distintas (p.ej., cm y kg), suele recomendarse estandarizar.
   - Si están en la misma escala y con varianzas similares, podría no ser necesario.
   - Si las varianzas son muy diferentes, estandarizar ayuda a dar peso similar a todas.
2. **Calcule la matriz de covarianzas.**
   - La matriz captura cómo varían conjuntamente las variables.
   - Su tamaño es $p \times p$, donde p es el número de variables.
3. **Obtenga la descomposición en valores/vectores propios (SVD).**
   - Los autovalores miden la varianza explicada por cada componente.
   - Los autovectores dan las direcciones (combinaciones lineales) de los componentes.
4. **Ordene los componentes.**
   - Ordene los valores propios de mayor a menor.
   - El vector correspondiente al mayor valor propio es el primer componente.

5. **Decida cuántos componentes conservar.**
   - *Criterio de Kaiser*: conservar valores propios > 1 (para matriz de correlación).
   - *Gráfico scree*: buscar el "codo" donde se estabiliza la varianza explicada.
   - *Varianza acumulada*: conservar componentes suficientes para 70–90 %.
6. **Obtenga puntuaciones y cargas.**
   - Proyecte los datos originales sobre los componentes seleccionados.
   - Use la fórmula: $PC = Q^T(X - \bar{x})$.
7. **Interprete los componentes.**
   - Examine las *cargas* para ver qué variables contribuyen más.
   - Valores absolutos grandes indican mayor influencia.
   - Use biplots para visualizar puntuaciones y cargas conjuntamente.
   - Aplique reducción de dimensión antes de regresión, clustering u otros análisis.
   - Reduzca ruido descartando componentes de baja varianza.

Parte III

Análisis factorial

Capítulo 7

Análisis factorial: modelos teóricos

El *análisis factorial* es una técnica estadística ampliamente usada para identificar patrones ocultos o estructuras latentes detrás de grandes conjuntos de variables observadas. Busca reducir la complejidad agrupando variables relacionadas en un menor número de *factores* subyacentes, más fáciles de interpretar.

Algunas áreas de aplicación comunes incluyen:

- **Psicología y ciencias sociales:** para descubrir rasgos latentes como la inteligencia, la ansiedad, dimensiones de la personalidad o niveles de satisfacción, a partir de respuestas a cuestionarios o encuestas.
- **Mercadotecnia e investigación del consumidor:** para identificar motivos de compra, percepciones de marca o segmentos de mercado a partir de datos de comportamiento de los clientes.
- **Finanzas y economía:** para revelar determinantes ocultos de los rendimientos bursátiles, indicadores macroeconómicos o factores de riesgo.
- **Educación:** para analizar reactivos de examen y determinar si miden habilidades o capacidades comunes.
- **Ciencias biológicas y de la salud:** para encontrar estructuras latentes en datos genéticos, listas de síntomas o resultados reportados por pacientes.

En la práctica se usa cuando se sospecha que unas cuantas dimensiones no observadas explican las correlaciones entre muchas variables. Es especialmente útil para construir escalas de medición, reducir datos y entender la estructura de conjuntos complejos.

7.1. Planteamiento del análisis factorial

El contexto teórico del análisis factorial es una relación de la forma

$$X = \mu + LF + \epsilon$$

donde:

- X es un vector aleatorio de p variables observadas (indicadores).
- μ es un vector constante de dimensión p.
- L es una matriz constante $p \times m$ conocida como la matriz de *cargas factoriales.*
- $F \sim \mathcal{N}(0, I_m)$ es un vector de m factores latentes (variables no observadas) que se asumen distribuidos normalmente con media cero y matriz de covarianzas identidad.
- $\epsilon \sim \mathcal{N}(0, \Psi)$ es un vector de errores únicos (varianzas específicas) que también se asumen distribuidos normalmente con media cero y matriz de covarianzas diagonal $\Psi = \text{diag}(\sigma_1^2, \ldots, \sigma_p^2)$.
- Se asume que los factores F y los errores ϵ son independientes; en particular, $\text{Cov}[F, \epsilon] = 0$.

Como consecuencia de estos supuestos, se tiene

$$\mathbb{E}[X] = \mathbb{E}[\mu + LF + \epsilon] = \mathbb{E}[\mu] + L\mathbb{E}[F] + \mathbb{E}[\epsilon] = \mu$$

y

$$\begin{aligned}
\text{Var}[X] &= \text{Var}[LF + \epsilon] \\
&= \mathbb{E}[(LF + \epsilon)(LF + \epsilon)^T] \\
&= \mathbb{E}[LFF^TL^T] + \mathbb{E}[\epsilon\epsilon^T] + \mathbb{E}[LF\epsilon^T] + \mathbb{E}[\epsilon F^TL^T] \\
&= L\mathbb{E}[FF^T]L^T + \mathbb{E}[\epsilon\epsilon^T] + L\mathbb{E}[F\epsilon^T] + \mathbb{E}[\epsilon F^T]L^T \\
&= LI_mL^T + \Psi + L0 + 0L^T \\
&= LL^T + \Psi
\end{aligned}$$

Además, como X es combinación lineal de normales, se sigue que

$$X \sim \mathcal{N}(\mu, LL^T + \Psi)$$

7.2. Descomposición de varianza

La descomposición $\text{Var}[X] = LL^T + \Psi$ muestra que la varianza total de las variables observadas puede separarse en dos componentes:

- **Varianza común (LL^T):** explicada por los factores comunes F, capta la variabilidad compartida atribuible a la estructura latente.
- **Varianza única (Ψ):** específica de cada variable observada; incluye errores de medición y variabilidad propia.

En particular, la varianza de cada variable observada X_i puede expresarse como

$$\text{Var}[X_i] = \sum_{j=1}^{m} L_{ij}^2 + \sigma_i^2$$

donde $h_i^2 = \sum_{j=1}^{m} L_{ij}^2$ es la *comunalidad* de X_i (parte explicada por factores comunes) y σ_i^2 la varianza única.

7.3. Estimación de puntuaciones factoriales

Suponga el modelo

$$X = \mu + LF + \epsilon$$

con las mismas condiciones. Entonces la distribución conjunta de X y F es

$$\begin{bmatrix} X \\ F \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mu \\ 0 \end{bmatrix}, \begin{bmatrix} LL^T + \Psi & L \\ L^T & I_m \end{bmatrix} \right)$$

Nótese que, en particular,

$$\text{Cov}[X, F] = \mathbb{E}[(X - \mu)(F - 0)^T] = \mathbb{E}[(LF + \epsilon)F^T] = L\mathbb{E}[FF^T] + \mathbb{E}[\epsilon F^T] = LI_m + 0 = L$$

Usando propiedades de la normal multivariante, se obtiene la distribución condicional de F dado X, que también es normal con media y covarianza

$$[F|X = x] \sim \mathcal{N}\left(L^T(LL^T + \Psi)^{-1}(x - \mu), I_m - L^T(LL^T + \Psi)^{-1}L\right)$$

La media de esta distribución condicional,

$$\mathbb{E}[F|X = x] = L^T(LL^T + \Psi)^{-1}(x - \mu)$$

se conoce como la *puntuación factorial* para la observación $X = x$. Proporciona una estimación de los factores latentes a partir de las variables observadas y las cargas. La matriz de covarianzas

$$I_m - L^T(LL^T + \Psi)^{-1}L$$

representa la incertidumbre asociada a las puntuaciones estimadas.

Comentario 7.3.1. Nótese que, si

$$f = \mathbb{E}[F|X = x] = L^T(LL^T + \Psi)^{-1}(x - \mu)$$

entonces, en general, el *residuo*

$$r(x) = x - \mu - Lf$$

es distinto de cero. De hecho, como función de X, el residuo tiene distribución

$$r(X) \sim \mathcal{N}\left(0, L\left(I_m + L^T\Psi^{-1}L\right)^{-1}L^T + \Psi\right)$$

7.4. Rotación de factores

Un aspecto importante del análisis factorial es que los factores no están definidos de manera única. Si F es un vector de factores, cualquier transformación ortogonal $F^* = QF$ (con $Q^TQ = I$) produce un nuevo conjunto de factores que explica la misma varianza. En particular,

$$\text{Var}[F^*] = \text{Var}[QF] = Q\,\text{Var}[F]Q^T = QI_mQ^T = I_m$$

Las cargas correspondientes se transforman como $L^* = LQ^T$. En otras palabras, si

$$X = \mu + LF + \epsilon$$

es un modelo factorial válido, entonces también lo es

$$X = \mu + L^*F^* + \epsilon = \mu + LQ^TQF + \epsilon = \mu + LF + \epsilon$$

Esto implica que la solución no es única y distintas rotaciones pueden conducir a interpretaciones diferentes. En el siguiente capítulo exploraremos métodos prácticos y técnicas de rotación para lograr soluciones más interpretables.

Capítulo 8

Análisis factorial: métodos prácticos

Recordemos del capítulo anterior que el modelo clásico de análisis factorial es

$$X = \mu + LF + \epsilon$$

donde:

- X es un vector aleatorio de p variables observadas (indicadores).
- μ es un vector constante de dimensión p.
- L es una matriz constante $p \times m$ conocida como la matriz de *cargas factoriales.*
- $F \sim \mathcal{N}(0, I_m)$ es un vector de m factores latentes (variables no observadas) que se asumen distribuidos normalmente con media cero y matriz de covarianzas identidad.
- $\epsilon \sim \mathcal{N}(0, \Psi)$ es un vector de errores únicos (varianzas específicas) que también se asumen distribuidos normalmente con media cero y matriz de covarianzas diagonal $\Psi = \text{diag}(\sigma_1^2, \ldots, \sigma_p^2)$.
- Se asume que los factores F y los errores ϵ son independientes; en particular, $\text{Cov}[F, \epsilon] = 0$.

Nótese que, como $\epsilon \sim \mathcal{N}(0, \Psi)$, este modelo queda completamente determinado por μ, L y Ψ. En este capítulo discutiremos métodos prácticos para estimar estos parámetros a partir de datos y para obtener las puntuaciones factoriales de F una vez estimado el modelo.

8.1. Evaluación de la factorizabilidad

El primer paso es evaluar si los datos son adecuados para AF; es decir, si las variables observadas exhiben correlaciones suficientes que justifiquen un modelo factorial. Varias pruebas y medidas pueden emplearse:

1. **Revisión de la matriz de correlaciones:** inspección visual en busca de patrones altos (p.ej., > 0.3) que sugieran factores comunes.
2. **Prueba de esfericidad de Bartlett:** contrasta identidad; un resultado significativo ($p < 0.05$) sugiere correlación suficiente.

3. **Kaiser–Meyer–Olkin (KMO):** mide la proporción de varianza atribuible a factores comunes; valores cercanos a 1 indican utilidad (típicamente > 0.6 es aceptable).

8.2. Decidir el número de factores

Determinar cuántos factores retener es crucial y puede afectar los resultados e interpretaciones. Algunos métodos:

1. **Criterio de Kaiser:** retener valores singulares > 1.
2. **Gráfico scree:** buscar el "codo".
3. **Análisis paralelo:** comparar con datos aleatorios del mismo tamaño.
4. **Índices de ajuste:** (RMSEA, TLI) para evaluar modelos con distinto número de factores.

8.3. Extracción de factores

Una vez decidido el número de factores, el siguiente paso es extraerlos. Métodos disponibles:

1. **Ejes principales (PAF):** se centra en la varianza compartida; estima comunalidades y extrae factores desde la matriz reducida.
2. **Máxima verosimilitud (ML):** estima cargas maximizando la verosimilitud; permite pruebas e índices de ajuste.

8.4. Rotación de factores

Tras extraer factores, suele ser útil rotarlos para lograr una estructura más simple e interpretable. La rotación puede ser ortogonal (sin correlación) u oblicua (con correlación). Métodos comunes:

1. **Rotación Varimax:** Método de rotación ortogonal que maximiza la varianza de las cargas al cuadrado dentro de cada factor, logrando una estructura más simple donde cada variable carga fuertemente en un factor y débilmente en los demás.
2. **Rotación Quartimax:** Otro método de rotación ortogonal que simplifica las filas de la matriz de cargas, buscando que cada variable tenga cargas altas en la menor cantidad posible de factores.
3. **Rotación Equamax:** Método ortogonal híbrido que combina los objetivos de varimax y quartimax, equilibrando la simplificación de las filas y las columnas de la matriz de cargas.
4. **Rotación Promax:** Método de rotación oblicua que permite la correlación entre factores. Comienza con una rotación varimax y luego eleva las cargas a una potencia para obtener una estructura más simple.

8.5. Cálculo de puntuaciones factoriales

Una vez estimadas las cargas factoriales y rotados los factores (cuando aplica), el siguiente paso es calcular las puntuaciones factoriales para cada observación en el conjunto de datos. Estas puntuaciones representan los valores estimados de los factores latentes para cada persona a partir de sus puntuaciones observadas. Existen varios métodos para obtenerlas:

1. **Método de regresión:** estima puntuaciones al regredir las variables observadas sobre las cargas estimadas; es insesgado, pero puede no preservar la varianza original de los factores.
2. **Método de Bartlett:** calcula puntuaciones minimizando la varianza residual de las variables observadas; produce puntuaciones más fiables y con menor error de medición.
3. **Método de Anderson–Rubin:** genera puntuaciones no correlacionadas y estandarizadas; útil cuando se asume ortogonalidad de los factores.

8.6. Evaluación del ajuste global

Después de ajustar un modelo de análisis factorial, es importante evaluar qué tanto el modelo reproduce los datos observados. Diversas pruebas estadísticas e índices de ajuste permiten realizar esta evaluación:

1. **Prueba χ^2 de ajuste:** contrasta que la covarianza implícita del modelo coincida con la observada; un resultado no significativo ($p > 0.05$) sugiere buen ajuste.
2. **RMSEA:** discrepancia por grado de libertad entre covarianzas observada y modelada; valores < 0.06 indican buen ajuste.
3. **TLI (Tucker–Lewis):** compara el ajuste del modelo con un modelo nulo; valores > 0.95 indican buen ajuste.
4. **SRMR:** discrepancia media estandarizada entre correlaciones observadas y predichas; valores < 0.08 indican buen ajuste.

8.7. Visualización del modelo

Visualizar los resultados de un análisis factorial facilita la interpretación de los factores y el entendimiento de las relaciones entre variables. Algunas técnicas comunes son:

1. **Gráfico scree:** muestra los valores singulares en orden descendente y ayuda a decidir cuántos factores conviene retener.
2. **Diagrama de cargas factoriales:** un diagrama de dispersión de las cargas para apreciar cómo se distribuyen las variables en los distintos factores, especialmente después de rotar.
3. **Biplot:** combina en una sola figura las puntuaciones factoriales y las cargas, permitiendo visualizar simultáneamente a las observaciones y las variables en el espacio de factores.
4. **Mapa de calor de cargas:** representa la magnitud de las cargas factoriales mediante colores, resaltando patrones y posibles agrupamientos.

Comentario 8.7.1. Los diagramas de cargas y los biplots solo tienen sentido cuando hay 2 o 3 factores; de otro modo no se visualizan adecuadamente.

8.8. Ejemplo: análisis factorial del conjunto Holzinger-Swineford

Mostraremos ahora un ejemplo con el conjunto "HolzingerSwineford1939" del paquete "lavaan". Incluye puntuaciones de 9 pruebas cognitivas aplicadas a 301 estudiantes de 7º y 8º. Las pruebas miden 3 habilidades latentes: visual/espacial, verbal y rapidez. En este ejemplo nos enfocaremos en x4 a x9 para ajustar un modelo de dos factores y visualizarlo en un biplot.

Una descripción detallada de cada prueba se presenta a continuación:

1. **Paragraph (x4)**: Comprensión lectora. Las y los estudiantes leen párrafos cortos y contestan preguntas sobre la idea principal, detalles e inferencias sencillas.
2. **Sentence (x5)**: Completar oraciones. Elegir la palabra o frase que mejor complete una oración para que sea clara y gramatical; enfatiza vocabulario en contexto y sintaxis.
3. **Word Meaning (x6)**: Vocabulario. Seleccionar el mejor sinónimo o definición para una palabra objetivo; refleja la amplitud del conocimiento léxico.
4. **Addition (x7)**: Fluidez aritmética cronometrada. Resolver rápidamente muchas sumas sencillas con precisión; captura velocidad de procesamiento con una carga mínima de razonamiento.
5. **Counting Dots (x8)**: Enumeración visual rápida. Contar con rapidez pequeños grupos de puntos a lo largo de muchos reactivos; mide el barrido visual y la precisión bajo presión de tiempo.
6. **Straight–Curved Capitals (x9)**: Velocidad de discriminación perceptual. Revisar letras mayúsculas y marcar aquellas con solo trazos rectos (versus cualquier trazo curvo) bajo límites estrictos de tiempo.

Para realizar este análisis, comenzaremos cargando las bibliotecas necesarias y el conjunto de datos:

```
# Load required libraries
library(lavaan)
library(psych)
library(scales)

# Load the Holzinger-Swineford1939 dataset
data(HolzingerSwineford1939)

# Select the relevant tests
hs_data <- as_tibble(HolzingerSwineford1939) %>%
  select("x4", "x5", "x6", "x7", "x8", "x9")
```

8.8.1. Evaluar la factorizabilidad

Iniciamos el proceso evaluando la factorizabilidad para x4–x9. Primero reunimos la información requerida:

```r
# Correlation matrix (pairwise complete observations)
Sigma <- cor(hs_data)

# Bartlett's test and KMO measure
n_eff <- nrow(hs_data)
bart <- cortest.bartlett(Sigma, n = n_eff)
kmo  <- KMO(Sigma)
```

Mostramos ahora el mapa de calor de correlaciones:

```r
corr_df <- Sigma %>%
  as_tibble(rownames = "Var1") %>%
  pivot_longer(-Var1, names_to = "Var2", values_to = "r")

# Preserve matrix order on axes
corr_df$Var1 <- factor(corr_df$Var1, levels = colnames(Sigma))
corr_df$Var2 <- factor(corr_df$Var2, levels = colnames(Sigma))

ggplot(corr_df, aes(x = Var2, y = Var1, fill = r)) +
  geom_tile(color = "black") +
  geom_text(aes(label = sprintf("%.2f", r)), size = 3) +
  scale_fill_gradient2(
    low = c_pal("C blue"), mid = "white", high = c_pal("C red"),
    midpoint = 0, limits = c(-1, 1)
  ) +
  coord_fixed() +
  labs(
    title = "Mapa de calor de correlaciones (x4-x9)",
    x = "", y = "", fill = "Correlación"
  ) +
  theme(
    plot.title = element_text(size = 18, face = "bold"),
    panel.background = element_blank(),
    panel.grid = element_blank(),
    axis.ticks = element_blank()
  )
```

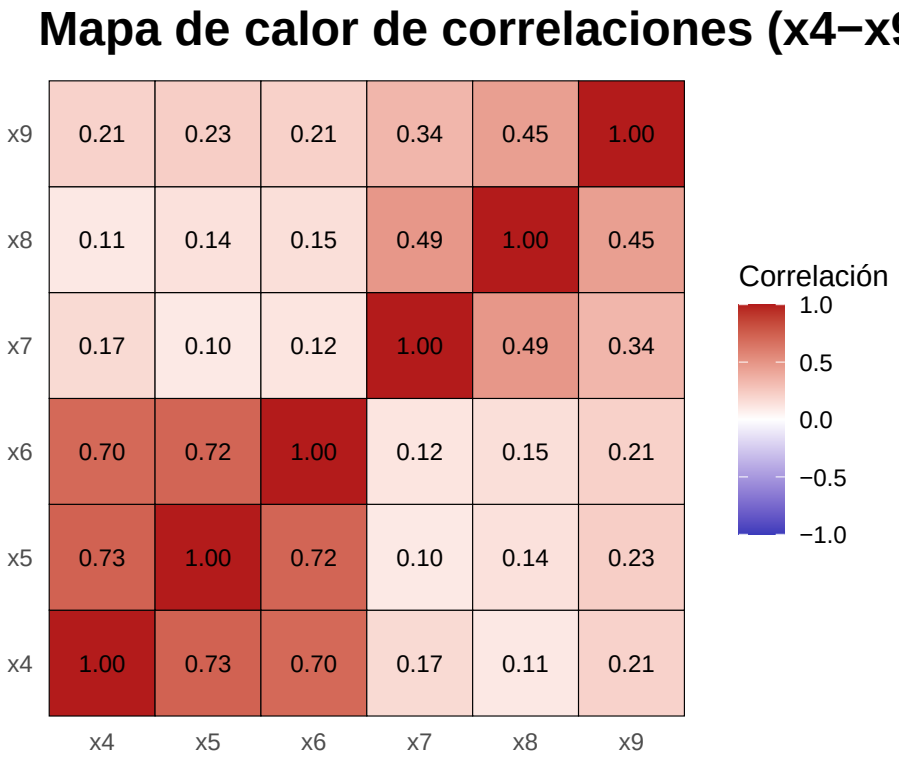


Figura 8.1: Mapa de calor de correlaciones para x4–x9. Se observan correlaciones fuertes entre las pruebas verbales (x4, x5, x6) y entre las de rapidez (x7, x8, x9), sugiriendo un modelo de dos factores.

Obsérvese la fuerte correlación entre pruebas verbales (x4, x5, x6) y entre rapidez (x7, x8, x9), pero no entre grupos. Esto sugiere que un modelo de dos factores es adecuado. Reportamos ahora Bartlett y KMO:

$$\text{Prueba de esfericidad de Bartlett} = 0$$
$$\text{Medida Kaiser-Meyer-Olkin} = 0.7308774$$

Como se observa, todas las pruebas indican que el AF es apropiado para este conjunto.

8.8.2. Decidir el número de factores

Usaremos un gráfico scree y uno de varianza acumulada para decidir cuántos factores retener. Como usamos la matriz de correlaciones, buscamos valores singulares > 1 (Kaiser) y el "codo" en el scree.

```
# Scree analysis from the correlation matrix
eig <- eigen(Sigma)

scree_df <- tibble(
  component = seq_along(eig$values),
  singular_value = eig$values
) %>%
```

```
  mutate(
    cumulative = cumsum(singular_value) / sum(singular_value)
  )

# Scree plot
scree_plot <- scree_df %>%
  ggplot(aes(x = component, y = singular_value)) +
  geom_point(size = 3, color = c_pal("C red")) +
  geom_line(color = c_pal("C blue")) +
  geom_hline(
    yintercept = 1,
    linetype = "dashed",
    color = c_pal("C green"),
    linewidth = 1.5
  ) +
  scale_x_continuous(breaks = scree_df$component) +
  labs(
    title = "Gráfico scree",
    x = "Componente principal",
    y = "Valor singular"
  ) +
  theme_krul()

# Cumulative variance plot
cum_plot <- ggplot(scree_df, aes(x = component, y = cumulative)) +
  geom_point(size = 3, color = c_pal("C red")) +
  geom_line(color = c_pal("C blue")) +
  scale_x_continuous(breaks = scree_df$component) +
  scale_y_continuous(
    labels = percent_format(accuracy = 1),
    limits = c(0, 1)
  ) +
  labs(
    title = "Varianza acumulada",
    x = "Componente principal",
    y = "Varianza acumulada explicada"
  ) +
  theme_krul()

# Combine scree plot and cumulative variance plot
scree_plot + cum_plot + plot_layout(ncol = 2) +
  plot_annotation(
    title = paste(
      "Gráfico scree y varianza acumulada para",
      "el conjunto Holzinger-Swineford"
```

```
    ),
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )
```

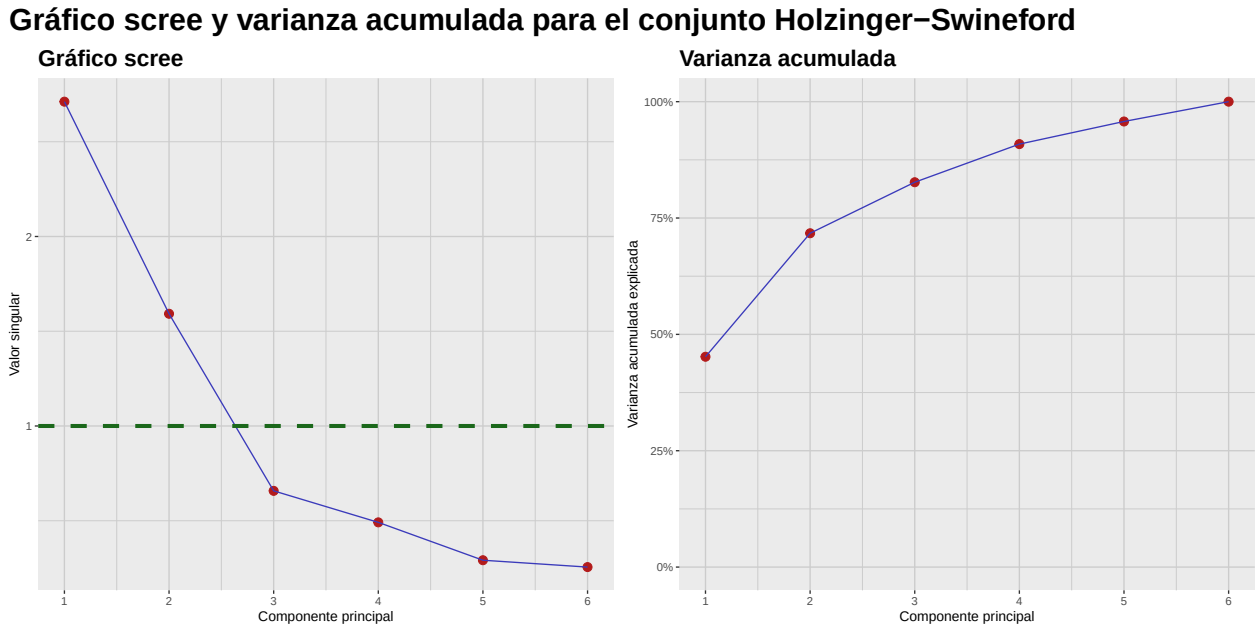


Figura 8.2: Gráfico scree y varianza acumulada para las pruebas x4–x9. Ambos gráficos sugieren que una solución de dos factores es adecuada para este conjunto de datos.

Nótese que tanto el gráfico scree como el de varianza acumulada sugieren que una solución de dos factores es adecuada para las seis pruebas consideradas.

8.8.3. Extracción, rotación y puntuación de factores

Aunque extraer, rotar y puntuar factores son pasos teóricos distintos, en la práctica suelen realizarse juntos con paquetes estadísticos. Aquí usaremos `psych::fa` para realizar los tres pasos de una vez: ajustar un modelo de 2 factores por máxima verosimilitud (ML), calcular puntuaciones por regresión y aplicar una rotación varimax para simplificar la estructura.

```
# Fit 2-factor FA using ML, regression scores and varimax rotation
fa_rot <- fa(
  r = hs_data,
  nfactors = 2,
  rotate = "varimax",
  fm = "ml",
```

```
  scores = "regression",
  use = "pairwise"
)

# Extract factor loadings into a tibble
loadings <- fa_rot$loadings[, 1:2] %>%
  unclass() %>%
  as_tibble(rownames = "Variable") %>%
  rename(F1 = 2, F2 = 3)
# Extract factor scores into a tibble
scores <- fa_rot$scores %>%
  as_tibble() %>%
  setNames(c("F1", "F2")) %>%
  mutate(Obs = row_number())
```

8.8.4. Evaluación del ajuste del modelo

Evaluamos qué tan bien la solución ML de 2 factores reproduce las correlaciones observadas. Para ello, reportamos χ^2, RMSEA, CFI, TLI y RMSR.

```
# Extract standard fit indices from psych::fa object (ML)
chisq <- fa_rot$STATISTIC
df    <- fa_rot$dof
pval  <- fa_rot$PVAL
rmsea <- as.numeric(fa_rot$RMSEA[1])
tli   <- fa_rot$TLI
bic   <- fa_rot$BIC

L   <- as.matrix(fa_rot$loadings)
Phi <- if (!is.null(fa_rot$Phi)) fa_rot$Phi else diag(ncol(L))
uni <- if (!is.null(fa_rot$uniquenesses)) {
  fa_rot$uniquenesses
} else {
  1 - rowSums((L %*% Phi) * L)
}
Rhat <- L %*% Phi %*% t(L) + diag(as.numeric(uni))

# Helper function to extract lower-triangle entries (off-diagonal)
lt_offdiag <- function(M) M[lower.tri(M, diag = FALSE)]

# Classic RMSR
res <- Sigma - Rhat
rmsr <- sqrt(mean(lt_offdiag(res)^2, na.rm = TRUE))

# Standardized RMSR (SRMR)
is_cov <- any(abs(diag(Sigma) - 1) > 1e-8)
```

```r
S_obs_cor  <- if (is_cov) stats::cov2cor(Sigma) else Sigma
S_hat_cor  <- if (is_cov) stats::cov2cor(Rhat)  else Rhat

res_cor <- S_obs_cor - S_hat_cor

# SRMR = sqrt( sum_{i<j}(res_ij^2) / sum_{i<j}(obs_ij^2) )
num <- sum(lt_offdiag(res_cor)^2, na.rm = TRUE)
den <- sum(lt_offdiag(S_obs_cor)^2, na.rm = TRUE)
srmr <- sqrt(num / den)
```

Los resultados son:

$$\begin{aligned} \text{Prueba } \chi^2 &= 0.1156716 \\ \text{RMSEA} &= 0.0531269 \\ \text{TLI} &= 0.9804573 \\ \text{RMSR} &= 0.0411927 \end{aligned}$$

Todas las métricas indican que el modelo de 2 factores ajusta bien los datos.

8.8.5. Visualización del modelo

Finalmente, visualizamos el modelo con un biplot. También rotaremos manualmente los factores para ilustrar qué pasaría sin una rotación adecuada.

```r
# Scaling factor for arrows (keep consistent across plots)
arrow_scale <- 3

# Base (unrotated) biplot
p_base <- ggplot() +
  geom_point(
    data = scores,
    aes(x = F1, y = F2),
    alpha = 1,
    color = c_pal("C red"),
    size = 0.5
  ) +
  geom_segment(
    data = loadings,
    aes(x = 0, y = 0, xend = F1 * arrow_scale, yend = F2 * arrow_scale),
    arrow = arrow(length = unit(0.2, "cm")),
    color = c_pal("C blue"),
    linewidth = 1.2
  ) +
  geom_text(
    data = loadings,
    aes(x = F1 * arrow_scale, y = F2 * arrow_scale, label = Variable),
```

```
    color = "black",
    hjust = -0.5
  ) +
  labs(
    title = "Rotación Varimax",
    x = "Factor 1",
    y = "Factor 2"
  ) +
  theme_krul()

# 45° rotation of scores and loadings
theta  <- pi / 4
cos_t  <- cos(theta)
sin_t  <- sin(theta)

scores_rot <- scores %>%
  mutate(
    F1_rot = F1 * cos_t - F2 * sin_t,
    F2_rot = F1 * sin_t + F2 * cos_t
  )

loadings_rot <- loadings %>%
  mutate(
    F1_rot = F1 * cos_t - F2 * sin_t,
    F2_rot = F1 * sin_t + F2 * cos_t
  )

# Rotated biplot
p_rot <- ggplot() +
  geom_point(
    data = scores_rot,
    aes(x = F1_rot, y = F2_rot),
    alpha = 1,
    color = c_pal("C red"),
    size = 0.5
  ) +
  geom_segment(
    data = loadings_rot,
    aes(x = 0, y = 0, xend = F1_rot * arrow_scale, yend = F2_rot * arrow_scale),
    arrow = arrow(length = unit(0.2, "cm")),
    color = c_pal("C blue"),
    linewidth = 1.2
  ) +
  geom_text(
    data = loadings_rot,
```

```
    aes(x = F1_rot * arrow_scale, y = F2_rot * arrow_scale, label = Variable),
    color = "black",
    hjust = -0.5
  ) +
  labs(
    title = "Rotación manual",
    x = "Factor 1 (rotado)",
    y = "Factor 2 (rotado)"
  ) +
  theme_krul()

# Optional: share axis limits for fair comparison
xr <- range(
  c(
    scores$F1,
    scores_rot$F1_rot,
    loadings$F1 * arrow_scale,
    loadings_rot$F1_rot * arrow_scale
  ),
  na.rm = TRUE
)

yr <- range(
  c(
    scores$F2,
    scores_rot$F2_rot,
    loadings$F2 * arrow_scale,
    loadings_rot$F2_rot * arrow_scale
  ),
  na.rm = TRUE
)
p_base <- p_base + coord_equal(xlim = xr, ylim = yr)
p_rot  <- p_rot  + coord_equal(xlim = xr, ylim = yr)

# Side-by-side using patchwork
p_base + p_rot + plot_layout(ncol = 2) +
  plot_annotation(
    title = paste(
      "Biplots del modelo de análisis factorial",
      "usando rotación Varimax y manual"
    ),
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
```

```
      )
    )
  )
```

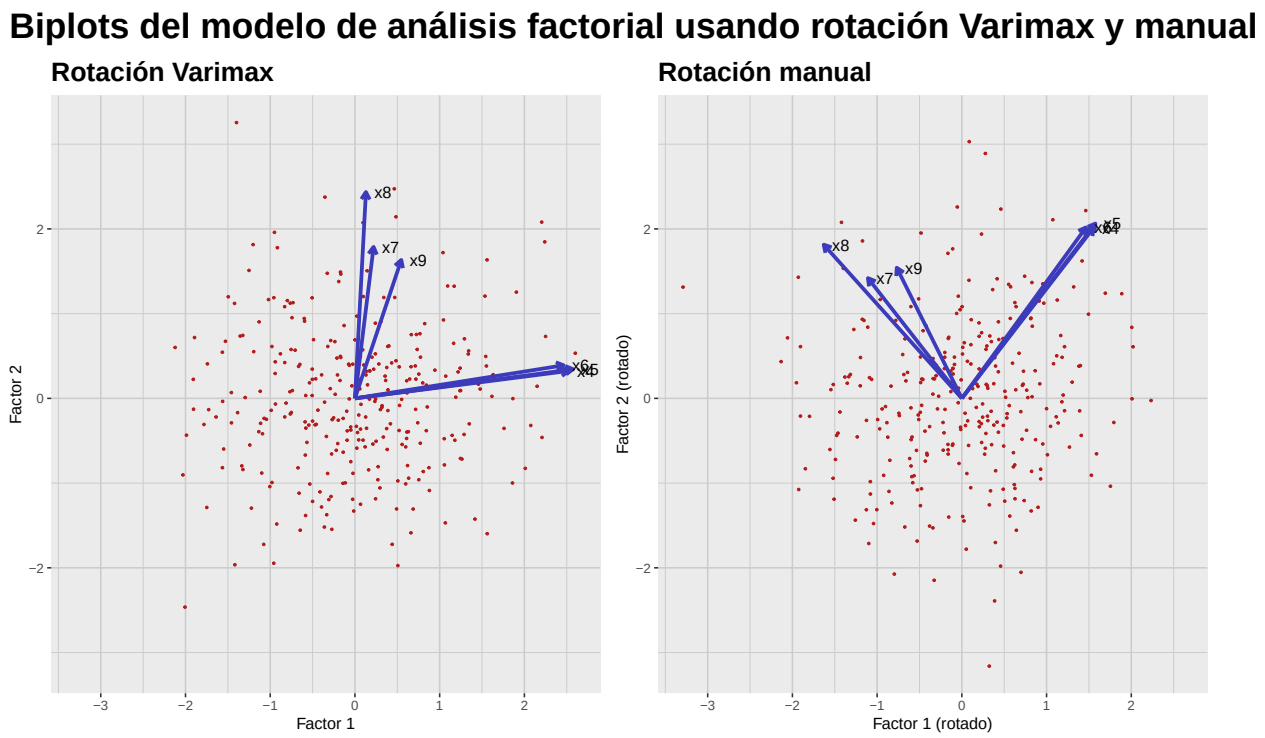


Figura 8.3: Biplots de la solución de 2 factores para x4–x9. Izquierda: rotación Varimax. Derecha: rotación manual 45°. La rotación Varimax produce una estructura más simple e interpretable: las pruebas verbales (x4, x5, x6) cargan alto en el Factor 1 y las de rapidez (x7, x8, x9) en el Factor 2.

Parte IV

Análisis de conglomerados

Capítulo 9

Análisis de conglomerados jerárquicos

Clustering (agrupamiento) es una técnica de aprendizaje no supervisado que agrupa puntos de datos similares con base en sus características. A diferencia del aprendizaje supervisado, el agrupamiento no depende de etiquetas o categorías predefinidas; en su lugar, identifica patrones y estructuras dentro de los propios datos. Los métodos de agrupamiento suelen clasificarse en dos familias principales:

1. **Agrupamiento jerárquico**: Estos métodos construyen un *dendrograma* (también llamado *diagrama de árbol*) que representa la estructura taxonómica de los datos. Es especialmente útil para conjuntos de datos pequeños y cuando se desconoce el número de conglomerados.
2. **Agrupamiento no jerárquico**: Métodos como K-medias y DBSCAN particionan los datos en un número predefinido de conglomerados o con base en densidad. Generalmente escalan mejor para conjuntos de datos grandes.

En este capítulo nos enfocaremos en el agrupamiento jerárquico, dejando los métodos no jerárquicos para el siguiente.

9.1. Entendiendo los dendrogramas

Un dendrograma es un diagrama en forma de árbol que ilustra la estructura taxonómica de los datos de acuerdo con un algoritmo de agrupamiento jerárquico. Proporciona una representación visual de cómo se agrupan los puntos según sus similitudes. El eje vertical representa la distancia o disimilitud entre conglomerados, mientras que el eje horizontal representa los puntos individuales o los propios conglomerados. Al "cortar" el dendrograma a una altura específica, podemos determinar el número de conglomerados que deseamos considerar para el análisis.

Como ilustración, consideremos el diagrama de dispersión del conjunto de datos Iris (considerando únicamente las variables "Sepal.Length" y "Sepal.Width"), donde cada punto está coloreado según la especie de la flor:

```
data(iris)

iris_plot <- iris %>%
  ggplot(aes(x = Sepal.Length, y = Sepal.Width, color = Species)) +
```

```
  geom_point(
    size = 0.8,
    alpha = 1
  ) +
  c_scale_color("C green", "C blue", "C red") +
  labs(
    title = "Conjunto de datos Iris: Sepal.Length vs Sepal.Width",
    x = "Sepal Length (cm)",
    y = "Sepal Width (cm)"
  ) +
  theme_krul() +
  theme(legend.position = "top")

iris_plot
```

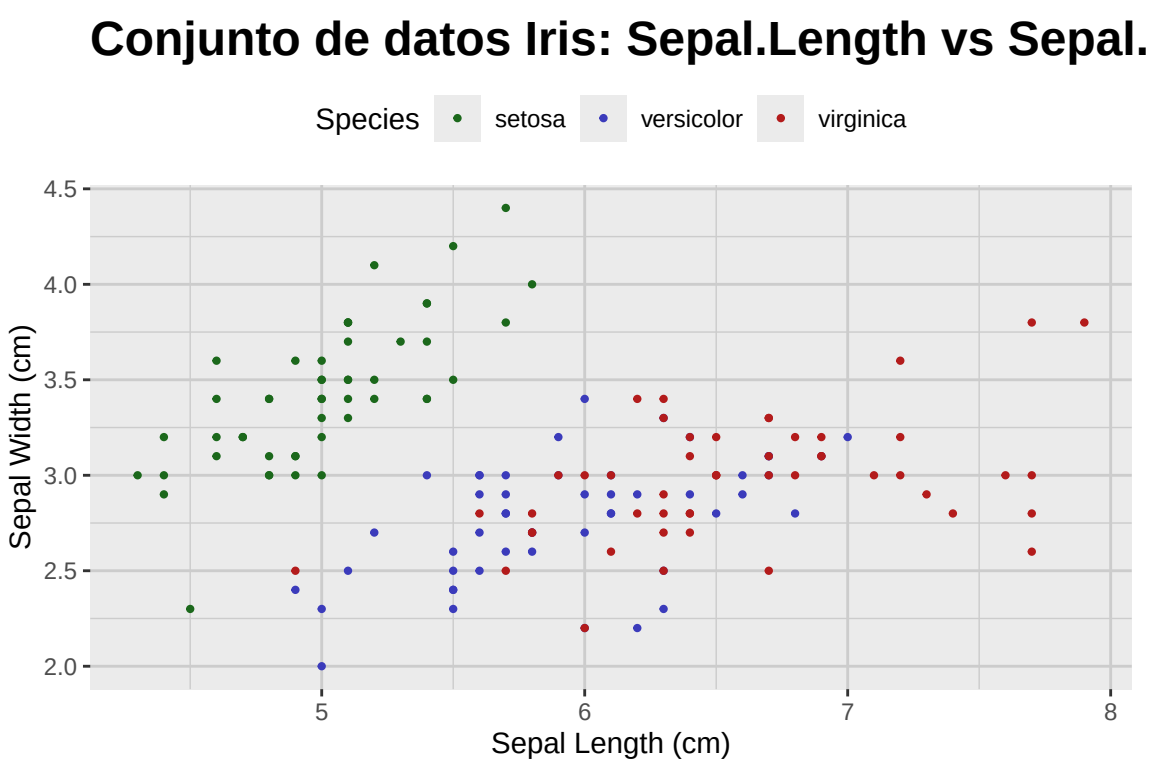


Figura 9.1: Diagrama de dispersión del conjunto de datos Iris, separado por especie y considerando únicamente las variables 'Sepal.Length' y 'Sepal.Width'.

Ahora consideraremos un dendrograma asociado a este conjunto y mostraremos cómo cortarlo a distintas alturas conduce a diferentes números de conglomerados:

```
# Use the built-in iris dataset only
iris_tbl <- as_tibble(iris)
```

```r
k_values <- c(1, 3, 4, 6)

# Cluster on the chosen 2D projection to match the plot
standardized_sepal_matrix <- iris_tbl %>%
  select(Sepal.Length, Sepal.Width) %>%
  scale()

distances <- stats::dist(standardized_sepal_matrix, method = "euclidean")
sepal_hclust <- stats::hclust(distances, method = "ward.D2")

sepal_tbl <- iris_tbl %>%
  select(Sepal.Length, Sepal.Width) %>%
  mutate(.row_id = row_number())

# Build long data for facets by k
long_pts <- lapply(k_values, function(k) {
  cluster <- stats::cutree(sepal_hclust, k = k)
  tibble::tibble(
    .row_id = seq_along(cluster),
    cluster = factor(as.integer(cluster)),
    k = factor(paste0("k = ", k), levels = paste0("k = ", k_values))
  )
})

long_pts <- bind_rows(long_pts)
sepal_cluster_tbl <- left_join(long_pts, sepal_tbl, by = ".row_id")

# Polygons for each (k, cluster)
sepal_poly_tbl <- cluster_polygons(
  sepal_cluster_tbl,
  Sepal.Length, Sepal.Width,
  k, cluster
)

sepal_cluster_plot <- sepal_cluster_tbl %>%
  ggplot(aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point(color = "black", alpha = 1, size = 0.8) +
  labs(
    title = "Clusters",
    x = "Sepal Length",
    y = "Sepal Width"
  ) +
  theme_krul() +
  facet_wrap(~k, ncol = 1, scales = "fixed")
```

```
sepal_cluster_plot <- sepal_cluster_plot +
  geom_path(
    data = sepal_poly_tbl,
    aes(
      x = Sepal.Length,
      y = Sepal.Width,
      group = interaction(k, cluster),
      color = cluster
    ),
    linewidth = 0.8,
    alpha = 0.5
  ) +
  c_scale_color(
    "C green",
    "C blue",
    "C red",
    "C magenta",
    "C yellow",
    "C cyan"
  ) +
  labs(color = "Cluster")

# Dendrogram geometry (replicated across facets)
sepal_dendro_data <- ggdendro::dendro_data(sepal_hclust, type = "rectangle")
k_fac <- factor(paste0("k = ", k_values), levels = paste0("k = ", k_values))

dendro_segments_faceted <- sepal_dendro_data$segments %>%
  mutate(.rep = 1) %>%
  tidyr::crossing(k = k_fac) %>%
  select(-.rep)

# One dashed cut line per k facet
dendro_cuts_tbl <- tibble::tibble(
  k = k_fac,
  y_cut = vapply(
    k_values,
    function(k) cut_dendrogram_height_for_k(sepal_hclust, k),
    numeric(1)
  )
)

dendro_faceted <- ggplot() +
  geom_segment(
    data = dendro_segments_faceted,
    aes(x = x, y = y, xend = xend, yend = yend),
```

```
    linewidth = 0.4
  ) +
  geom_hline(
    data = dendro_cuts_tbl,
    aes(yintercept = y_cut),
    linetype = "dashed",
    color = c_pal("C red")
  ) +
  labs(title = "Dendrogram cuts", x = NULL, y = "Height") +
  theme_krul() +
  theme(
    axis.text.x = element_blank(),
    axis.ticks.x = element_blank(),
    panel.grid = element_blank()
  ) +
  facet_wrap(~k, ncol = 1, scales = "fixed")

combined_plot <-  (dendro_faceted | sepal_cluster_plot) +
  plot_layout(widths = c(1, 2)) +
  plot_annotation(
    title = "Agrupamiento jerárquico del conjunto de datos Iris.",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

combined_plot
```

Agrupamiento jerárquico del conjunto de datos Iris.

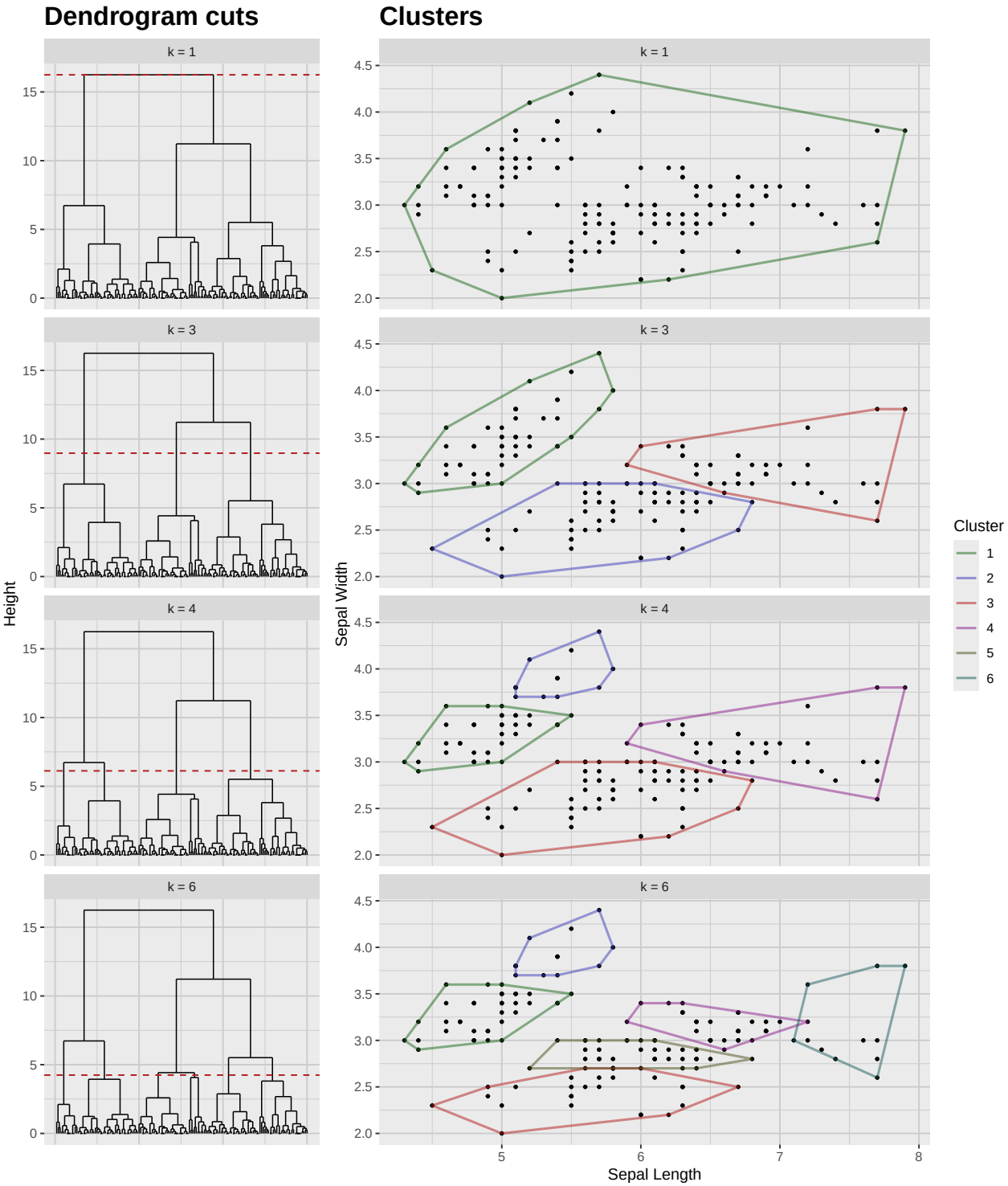


Figura 9.2: Agrupamiento jerárquico del conjunto de datos Iris considerando únicamente las variables 'Sepal.Length' y 'Sepal.Width'. Las líneas punteadas indican los puntos de corte para los números de clústeres dados (k = 1, 3, 4, 6).

9.2. Realización del agrupamiento jerárquico

Existen varios métodos para realizar agrupamiento jerárquico, pero los más comunes son:

- **Agrupamiento aglomerativo**: Enfoque ascendente en el que cada punto inicia como su propio conglomerado y, de manera iterativa, se fusionan parejas de conglomerados según su similitud hasta formar uno solo o hasta cumplir un criterio de parada.
- **Agrupamiento divisivo**: Enfoque descendente en el que todos los puntos empiezan en un único conglomerado y este se divide recursivamente en conglomerados más pequeños según su disimilitud hasta que cada punto queda solo o se cumple un criterio de parada.

Una vez elegido el método, es necesario definir una métrica de distancia para medir la similitud entre puntos. Algunas métricas comunes son:

- **Distancia euclidiana**: La métrica más utilizada; calcula la distancia en línea recta entre dos puntos en un espacio multidimensional.
- **Distancia de Mahalanobis**: Considera las correlaciones entre variables y es útil al tratar con datos multivariados.
- **Distancia de Canberra**: Sensible a pequeños cambios en los datos y útil cuando se trabaja con datos dispersos (escasos). La fórmula de la distancia de Canberra entre dos puntos p y q es:

  $$d(p,q) = \sum_{i=1}^{n} \frac{|p_i - q_i|}{|p_i| + |q_i|}$$

  donde n es el número de dimensiones, y p_i y q_i son las coordenadas de los puntos p y q, respectivamente.
- **Distancia Manhattan**: También conocida como norma L^1 o distancia de taxista; calcula la distancia entre dos puntos sumando las diferencias absolutas de sus coordenadas. Es particularmente útil en alta dimensión o cuando los datos tienen estructura en rejilla. La fórmula para dos puntos p y q es:

  $$d(p,q) = \sum_{i=1}^{n} |p_i - q_i|$$

- **Distancia de Minkowski**: Generaliza las distancias Euclidiana y Manhattan, permitiendo distintos valores del parámetro p para controlar el cálculo. Para dos puntos p y q se define como:

  $$d(p,q) = \left(\sum_{i=1}^{n} |p_i - q_i|^p \right)^{1/p}$$

  Cuando $p = 1$, corresponde a la distancia Manhattan; cuando $p = 2$, a la distancia Euclidiana.

Finalmente, debemos elegir un criterio de enlace para determinar cómo se calcula la distancia entre conglomerados. Algunos criterios comunes son:

- **Enlace simple (single)**: Define la distancia entre dos conglomerados como la distancia mínima entre cualquier par de puntos de cada conglomerado. Tiende a producir cadenas largas y es sensible al ruido y a valores atípicos.

- **Enlace completo (complete)**: Define la distancia entre dos conglomerados como la distancia máxima entre cualquier par de puntos de cada conglomerado. Tiende a crear conglomerados compactos y aproximadamente esféricos, y es menos sensible al ruido y a atípicos que el enlace simple.
- **Enlace promedio (average)**: Define la distancia entre dos conglomerados como la distancia promedio entre todos los pares de puntos de cada conglomerado. Proporciona un balance entre enlace simple y completo y es menos sensible al ruido y a atípicos.
- **Enlace de Ward**: Minimiza la varianza total intra-conglomerado al fusionar los conglomerados que ocasionan el menor incremento de varianza. Suele producir conglomerados compactos y aproximadamente esféricos, y es menos sensible al ruido y a atípicos que otros criterios de enlace.

Una vez elegidos el método, la métrica de distancia y el criterio de enlace, podemos realizar el agrupamiento jerárquico con la función `hclust` en R. Como se mostró en el ejemplo anterior, luego podemos visualizar los resultados con un dendrograma y cortarlo a una altura específica para determinar el número de conglomerados.

Capítulo 10

Agrupamiento no jerárquico

En el *agrupamiento no jerárquico* particionamos los datos en un número predeterminado de conglomerados. En este caso no obtenemos un dendrograma, sino un conjunto plano de conglomerados. Existen muchos métodos de agrupamiento no jerárquico, pero la mayoría pueden clasificarse como *métodos de particionado*, *métodos basados en densidad* o *métodos basados en modelos*.

10.1. Métodos de particionado

Estos métodos crean una partición de los datos en un conjunto de k conglomerados, donde k lo especifica el usuario. Se basan en optimizar alguna función objetivo que mide la calidad del agrupamiento. Los métodos de particionado más comunes son:

- **Agrupamiento k-means**: Este método busca particionar los datos en k conglomerados de forma que se minimice la suma de distancias cuadráticas entre cada punto y el centroide de su conglomerado asignado. El algoritmo asigna iterativamente los puntos al centroide más cercano y actualiza los centroides hasta converger.
- **Agrupamiento k-medoids**: Similar a k-means, pero en lugar de usar la media de los puntos como centroide, utiliza un punto real de los datos (el medoide) que minimiza la suma de distancias a los demás puntos del conglomerado. Esto hace a k-medoids más robusto al ruido y a valores atípicos.
- **CLARA (Clustering LARge Applications)**: Extensión de k-medoids diseñada para conjuntos grandes; muestrea un subconjunto de datos y aplica k-medoids a esa muestra.
- **PAM (Partitioning Around Medoids)**: Implementación específica de k-medoids que usa una estrategia voraz para encontrar los medoides y asignar puntos a conglomerados.

10.2. Métodos basados en densidad

Estos métodos identifican conglomerados con base en la densidad de puntos en el espacio de datos. Pueden encontrar conglomerados de formas arbitrarias y son robustos al ruido. Los métodos más comunes son:

- **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)**: Define conglomerados como zonas de alta densidad separadas por regiones de baja densidad. Requiere dos parámetros: el radio ϵ que define la vecindad de un punto y el número mínimo de puntos requerido para formar una región densa. Los puntos que no pertenecen a ningún conglomerado se consideran ruido.
- **OPTICS (Ordering Points To Identify the Clustering Structure)**: Extensión de DBSCAN que ordena los puntos según su densidad, permitiendo identificar conglomerados a distintos niveles de densidad.
- **Mean Shift**: Método que desplaza iterativamente cada punto hacia la media de los puntos en su vecindad, encontrando los modos de la distribución. Los conglomerados se forman alrededor de esos modos.

10.3. Métodos basados en modelos

Suponen que los datos provienen de una mezcla de distribuciones de probabilidad subyacentes y buscan identificar dichas distribuciones, asignando puntos a conglomerados según su verosimilitud de pertenencia. Los más comunes son:

- **Modelos de Mezclas Gaussianas (GMM)**: Modelan los datos como mezcla de varias distribuciones gaussianas, cada una representando un conglomerado. Los parámetros se estiman mediante el algoritmo de Expectación–Maximización (EM) y los puntos se asignan según probabilidades posteriores.
- **Agrupamiento Bayesiano**: Utiliza inferencia bayesiana para estimar parámetros de las distribuciones subyacentes y asignar puntos a conglomerados. Puede incorporar conocimiento previo y manejar la incertidumbre del proceso de agrupamiento.
- **Análisis de Clases Latentes (LCA)**: Usado para datos categóricos; asume que los datos provienen de una mezcla de clases latentes. Estima parámetros de las clases y asigna puntos según probabilidades posteriores.

10.4. Elección del método adecuado

La elección del método de agrupamiento no jerárquico depende de las características de los datos y de los objetivos del análisis. Algunas pautas:

- Si los datos son continuos y se espera que los conglomerados sean aproximadamente esféricos y de tamaño similar, k-means o k-medoids pueden ser apropiados.
- Si hay ruido o valores atípicos, o se esperan conglomerados de diferentes formas y tamaños, métodos basados en densidad como DBSCAN u OPTICS pueden ser más adecuados.
- Si se cree que los datos provienen de una mezcla de distribuciones subyacentes, métodos basados en modelos como GMM o el agrupamiento bayesiano pueden ser la mejor opción.
- Si los datos son categóricos, el Análisis de Clases Latentes (LCA) puede ser el método más apropiado.

- Si el conjunto es grande, considere métodos como CLARA, diseñados para manejar grandes volúmenes eficientemente.

10.5. Evaluación de los resultados de agrupamiento

Evaluar la calidad del agrupamiento es crucial para confirmar que el método elegido capturó la estructura subyacente. Algunas técnicas comunes son:

- **Validación interna**: Usa métricas basadas en los propios datos, sin referencia externa. Entre las más comunes:
  - *Índice de Silueta*: Mide qué tan similar es un punto a su propio conglomerado frente a otros. Valores altos indican conglomerados bien definidos.
  - *Índice de Davies–Bouldin*: Evalúa la razón de similitud promedio de cada conglomerado con su más similar. Valores bajos indican mejor agrupamiento.
  - *Índice de Calinski–Harabasz*: Mide la razón varianza entre–conglomerados vs. intra–conglomerados. Valores altos indican conglomerados mejor definidos.
- **Validación externa**: Compara contra etiquetas conocidas o verdad-terreno, si existen. Métricas comunes:
  - *Índice de Rand Ajustado (ARI)*: Mide la similitud con la verdad-terreno, ajustando por azar. Valores altos indican mayor acuerdo.
  - *Información Mutua Normalizada (NMI)*: Cuantifica la información compartida entre el agrupamiento y la verdad-terreno. Valores altos indican mayor acuerdo.
- **Análisis de estabilidad**: Evalúa la robustez aplicando el algoritmo a subconjuntos o añadiendo ruido. Resultados consistentes entre corridas indican estabilidad.
- **Inspección visual**: Diagramas de dispersión, mapas de calor o dendrogramas (si aplica) ayudan a entender la estructura y a detectar posibles problemas.

10.6. Ejemplo en R

Mostramos ahora cómo implementar distintos métodos de agrupamiento no jerárquico en R usando el conjunto `iris`. Iniciamos cargando las librerías necesarias.

```
library(cluster)
library(dbscan)
library(mclust)
library(krulRutils)
```

Luego preparamos los datos y mostramos una figura comparando los resultados de distintos métodos de agrupamiento.

```
set.seed(123)

# Usar únicamente Sepal.Width y Sepal.Length
```

```r
iris_tbl <- as_tibble(iris) %>%
  select(Sepal.Width, Sepal.Length)

iris_matrix <- as.matrix(iris_tbl)

# k-means (particionado)
iris_kmeans <- kmeans(iris_matrix, centers = 3, nstart = 50)$cluster

# PAM (k-medoids, particionado)
iris_pam <- pam(iris_matrix, k = 3)$clustering

# GMM (basado en modelos, mezclas gaussianas)
gmm <- Mclust(iris_matrix, G = 3)$classification


db_raw <- choose_dbscan(iris_matrix, min_pts = 5)
db <- assign_noise_to_nearest(iris_matrix, db_raw$cluster)

# Unir resultados para graficación facetada
labs <- list(
  `k-means` = iris_kmeans,
  PAM = iris_pam,
  GMM = gmm,
  DBSCAN = db
)

plot_df <- bind_rows(lapply(names(labs), function(m) {
  tibble(
    sepal_width = iris_tbl$Sepal.Width,
    sepal_length = iris_tbl$Sepal.Length,
    method = m,
    cluster = factor(labs[[m]])
  )
}))

ggplot(plot_df, aes(sepal_width, sepal_length, color = cluster)) +
  geom_point(size = 1.3, alpha = 1) +
  c_scale_color("C red", "C green", "C blue") +
  facet_wrap(~ method, ncol = 2) +
  labs(title = "Agrupamiento de Iris (sépalos)",
       x = "Ancho de sépalo", y = "Largo de sépalo") +
  theme_krul()
```

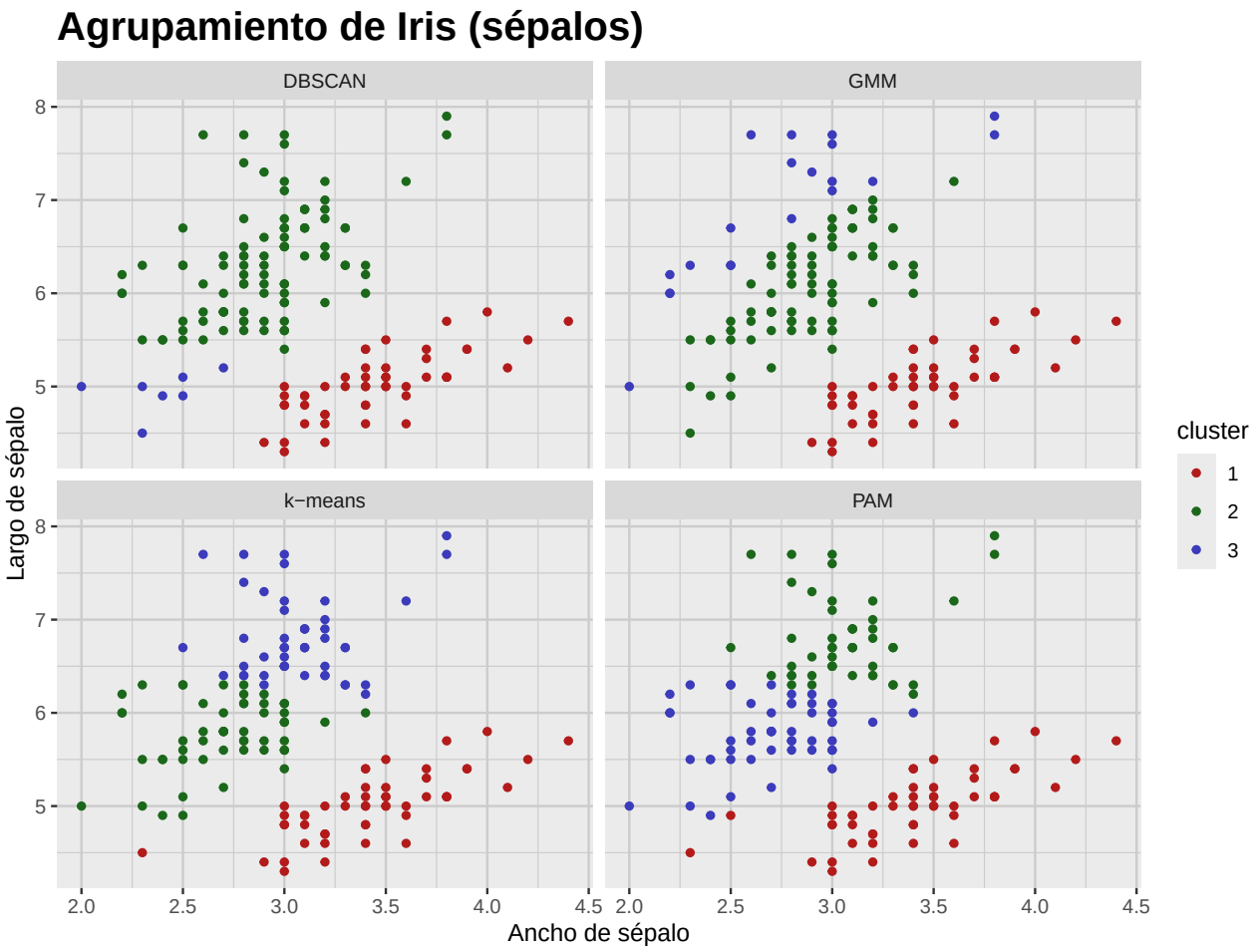


Figura 10.1: Resultados de agrupamiento en el conjunto iris usando únicamente Sepal.Width y Sepal.Length. Nótese la diferencia de formas de los conglomerados según el método.

IV. Análisis de conglomerados